

Source-Guided Roadmap for Reimplementing and Machine-Checking the HAETAE Provable-Security Proof

Technical report for the HAETAE EasyCrypt proof surface

May 2026

Abstract

This report is a repository-guided roadmap for reimplementing the HAETAE provable-security proof surface. It is written for a reader who has not seen the repository before and wants to understand the pen-and-paper proof boundary, the EasyCrypt file architecture, and the verification workflow. It is not, by itself, a literal replacement for the `.ec` source files: several blueprint snippets are schematic, and a faithful machine-checked reimplementation still requires the existing EasyCrypt files, generated declaration inventory, and EasyCrypt libraries as companion artifacts. A claim of report-only machine-checkable reproduction should therefore be rejected unless this roadmap is expanded into a complete source-and-proof archive. The report identifies the exact theorem being checked, separates checked facts from external assumptions, gives the source-file dependency order, lists the mathematical object families that must be introduced, and records the verification commands that certify the proof gate. The document-only reproducibility question has a negative answer: the report can explain the checked formal proof boundary, but it cannot by itself establish HAETAE algorithm security or reproduce the EasyCrypt verification. The checked theorem is a conditional random-oracle-model statement for the formal EasyCrypt model. It is not a standard-model SHAKE theorem, not an implementation-correctness theorem, and not a non-vacuous concrete security instantiation until the currently oversized rejection-sampling antecedent is repaired. When full `.ec` proof scripts are provided as companion artifacts, this report also states the documentation standard needed for a mathematician or cryptologist to understand the mathematical content of those scripts without relying on unstated repository lore. This version records that standard and an initial coverage ledger; it should not be described as complete script-companion documentation until every required coverage row is discharged.

Contents

Chapter 1

Scope and End Product

1.1 What the reader should be able to reproduce

After following this report together with the companion EasyCrypt sources, a reader should be able to recreate a proof surface with the following properties.

1. The EasyCrypt files compile as a manifest-driven proof gate.
2. The generic random-oracle signature game is instantiated by a formal HAETAE scheme module.
3. Correctness is proved for the formal model.
4. The EUF-CMA game is related to a sequence of simulated, transcript, NMA, sampling, and extraction games.
5. The final checked security implication has the same dependency boundary as the current artifact: it is conditional on explicit hop/reduction premises and abstract MLWE/Module-SIS hardness bounds.
6. The counted-ROM loss ledger is normalized to the concrete expression

$$\frac{37}{2^{58}}$$

for the fixed wrapper constants $q_H = q_S = 2$.

7. The proof explicitly records the current vacuity issue: `rejection_sampling_loss_term` ≥ 1 , and in fact the min-entropy part alone is 12, so a final premise that adds this term to a probability cannot be treated as a useful concrete-security premise.

1.2 Sufficiency status

This report should be read as a technical map and acceptance checklist, not as a complete standalone source of the proof. It is sufficient for understanding the shape and limits of the checked theorem, for navigating the proof files in the right order, and for checking that a reconstructed development has the expected manifest, theorem statement, and loss ledger. It is not sufficient, on its own, to type all EasyCrypt code from a blank file and recover the current proof.

The reason is structural rather than cosmetic. A machine-checked EasyCrypt development depends on exact declarations, module restriction sets, local helper lemmas, proof tactics, import paths, and library behavior. This report records the required proof architecture and representative source forms, but it does not inline all 22046 lines of checked EasyCrypt. Whenever a snippet contains `...`, it is a blueprint placeholder, not compilable code. A reader attempting a literal reimplementation must replace every such placeholder by the corresponding complete definition or proof from the companion artifacts.

1.3 Document-only verdict

The question “Can a reader understand the provable security of the HAETAE digital signature algorithm and reproduce the machine-checked verification based solely on this technical document?” has the following answer.

No. The document alone is enough to orient a reader to the checked formal-model theorem, but it is not enough to certify the HAETAE algorithm as a deployed digital signature scheme and it is not enough to recreate the machine-checked EasyCrypt artifact from a blank workspace.

There are three distinct claims, and only the weakest one is supported by the report alone.

1. **Understanding the checked theorem boundary: yes.** The report states the final theorem shape, the random-oracle setting, the fixed counted-ROM ledger, the declared hardness premises, and the rejection-sampling vacuity warning.
2. **Understanding full HAETAE algorithm security: no.** The report does not supply all bridges from the formal EasyCrypt model to the specification, SHAKE instantiation, concrete lattice estimates, implementation behavior, or side-channel properties.
3. **Reproducing machine-checked verification from the document alone: no.** The report omits literal EasyCrypt source, complete proof scripts, exact module restrictions, full helper-lemma bodies, and a full import/library contract. Those are exactly the objects that the checker consumes.

The strongest truthful claim is therefore conditional: with the companion source tree and libraries, the report guides a reader through the proof surface and the verification gate. Without those artifacts, it is an audit roadmap, not a standalone proof.

1.4 Can this document stand alone?

The criticism that motivated this revision is correct: the report, by itself, cannot be both a readable roadmap and a complete machine-checkable source archive. The following table states exactly what can and cannot be recovered from the document alone.

Goal	From this report alone?	Reason
------	-------------------------	--------

Understand the final theorem boundary	Yes	The theorem shape, ROM ledger, declared hardness premises, and vacuity warning are stated explicitly.
Reconstruct the pen-and-paper proof outline	Partly	The hybrid order and loss accounting are given, but each step still has to be checked against the source lemmas that implement it.
Type a fresh EasyCrypt development from a blank directory	No	The report does not inline every declaration, helper lemma, proof script, import, and module restriction set.
Machine-check equivalence to the current artifact	No	Equivalence requires the companion source tree or an independently expanded document containing literal source and proof scripts.
Conclude end-to-end HAETAETAE implementation security	No	The checked theorem is formal-model, random-oracle-model, fixed-ledger, and conditional; byte-level, constant-time, SHAKE-instantiation, and concrete hardness claims are outside the checked result.

Consequently, any phrase such as “reproduce the machine-checked verification based solely on this report” must be read as false for the present artifact. The accurate claim is narrower: a reader can use the report to understand the proof boundary and to reproduce the verification when the report is paired with the listed source artifacts.

1.5 Formal-model understanding versus algorithm security

The phrase “the provable security of HAETAETAE” is ambiguous. In this report it means the checked EasyCrypt theorem about the formal HAETAETAE model, not a full security certification of every object that a cryptographer or implementer might call the HAETAETAE digital signature algorithm. A reader can learn the formal proof boundary from this document: the game being bounded, the random-oracle abstraction, the declared hardness games, the counted-ROM loss ledger, and the final vacuity warning. That is a real but narrower achievement than understanding an end-to-end algorithm-security theorem.

To turn formal-model understanding into algorithm-security understanding, the following bridges would also have to be supplied and checked or separately audited.

- A specification-to-model bridge showing that the EasyCrypt operations faithfully encode the HAETAETAE specification being claimed.
- A hash-instantiation argument explaining why the random-oracle theorem is applicable to the concrete SHAKE-based algorithm, or an explicit decision not to make that claim.
- Concrete parameter analysis for the MLWE and Module-SIS assumptions, including any reduction losses not internalized by the EasyCrypt theorem.
- Implementation-correctness and side-channel arguments for concrete code, if the claim is about deployed signing and verification programs.
- A repaired or reinterpreted rejection-sampling premise if the theorem is to be cited as a non-vacuous concrete bound.

Because those bridges are not all present as checked theorems, the document should be used to understand the formal verification surface, not as sole evidence that the deployed HAETAE algorithm has been proved secure.

1.6 Companion artifacts required

A faithful reimplementatation therefore requires at least the following companion materials.

- The manifest files under `provable-security/easycrypt/`, or an equivalent complete source tree.
- The generic EasyCrypt libraries imported by those files.
- The `kyber-security` include path, unless every imported dependency from that path is audited and replaced.
- The declaration inventory generated by `dissertation/etc/count-easycrypt-declarations.sh`.
- The HAETAE specification PDF and model/specification audit material if the formal EasyCrypt model is to be interpreted as the February 2026 HAETAE specification.

Without those artifacts, the report can support a mathematical review of the proof boundary, but it cannot certify that a newly written EasyCrypt development is definitionally identical to the checked one.

1.7 Source-truth contract

A faithful reproduction must declare its source of truth before any theorem is accepted.

1. In *audit mode*, the source of truth is the existing `provable-security/easycrypt/` tree plus the manifest. The report is a guide for reading, checking, and rebuilding that tree.
2. In *literal standalone mode*, the source of truth would have to be a larger document that contains every active EasyCrypt file in full, every proof script, every module restriction set, and every imported helper dependency that affects type checking or proof checking.
3. In *independent reimplementatation mode*, the new development may choose different local names or proof scripts, but it must provide a traceability matrix from each manifest file, declaration family, theorem, and loss term in this report to a checked EasyCrypt object in the new tree.

The current report intentionally supports audit mode and independent reimplementatation mode with companion sources. It does not satisfy literal standalone mode. Treating it as if it did would erase exactly the facts that EasyCrypt checks: definitional equality, dependency resolution, tactic success, and module-separation restrictions.

1.8 Script companion mode

There is a fourth, weaker-than-standalone but stronger-than-roadmap use case: the `.ec` proof scripts are provided alongside this document, and the document is responsible for making their mathematical content understandable to mathematicians and cryptologists who have not previously studied the project. This is *script companion mode*.

Script companion mode does not mean that the prose replaces EasyCrypt. The scripts remain the machine-checkable artifact. It means that a reader should be able to open a script, look up every important declaration and proof block in this report, and understand what mathematical claim is being checked, why the claim is needed, which assumptions it uses, and what each proof step contributes. The reader may still need EasyCrypt to check the proof, but should not need unstated repository lore to understand the proof.

In script companion mode, the documentation must provide the following for each manifest file.

Documentation item	Required content
File role	The mathematical purpose of the file, its upstream dependencies, and the later files that rely on it.
Declaration dictionary	Every type, operation, module type, module, predicate, event, and distribution that is not standard EasyCrypt vocabulary, with a mathematical reading.
State interpretation	For every game or oracle module, the meaning of each state variable, log, counter, bad flag, and sampled value.
Restriction explanation	For every security-critical <code>declaremodule</code> restriction set, the information-flow reason for the exclusions.
Lemma ledger	A table of important lemmas and theorems giving the EasyCrypt name, the informal mathematical statement, dependencies, and the later result that consumes the lemma.
Proof-script commentary	A block-by-block explanation of the tactic script: which invariant is opened, which equivalence or probability rule is applied, which algebraic rewrite is used, and which side condition is delegated to <code>smt</code> .
Assumption boundary	A statement of whether a fact is checked locally, imported from a library, declared as a hardness interface, or left as an external cryptographic assumption.

This requirement is intentionally stronger than merely listing theorem names. For example, a line such as `byphoare` or `byequiv` must be explained by the game transformation it establishes; a call to `smt` must be associated with the arithmetic, set-theoretic, or losslessness side condition it discharges; and a module restriction list must be explained as an adversarial-separation condition rather than as syntax.

1.9 Per-lemma explanation template

Every nontrivial EasyCrypt lemma used by the final theorem should have an entry with the following fields.

1. **EasyCrypt anchor:** file name, lemma name, and whether the proof is local, imported, or a direct composition of earlier lemmas.

2. **Mathematical statement:** a prose or displayed-math rendering that does not require EasyCrypt syntax to parse.
3. **Objects involved:** the games, modules, distributions, events, state variables, and loss terms appearing in the statement.
4. **Assumptions:** module restrictions, losslessness facts, positivity facts, hardness premises, or previous hop lemmas consumed by the proof.
5. **Proof idea:** the mathematical argument before tactic details.
6. **Script reading:** an explanation of the proof script grouped into meaningful blocks, including what is proved by rewriting, by probabilistic program logic, by equivalence reasoning, and by SMT.
7. **Security role:** the place of the lemma in the global reduction: exact hop, bad-event bound, sampling-distance bound, extraction step, hardness adapter, or arithmetic normalization.
8. **Failure mode:** what would go wrong if the lemma were misstated, weakened, or used outside its hypotheses.

The goal is not to duplicate every tactic token in prose. The goal is that a mathematician or cryptologist can read the `.ec` script and this entry together and recover the proof argument without asking what an identifier, game state, or proof step was intended to mean.

1.10 Minimum EasyCrypt vocabulary for readers

When proof scripts are supplied, this report must also define the EasyCrypt vocabulary used by the scripts at the level needed to read the proof. At minimum, it must explain:

- `op`, `pred`, `type`, `moduletype`, `module`, and `declaremodule`;
- `proc`, program state, sampling, procedure calls, and return values;
- probability expressions `Pr[...@&m:...]`;
- Hoare, probabilistic Hoare, and equivalence judgments;
- `byphoare`, `byequiv`, `islossless`, `lossless`, `rewrite`, `case`, `move`, `apply`, and `smt`;
- the meaning of module restriction sets such as `-H`, `-A`, `-HAETAE`;
- the difference between a checked lemma, a declared module interface, and an external cryptographic assumption.

These explanations should be tied to the HAETAE scripts, not presented as a generic EasyCrypt tutorial. Each tactic or syntactic form should be introduced where it first matters for a HAETAE proof step.

1.11 Current script-companion coverage status

The criticism that this document still only states the script-companion standard is correct. The current report contains a useful roadmap and several mathematical readings, but it is not yet a complete commentary for every security-relevant block of the supplied `.ec` scripts. The table below is therefore part of the technical specification: it records what remains before the document can honestly claim that mathematicians and cryptologists can understand the scripts’ proof content solely through this report plus the scripts themselves.

Coverage item	Current status	Required to close
File roles for manifest files	Partial	The file-by-file blueprint gives the main role of each file, but it must be cross-checked against the actual <code>.ec</code> scripts and expanded with upstream and downstream dependencies.
Declaration dictionary	Partial	Every nonstandard type, operation, predicate, distribution, module type, module, and event in the scripts must have a mathematical reading and script anchor.
State interpretation	Partial	Game modules, signing oracles, random-oracle modules, transcript simulators, and samplers need complete state-variable tables.
Restriction explanations	Partial	Representative restriction sets are shown, but every security-critical <code>declaremodule</code> restriction must be explained by its information-flow purpose.
Lemma ledger	Partial	The report lists important lemma names, but each nontrivial lemma used in the final theorem needs a full entry using the per-lemma template.
Proof-script commentary	Not complete	Tactic scripts must be grouped into mathematical proof blocks explaining <code>byphoare</code> , <code>byequiv</code> , rewrites, losslessness arguments, and <code>smt</code> side conditions.
EasyCrypt vocabulary tied to HAETAE	Partial	The report states the vocabulary that must be explained, but the explanations must be integrated at the first HAETAE proof point where each construct matters.
Assumption boundary	Partial	The major boundaries are recorded, but each imported library fact, declared hardness interface, and external cryptographic assumption must be labelled in the lemma ledger.

Until the rows above are complete, the report should be cited as a script-companion specification and partial reading guide, not as a completed script-companion document. The completion test is concrete: for every security-relevant declaration and proof block in the supplied `.ec` scripts, a reader must be able to find a nearby entry in this report explaining the mathematical statement, the proof idea, the tactic role, and the place of the block in the global security reduction.

1.12 Coverage ledger template

A complete script-companion version should maintain a ledger with one row per security-relevant script block. The ledger may be embedded in this report or generated as an appendix, but it must be reviewable as prose. Each row should have the following shape.

Field	Meaning
Script anchor	File name, line or lemma name, and whether the block is a definition, module, declaration, lemma, proof, or tactic sub-block.
Mathematical reading	The human mathematical object or claim represented by the script block.
Dependencies	Prior declarations, imported theories, assumptions, module restrictions, or lemmas needed to read the block.
Proof explanation	For proof blocks, the argument split into exact equivalence, probabilistic bound, invariant preservation, algebraic rewrite, losslessness proof, extraction step, or arithmetic normalization.
Checker role	What EasyCrypt verifies at this block: type checking, losslessness, Hoare judgment, equivalence judgment, probability inequality, or SMT side condition.
Security role	Why the block matters for the HAETAE proof: correctness, hybrid hop, random-oracle programming, transcript validity, rejection bound, hardness adapter, or final theorem composition.
Status	Complete only when the prose is sufficient for a cryptographer to read the script block without guessing the intended mathematics.

This ledger is the mechanism that turns the current roadmap into a genuine script companion. A build of the $\text{\LaTeX}$ document is not enough; the ledger must also be substantively complete against the supplied scripts.

1.13 Initial filled coverage entries

The entries below are the first concrete script-companion rows. They are included to demonstrate the required level of explanation and to prevent the coverage ledger from remaining a purely aspirational template. The status is “partial” unless the row covers every dependent declaration and proof block in the surrounding script region.

Script anchor	Mathematical reading	Current explanation and remaining work
---------------	----------------------	--

Sig_ROM.ec, types and freshness operations	Generic signature-game vocabulary. The abstract types <code>pkey</code> , <code>skey</code> , <code>message</code> , <code>context</code> , <code>signature</code> , <code>ro_query</code> , and <code>ro_output</code> define the carrier sets of a random-oracle signature experiment. The predicates <code>fresh_msg</code> and <code>fresh_sig</code> express whether a forgery target was absent from the signing log.	The report explains the EUF-CMA/NMA game boundary and freshness condition. To close the row, add a declaration dictionary entry for every generic type, the signed-query variant used by SUF-CMA, and the exact list operations used in the freshness lemmas.
Sig_ROM.ec, EUF_CMA and UF_NMA modules	EUF_CMA is the adaptive chosen-message experiment with a signing oracle that records queried message-context pairs. UF_NMA is the no-message experiment in which the adversary only has random-oracle access.	The mathematical game notation table identifies both games. To close the row, add state-variable tables for <code>EUF_CMA.sk</code> , <code>EUF_CMA.queries</code> , and the local variables <code>pk</code> , <code>sk</code> , <code>m</code> , <code>ctx</code> , <code>sig</code> , and <code>ok</code> .
Sig_ROM.ec, nma_as_cma_exact	The NMA adversary adapter is distribution-preserving when embedded into the CMA game with an unused signing oracle: the two success probabilities are equal.	The proof is an EasyCrypt equivalence proof using <code>byequiv</code> , inlining the adapter and simulating matching calls to the adversary, scheme, and oracle. To close the row, add a block-by-block explanation of the <code>call</code> , <code>wp</code> , <code>sim</code> , and final <code>auto</code> steps.
HAETAE_Reductions.ec, hardness and loss operations	The operations <code>mlwe_hardness_term</code> , <code>module_sis_hardness_term</code> , and <code>bimodal_to_msis_reduction_loss_term</code> are abstract real-valued assumption/loss slots. The counted-ROM terms encode collision, prequery, and min-entropy losses for $q_H = q_S = 2$ and challenge support 2^{58} .	The report gives the counted-ROM arithmetic and the $37/2^{58}$ calculation. To close the row, add dictionary entries for every real-valued operation in the file and identify which are abstract assumptions, definitions, or checked arithmetic normalizations.

HAETAEC_Reductions.ec, haetae_euf_ counted_rom_ bound_groupedE	This lemma rewrites the final counted-ROM bound into the sum of MLWE hardness, Module-SIS hardness, bimodal-to-MSIS loss, and counted random-oracle programming loss.	The proof is pure real algebra: unfold the grouped definitions and discharge the equality by the ring tactic. To close the row, add similar explanation entries for the nonnegativity, positivity, subunit, concrete-cardinality, and vacuity lemmas in the same file.
HAETAEC_HopGames.ec, section ROMInternalTranscriptSigningAttemptExactHyperballPaperSimNMA	This region relates an NMA game using a random-oracle to one using the exact-hyperball sampler, while tracking rejection and sampler-prequery losses.	The report describes the layer as NMA and transcript game lifting. To close the row, add a commentary for the paper-sample lifting lemma, the budgeted lifting lemmas, the clean-conditioned lifting lemma, and the bad prequery bound that is imported into the inequality chain.
HAETAEC_HopGames.ec, rom_internal_ budgeted_nma_ ro_signing_ attempt_ro_ exact_hyperball_ loss_from_clean_ lifting	The lemma decomposes the success event into a clean branch and a sampler prequery bad event. It bounds the clean branch by the exact-hyperball game plus rejection loss, and charges the bad branch to a counted-ROM prequery term.	The script uses union/subdistribution inequalities, a clean-lifting premise, and the lemma <code>budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound</code> . To close the row, explain each <code>ler_trans</code> , <code>Pr[mu_sub]</code> , <code>Pr[mu_or]</code> , <code>ler_add</code> , and final bad-event-bound application.
HAETAEC_Security.ec, section MechanizedROMTranscriptConstructionML	This section is the final composition lane. It proves the final EUF-CMA view by structural, public-simulation, paper-simulation, rejection-simulation, exact-hyperball, and random-oracle exact-hyperball NMA views before applying MLWE and bimodal/Module-SIS assumptions.	The report states the final theorem shape and representative restriction sets. To close the row, every lemma in this section must be entered in the ledger, including its predecessor theorem, hop premise, hardness premise, arithmetic bound, and whether the proof is a rewrite, transitivity step, or final composition.

HAETAE_ Security.ec, declaremoduleH, A, Samp, Bmlwe, Bbimodal in the final composition section	The restrictions enforce adversarial separation: the random oracle and adversary cannot call hidden game internals, the sampler cannot inspect the adversary or oracle beyond its interface, and the hardness solvers remain abstract modules.	The report gives a representative restriction block and explains why restriction sets are proof obligations. To close the row, every excluded module name in the final block must be mapped to the hidden state or procedure it protects.
--	--	---

1.14 Script companion coverage for Sig ROM

The generic signature-game file is small enough that its script-companion coverage can be closed in this report. The file is an abstract theory: it does not know HAETAE-specific algebra, rejection sampling, or lattice assumptions. It defines the reusable game vocabulary that later files instantiate with the HAETAE scheme and random oracle.

Script item	Mathematical reading
pkey, skey, message, context, signature	Abstract carrier sets for public keys, secret keys, signed messages, auxiliary contexts, and signatures. No algebraic structure is assumed here.
ro_query, ro_output, dro_output	The input and output domains of the random oracle, together with a lossless output distribution used by the cloned full-random-oracle library.
query, signed_query	The signing-log domains. A query records a message-context pair; a signed_query records the same pair together with the signature returned by the signing oracle.
fresh_msg	EUFCMA freshness: the message-context pair in the alleged forgery was not previously submitted to the signing oracle.
fresh_sig	Strong unforgeability freshness: the full message-context-signature triple was not previously returned by the signing oracle.
Oracle and POracle	Oracle exposes random-oracle initialization and query access; POracle exposes query access only. Adversaries receive POracle so they cannot reset the oracle.
Scheme(0)	The signature-scheme interface parameterized by public oracle access: key generation, signing, and verification.
CORR_ADV(0)	The correctness-game adversary, which chooses the message and context after seeing both public and secret keys.
Adversary(H, 0)	The EUFCMA adversary. Its procedure may call the random oracle and the signing oracle.
NMA_Adversary(H)	The no-message adversary. It may call the random oracle but has no signing oracle.
NMA_As_CMA	An adapter that lets an NMA adversary inhabit the CMA adversary interface while ignoring the signing oracle.

The game modules have the following state interpretation.

Game state or local	Meaning
<code>Correctness.main</code>	Initializes the random oracle, samples a key pair, asks the correctness adversary for a message and context, signs them, verifies the signature, and returns failure $\neg ok$. Thus a zero probability result is perfect correctness for the formal model.
<code>EUFCMA.sk</code>	The secret key stored by the EUF-CMA game so the internal signing oracle can answer adversarial signing queries.
<code>EUFCMA.queries</code>	The list of message-context pairs queried to the signing oracle. It is reset to the empty list before the adversary runs.
<code>EUFCMA.O.sign</code>	The signing oracle: it signs with <code>sk</code> , logs (m, ctx) , and returns the signature.
<code>EUFCMA.main</code>	The adaptive unforgeability experiment. It accepts iff verification succeeds and <code>fresh_msg</code> holds for the forgery message-context pair.
<code>UFNMA.main</code>	The no-message experiment. It samples keys, gives only random-oracle access to the adversary, and accepts iff verification succeeds.
<code>SUFCMA.main</code>	The strong unforgeability analogue of EUF-CMA. It logs returned signatures and requires freshness of the full signed query. It is defined for completeness of the generic interface, although the HAETAE top-level theorem uses EUF-CMA/NMA.

The two freshness lemmas are proof-script sanity checks. `fresh_msg_nil` and `fresh_sig_nil` unfold the corresponding freshness predicate and solve membership in the empty list by simplification. No cryptographic assumption is used.

The lemma `nma_as_cma_exact` is the main proof block in this file. Its mathematical statement is

$$\Pr[\text{EUFCMA}(H, S, \text{NMA_As_CMA}(A))] = \Pr[\text{UFNMA}(H, S, A)].$$

The proof uses `byequiv` to compare two programs in lockstep. The pre- and postcondition $=\{\text{globH}, \text{globS}, \text{globA}\} ==> \{\text{res}\}$ says that if the oracle, scheme, and adversary global states agree initially, then the two experiments return the same Boolean result. The `inline` step expands the adapter so the proof can see that it simply calls the NMA adversary. Each `call...bysim` aligns matching calls to the adversary, scheme, and random oracle. The final `auto=>/>` discharges the remaining first-order obligations, including that the unused signing oracle made no logged query.

This closes the script-companion coverage for `Sig_ROM.ec` at the level needed for the HAETAE proof: every declaration, game state, and proof block in the generic interface has a mathematical reading. Future additions would only be needed if later proofs begin using `SUFCMA` in a security theorem.

1.15 Script companion coverage for HAETAE Reductions

`HAETAE_Reductions.ec` is the arithmetic ledger for the proof. It does not define a cryptographic game. Instead, it defines the real-valued loss and hardness terms that appear in the final theorem, then proves the algebraic and order facts needed to compose those terms.

Operation family	Mathematical reading
mlwe_hardness_term	Abstract upper bound for the success probability of the MLWE reduction module. It is not estimated in this file.
module_sis_hardness_term	Abstract upper bound for the Module-SIS reduction module. It is also not a concrete bit-security estimate.
bimodal_to_msis_reduction_loss_term	Loss slot for converting the bimodal self-target relation to Module-SIS. The current checked value is 0.
bimodal_selftarget_msis_hardness_term	The grouped bimodal hardness contribution: $\text{module_sis_hardness_term} + \text{bimodal_to_msis_reduction_loss_term}$.
signature_query_count, hash_query_count	The fixed counted-ROM wrapper constants $q_S = q_H = 2$. These are not symbolic adversarial query bounds.
challenge_support_bits_lower_bound, challenge_support_cardinality_lower_bound	The challenge-support lower bound 58 bits and 2^{58} elements, encoded as the concrete real 288230376151711744.
counted_rom_collision_term	Collision loss $(q_H + q_S + 1)^2/N = 25/2^{58}$.
counted_rom_prequery_term	Challenge prequery loss $q_S(q_H + 1)/N = 6/2^{58}$.
counted_rom_min_entropy_term	Counted min-entropy loss $q_S(q_H + 1)/N = 6/2^{58}$.
counted_rom_programming_loss_term	The counted-ROM sum $25/2^{58} + 6/2^{58} + 6/2^{58} = 37/2^{58}$.
fs_with_aborts_* and rejection_sampling_loss_term	The older paper-style Fiat-Shamir-with-aborts/rejection ledger. Its min-entropy component rewrites to 12, so it is tracked as a vacuity warning rather than a useful concrete loss.
haetae_euf_bound	Grouped legacy bound using the rejection-sampling ledger.
haetae_euf_counted_rom_bound	The final counted-ROM bound used by the top-level theorem.

The proof families in this file should be read as follows.

Lemma family	Proof-script reading
haetae_euf_boundE, haetae_euf_bound_groupedE, haetae_euf_bound_rejection_groupedE, haetae_euf_counted_rom_bound_groupedE	These lemmas unfold definitions and regroup real sums. The script pattern is <code>rewrite/definition...;ring;</code> mathematically, it proves only associativity, commutativity, and unfolding of named loss groups.

Nonnegativity lemmas	Lemmas such as <code>rom_hash_query_budget_nonnegative</code> , <code>counted_rom_collision_term_nonnegative</code> , and <code>rejection_sampling_loss_term_nonnegative</code> prove that probabilities and loss terms can be added or used in transitivity arguments without reversing inequalities. Tactics such as <code>addr_ge0</code> , <code>mulr_ge0</code> , and <code>divr_ge0</code> correspond to elementary real-order closure.
Support positivity lemmas	<code>challenge_support_cardinality_lower_bound_positive</code> and its nonnegative corollary justify divisions by 2^{58} and the use of <code>ltr_pdivr_mulr</code> .
Concrete rewrites	<code>counted_rom_collision_termE</code> , <code>counted_rom_prequery_termE</code> , <code>counted_rom_min_entropy_termE</code> , and <code>counted_rom_programming_loss_termE</code> compute the numerators 25, 6, 6, and 37. The scripts unfold query counts and use <code>ring</code> .
Subunit and dominance lemmas	<code>counted_rom_programming_loss_term_subunit</code> , <code>counted_rom_prequery_reprogramming_term_subunit</code> , and support dominance lemmas show the counted-ROM losses are less than 1. These facts are meaningful concrete sanity checks for the counted ledger.
Cardinality-summary lemmas	<code>counted_rom_loss_from_query_budgets_and_support</code> and related lemmas package both the formula for the loss and the strict domination of the numerator by the support size.
Vacuity lemmas	<code>fs_with_aborts_min_entropy_termE</code> , <code>fs_with_aborts_min_entropy_term_ge1</code> , and <code>rejection_sampling_loss_term_ge1</code> prove that the paper-style rejection-sampling ledger is already at least 1, with a min-entropy component exactly 12. This is why final claims must preserve the vacuity warning.
<code>bimodal_to_msis_reduction_loss_termE</code>	This confirms the current adapter-loss slot is exactly 0. It is a definition-unfolding fact, not a new hardness theorem.

This closes the script-companion coverage for `HAETAE_Reductions.ec` as an arithmetic ledger: every operation family and lemma family has a mathematical role and a proof-script reading. It does not close the cryptographic assumption coverage for MLWE or Module-SIS; those remain attached to `HAETAE_Assumptions.ec` and the final theorem composition.

1.16 Feedback 3 criticism for HopGames and Security

Before this expansion, the answer for `HAETAE_HopGames.ec` and `HAETAE_Security.ec` was still negative: a mathematician or cryptologist could not reliably understand those EasyCrypt scripts from the report alone, even with the scripts open. The evidence is internal to the document. The coverage table still marked the declaration dictionary, state interpretation, restriction explanations, lemma ledger, and proof-script commentary as partial. The existing `HAETAE_HopGames.ec` section only gave four implementation layers and a few representative lemma names; it did not explain the long chain of transcript games, counted-ROM disciplines, sampler bad events, erasure equivalences,

and exact-hyperball bridges. The existing `HAETAE_Security.ec` section correctly said that the file assembles the final theorem family, but it did not tell a reader how the successive composition lemmas differ, which premises are imported from hop games, which premises are hardness assumptions, and which steps are just checked real-order algebra.

The main logical risk was not that the EasyCrypt scripts were unchecked. The risk was that the prose made the proof look flatter than it is. In particular, the scripts distinguish:

- exact equivalences, such as erasing transcript logging or replacing one sampler wrapper by an extensionally equal wrapper;
- Hoare invariants, such as preservation of transcript soundness, bad-event flags, and query budgets;
- probabilistic inequalities, such as splitting a success event into a clean branch and a bad branch;
- external or conditional premises, such as MLWE hardness, Module-SIS hardness, and the structural-to-exact-hyperball sampling obligation;
- explanatory or coarse fallback facts, especially bounds that are true because `rejection_sampling_loss_term` is already at least 1.

Therefore, the revision below does not claim a new cryptographic theorem. It adds a companion map that explains what the machine checker proves in these two files, what remains a premise supplied by another checked file or an external assumption, and which statements are explanatory bridges or vacuity warnings.

1.17 Script companion coverage for HAETAE HopGames

`HAETAE_HopGames.ec` is the largest script because it turns the generic EUF-CMA game into transcript-aware and sampler-aware games. It is not a single game hop. It is a stack of games, invariants, exact equivalences, and bad-event bounds. The table below is the reading guide for its main proof layers.

Script layer	Mathematical and proof-script reading
<code>SigningSimulator,</code> <code>RealSigningSimulator,</code> <code>EUF_CMA_SimulatedSign</code>	Abstracts the signing oracle behind a simulator interface. The simulated game has the same EUF-CMA success predicate as <code>SIG.EUF_CMA</code> , but calls <code>Sim.sign</code> and records the queried message-context pairs. The checker verifies procedure calls and state updates; no hardness assumption is used.
<code>transcript_log_</code> <code>consistent,</code> <code>transcript_</code> <code>log_ready,</code> <code>internal_</code> <code>transcript_state_sound</code>	Defines the invariant connecting three views of signing history: <code>queries</code> , <code>transcripts</code> , and <code>records</code> . Read the lemmas ending in <code>_nil</code> , <code>_cons</code> , <code>_valid</code> , and <code>_no_min_entropy</code> as list invariant lemmas. They are checked structural facts about logs, not cryptographic reductions.

Budget predicates		<code>budgeted_programmed_query_count_discipline</code>, <code>budgeted_paper_sim_signing_count_discipline</code>, <code>budgeted_paper_sim_sampler_freshness_discipline</code>, and related predicates are the explicit query-budget conditions. They are the reason the counted-ROM terms in <code>HAETAEReductions.ec</code> can use the concrete constants $q_H = q_S = 2$.
<code>sampler_rom_covered</code>		States that every sampler-expand query already present in the lazy-ROM map is accounted for either as an adversary hash query or as a sampler query. The “fresh after clean seed” lemmas convert absence from the two logs plus a clean flag into absence from the ROM table.
Public-simulation operations	opera-	<code>public_sim_*</code> operations reconstruct the public pieces of a signing transcript from public coins and the public key. Their <code>_E</code> lemmas are definition-expansion facts used in equivalence proofs.
Paper-simulation operations	sample	<code>paper_sim_*</code>, <code>paper_sim_sample_from_public_coins</code>, <code>paper_sim_sample_from_real_signing_coins</code>, and <code>paper_sim_sample_from_rejection_attempt</code> nor- malize several sampler representations to a common <code>paper_sim_signature_sample</code>. The checker proves field equalities and well-formedness; this is explanatory plumbing for later game hops.
Sampler modules		<code>ROMPaperSimSigningSampler</code>, <code>RealSigningPaperSimSampler</code>, <code>ROSigningAttemptPaperSimSampler</code>, <code>HAETAERejctionPaperSimSampler</code>, <code>ExactHyperballHAETAERejctionPaperSimSampler</code>, <code>ExactHyperballPaperSimSampler</code>, and <code>ROExactHyperballPaperSimSampler</code> are alternate im- plementations of the signing-sample source. The proof uses them as named endpoints in NMA games.
NMA wrappers		<code>ROMInternalTranscriptAsNMA</code>, <code>ROMInternalTranscriptPublicSimAsNMA</code>, <code>ROMInternalTranscriptPaperSimAsNMA</code>, and <code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code> turn an adaptive adversary plus a simulated signing oracle into an NMA adversary that internally runs the transcript game. This is the central structural trick of the proof.
Counted and programmed transcript games		<code>EUFCMA_ROMInternalTranscriptCountedSign</code>, <code>EUFCMA_ROMProgrammedTranscriptCountedSign</code>, <code>EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign</code>, and <code>EUFCMA_ROMProgrammedTranscriptBudgetedSampledSign</code> expose the random oracle bad flags, programmed sites, and bounded hash/sign counters.

Invariant sections		<code>InternalTranscriptSignSound,</code> <code>CountedROMInternalTranscriptDiscipline,</code> <code>ProgrammedROMTranscriptDiscipline,</code> <code>BudgetedProgrammedROMTranscriptDiscipline,</code> and <code>BudgetedSampledROMTranscriptDiscipline</code> prove that the game states stay sound after oracle calls and adversary calls. These are Hoare proofs: <code>proc</code> , <code>wp</code> , <code>call</code> , <code>rnd</code> , and <code>auto</code> discharge preservation of explicit predicates.
	<code>BudgetedProgrammedToSampledBridge</code>	<code>EdDirectBridge</code> is the bridge between a counted game that samples signing coins directly and a sampled game using <code>DirectSigningCoinSampler</code> . The long relational postconditions state equality of all shared state fields and bad flags.
Erasure and exactness sections		<code>HAETAEROMInternalTranscriptExact,</code> <code>ROMInternalTranscriptStructuralNMA,</code> <code>ROMInternalTranscriptPublicSimNMA,</code> <code>ROMInternalTranscriptROMPaperSimNMA,</code> <code>ROMInternalTranscriptRealSigningPaperSimNMA,</code> <code>ROMInternalTranscriptROSigningAttemptPaperSimNMA,</code> <code>TranscriptLoggingErasure,</code> and <code>RealSigningSimulatorExact</code> are <code>byequiv</code> sections. They prove equality or one-sided implication of success events by matching calls and preserving corresponding state.
Exact-hyperball layer	lifting	<code>structural_to_exact_hyperball_paper_sample_loss_obligation</code> is a conditional per-call sampling-loss premise. Lemmas around <code>concrete_ro_signing_attempt_to_exact_hyperball_*</code> transport this premise through lazy-ROM freshness and clean-event hypotheses.
Concrete layer	<code>HAETAE_ROM.FRO</code>	<code>ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSim</code> specializes the abstract oracle lifting to the concrete lazy random oracle. Several lemmas here still conclude with coarse probability bounds because the rejection term is already at least 1.
Rejection and exactness	no-fallback	<code>ROMInternalTranscriptHAETAERejectionPaperSimNMA</code> and <code>ROMInternalTranscriptExactHyperballPaperSimNMA</code> connect rejection-sampling and exact-hyperball samplers. The “no fallback” lemmas rely on support facts about <code>dexact_hyperball_signing_attempt_state</code> .
Special-soundness interface layer		<code>BimodalSpecialSoundness_Game</code> , <code>SpecialSoundness_Game</code> , <code>BimodalTranscriptExtractor_As_ModuleSIS</code> , and the <code>special_soundness_interfaces_*</code> lemmas are interface checks for the forking/special-soundness boundary. They do not prove a new lattice theorem; they package obligations imported from the assumption and transcript files.

The main mutable state of `HAETAE_HopGames.ec` has the following mathematical interpretation.

State component	Meaning
-----------------	---------

<code>pk_current, sk_current</code>	The public and secret key currently installed in a transcript game or NMA wrapper. Invariants typically require <code>pk_current = public_key_of_secret(sk_current)</code> .
<code>queries</code>	The generic EUF-CMA signing log, containing only message-context pairs.
<code>transcripts</code>	The transcript log used by ROM programming and special-soundness reasoning. Validity and no-min-entropy-failure predicates are stated over this list.
<code>records</code>	The synchronized list of message, context, signature, and transcript records. It is the bridge between the generic signing log and the transcript log.
<code>C.hash_queries</code>	The counted lazy-ROM log of hash queries. Challenge-query counts are derived from this list.
<code>C.programmed_sites</code> and <code>C.signature_sites</code>	The random-oracle sites that have been programmed, and the sites induced by signing transcripts. Conflict-freedom of these sites is the ROM programming discipline.
<code>C.bad_prequery</code> , <code>C.bad_reprogram</code> , <code>C.bad_min_entropy</code>	Bad-event flags for prequerying the challenge site, attempting inconsistent reprogramming, and hitting a transcript min-entropy failure. The counted-ROM proof bounds these flags rather than hiding them in prose.
<code>adversary_hash_count</code> , <code>signing_count</code>	Budget counters. The scripts intentionally saturate them at the fixed budgets instead of leaving the number of queries symbolic.
<code>adversary_hash_queries</code> , <code>sampler_expand_queries</code> , <code>sampler_bad_prequery</code> Sampler <code>sk_current</code> fields	The budgeted paper-simulation NMA wrapper's sampler-specific query log and bad event. These variables support the split between clean sampling and sampler prequery failure. Sampler modules store the secret key needed to produce signing samples. The equivalence proofs require equality of these fields across paired samplers.

The proof-script patterns in this file should be read as follows.

Pattern	Reader's interpretation
<code>hoare[...]</code> with invariant preservation	The checker proves that each oracle or adversary call preserves an explicit state predicate. These blocks are not probability bounds; they are deterministic or almost-sure invariant arguments over program state.
<code>byphoare</code> converting Hoare facts to probabilities	Used for zero-probability bad-event lemmas such as transcript bad-forgery and min-entropy bad-event claims. The mathematical step is: if a Hoare proof shows the event is impossible, its probability is 0.
<code>byequiv</code> and long relational postconditions	The checker compares two games in lockstep. Equalities such as <code>=\{globH, globA\}</code> and field-by-field equalities are the formal statement that the two executions expose the same interface and return the same result, or that a left success implies a right success.

<code>Pr[mu_split],</code> <code>Pr[mu_sub], Pr[mu_or]</code>	These are probability algebra steps. They split a success event into clean and bad branches, drop conjuncts to obtain upper bounds, or union-bound bad events.
<code>ler_trans, ler_add, and</code> <code>smt(mu_bounded)</code>	These are real-order composition steps. They do not add cryptographic content; they combine already-proved inequalities and use that probabilities lie between 0 and 1.
Coarse fallback bounds	When a proof shows a desired inequality using <code>rejection_sampling_loss_term_ge1</code> and <code>mu_bounded</code> , the result is machine-checked but vacuous as a concrete-security estimate. The report must keep this distinction visible.

For `HAETAE_HopGames.ec`, the machine-checked content is the typing, Hoare judgments, equivalence judgments, probability inequalities, and real-order side conditions appearing in the script. The MLWE and Module-SIS hardness terms are not proved here. The structural-to-exact-hyperball paper-sample loss is a conditional premise when it appears as `structural_to_exact_hyperball_paper_sample_loss_obligation`. The prose explanations in this section are only a reading guide for those checked statements.

1.17.1 Lemma-ledger companion for HAETAE HopGames

Lemma-ledger criticism for HAETAE HopGames. The current report plus `HAETAE_HopGames.ec` is still not a source-independent reconstruction manual. The script has more than eight thousand lines and many theorem surfaces that look similar but have different logical status: exact `equiv` results, Hoare invariant preservation, probability upper bounds, counted-ROM bad-event bounds, rejection-sampling premises, and coarse fallback inequalities. Without a ledger, a reader could mistake a checked erasure equivalence for a sampling-distance theorem, or could miss that the structural-to-exact-hyperball bridge is conditional when it is used as an antecedent. The compact rows below identify the proof obligations that matter for the final EUF-CMA path while keeping the EasyCrypt source as the authoritative tactic-level artifact.

EasyCrypt item and lines	Mathematical statement	Category	Premises and imports	Downstream role
<code>requireimport</code> and <code>import</code> block; 1–21	Imports the generic signature games, HAETAE scheme model, distributions, assumption games, transcript predicates, loss terms, rejection facts, ROM interface, and counted-ROM programming layer.	Imported/library fact and external-assumption boundary.	Uses <code>Sig_ROM.ec</code> , <code>HAETAE_Assumptions.ec</code> , <code>HAETAE_Transcript.ec</code> , <code>HAETAE_Rejection.ec</code> , and <code>HAETAE_ROM_Programming.ec</code> .	Provides all game, hardness, transcript, and bad-event symbols later composed by <code>HAETAE_Security.ec</code> .
<code>SigningSimulator</code> , <code>RealSigningSimulator</code> , <code>EUF_CMA_SimulatedSign</code> ; 23–80	Introduces an abstract signing-simulator interface, realizes it by the real scheme signer, and defines a simulator-backed EUF-CMA game with signing-query logging.	Machine-checked game/module boundary.	Requires a <code>SIG.Scheme</code> , <code>SIG.POracle</code> , and adversary module.	Starting point for the simulator-erasure and real-signing exactness lemmas used in the security composition.

<code>transcript_log_consistent</code> , <code>transcript_log_ready</code> , <code>internal_transcript_state_sound</code> ; 81–441	Defines and proves list invariants connecting message-context queries, transcripts, signing records, public-key consistency, transcript validity, and min-entropy clear logs.	Machine-checked fact and invariant family.	Uses signing-record soundness and ROM-programming no-failure predicates.	Supplies the state predicates preserved by internal transcript games and bad forgery bounds.
<code>EUF_CMA_TranscriptSimulatedSigning</code> , <code>EUF_CMA_InternalTranscriptSigning</code> , <code>EUF_CMA_ROMInternalTranscriptSigning</code> ; 442–612	Adds transcript logs to simulated signing, replaces the scheme call by internal signing, and then exposes sampler, message-hash, challenge-hash, and matrix ROM calls in the signing game.	Machine-checked game modules.	Uses transcript constructors, <code>drandom_coins</code> , <code>sign_internal</code> , and typed ROM query constructors.	Forms the main source games for the ROM-transcript hop in HAETAE_Security.ec .
<code>ROMInternalTranscriptASNMA</code> ; 613–664	Replaces the adaptive ROM-internal transcript game into an NMA adversary wrapper that internally simulates signing transcripts.	Reduction adapter.	Uses the adversary through the same oracle/signing interface and reconstructs the secret key from the public-key seed field.	First structural bridge from EUF-CMA to UF-NMA in the final composition.
<code>public_sim_*</code> operations and <code>ROMInternalTranscriptPublicSimNMA</code> ; 665–854	Replaces secret-key-dependent signing components; deterministic reconstructions and proves the <code>_E</code> equalities against real signing.	Machine-checked fact and reduction adapter.	Uses deterministic vector/poly generation, public-key challenge terms, and signature-field definitions.	Enables exact replacement of structural NMA by public-simulation NMA.
<code>paper_sim_signature_sample</code> , <code>paper_sim_*</code> , public/real/rejection sample families; 855–1351	Normalizes public coins, real signing coins, and rejection-attempt states into one paper-simulation sample type; proves well-formedness and field equality lemmas.	Machine-checked fact with explanatory sampler-model boundary.	Uses signature well-formedness, rejection attempt predicates, and current no-abort facts.	Provides the sample carrier used by all paper-simulation NMA wrappers.
<code>PaperSimSigningSampler</code> and sampler modules; 1352–1586	Defines the sampling interface and seven sampler implementations: ROM public sample, real signing sample, RO signing attempt, HAETAE rejection attempt, exact-hyperball rejection, exact-hyperball, and RO exact-hyperball.	Machine-checked module/procedure boundary.	Uses <code>drandom_coins</code> , ROM sampler queries, rejection distributions, and <code>dexact_hyperball_signing_attempt_state</code> .	Names the endpoints of the paper-simulation and exact-hyperball hop chain.
<code>StructuralAttemptPaperSample</code> , <code>ExactHyperballPaperSample</code> , <code>ROMInternalTranscriptPaperSimNMA</code> , <code>ROMInternalTranscriptBudgetedPaperSimNMA</code> ; 1587–1765	Defines direct one-procedure sample sources and NMA games, including a <code>ROMInternalTranscriptBudgetedPaperSimNMA</code> ; with adversary-hash count, signing count, sampler-query log, and sampler-prequery bad flag.	Machine-checked game modules and reduction adapters.	Uses <code>PaperSimSigningSampler</code> hash-query budgets, signature-query budgets, and sampler prequery predicates.	Provides the budgeted NMA games whose probabilities are charged to counted-ROM and rejection-sampling loss terms.

ROMInternalTranscriptBudgetedSign;NMAQueryBudget 1766–2622	BudgetedSign;NMAQueryBudget validation, sampler-expand point/log/budget bounds, lazy-ROM sampler freshness, and clean sampler one-call facts.	Machine-checked Hoare/probability invariant section.	Premises include query-budget disciplines, sampler-ROM coverage, and losslessness/freshness of concrete lazy ROM calls.	Supplies the one-call and NMA-level sampler-prequery estimates used before the exact-hyperball bridge.
EUFCMA_ROMInternalTranscriptCountedSign; EUFCMA_ROMProgrammedTranscriptCountedSign; EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign; EUFCMA_ROMProgrammedTranscriptBudgetedSampledSign; 2624–2978	Introduces counted/programmed ROM games, then adds explicit hash/sign budgeted counted sign, programmed challenge sites, and a sampled signing budgeted counted sign.	Machine-checked game modules.	Uses CountedLazyROM, ProgramChallenge, challenge-query counters, and direct signing-coin sampler interfaces.	These games expose the bad flags later bounded by counted-ROM programming loss.
InternalTranscriptSign;Bonds 2979–3574	Bonds internal and ROM-internal transcript-game state soundness, transcript validity, log readiness, no-min-entropy, signature-site-clearness, and bad forgery zero/count bounds.	Machine-checked Hoare and probability fact family.	Uses internal transcript invariants and byphoare zero-event reasoning.	Feeds the ROM-transcript clear-site reductions in the security file.
CountedROMInternalTranscriptDiscipline 3576–3769	CountedROMInternalTranscriptDiscipline budget state and derives bad-clear, bad-false, counted bad zero, budget-sound, and subunit bad-event lemmas.	Machine-checked invariant and probability boundary.	Uses CountedLazyROM state facts and counted-ROM programming bound obligation.	Connects counted bad flags to counted_rom_programming_loss_term.
ProgrammedROMTranscriptDiscipline 3771–3904	ProgrammedROMTranscriptDiscipline games preserve min-entropy-clear state and bounds min-entropy and prequery/reprogramming bad events from a budget expression.	Machine-checked invariant and probability bound.	Uses transcript no-failure lemmas and counted-ROM bad-flag wrappers.	Supplies the programmed-ROM discipline consumed by budgeted variants.
BudgetedProgrammedROMTranscriptDiscipline 3906–4518	BudgetedProgrammedROMTranscriptDiscipline discipline, reprogram discipline, query-budget preservation, adaptive prequery lifting, and checked concrete bad-event bounds for the budgeted programmed game.	Machine-checked invariant and query-budget family.	Premises include hash_query_budget_count, challenge-query counts, and honest-site conflict-freedom.	Produces the budget-certified branch bounds used in counted-ROM security composition.
BudgetedSampledROMTranscriptDiscipline 4520–5228	BudgetedSampledROMTranscriptDiscipline budget and bad-event discipline for the sampled-direct game and proves sampler-message-hash prequery and prequery/reprogramming bad bounds.	Machine-checked invariant and probability bound.	Uses DirectSigningCoinSampler and the ROM-programming prequery lemmas.	Prepares the sampled-direct endpoint for equivalence with the programmed game.
BudgetedProgrammedToSampledDirectBridge 5230–5599	BudgetedProgrammedToSampledDirectBridge for hash, sign, adversary, and main procedures between budgeted programmed counted and sampled-direct games; transfers bad-event probabilities.	Checked equiv family and reduction adapter.	Requires module restrictions preventing access to CountedLazyROM internals and shared game state.	Lets the final proof use direct-sampler prequery estimates for the programmed game.

HAETAEROMInternalTranscriptPrivateExact; oracle/adversary erasure equivalences and haetae_rom_internal_transcript_erasure_exact . 5601–5704	Checked exact equivalence.	Uses inlining of HAETAE and ROM-internal transcript games under module separation restrictions.	First exact hop from generic EUF-CMA to the ROM-internal transcript game in HAETAE_Security.ec .
ROMInternalTranscriptStructuralNMA; ROM-internal transcript game to the structural NMA wrapper and proves clear-site and ordinary structural NMA bounds. 5706–5868	Checked equivalence/probability inequality.	Uses oracle/adversary equivalences and transcript-log clear-site events.	Main structural EUF-CMA-to-NMA reduction used by counted and legacy security lemmas.
ROMInternalTranscriptPublicSimNMA; ROM-internal transcript game to the public simulation NMA wrapper and proves clear-site and ordinary structural NMA bounds. ROMInternalTranscriptROMPaperSimNMA; ROM-internal transcript game to the ROM paper simulation NMA wrapper and proves clear-site and ordinary structural NMA bounds. ROMInternalTranscriptReadSigningAttemptPaperSimNMA; ROM-internal transcript game to the read-signing-attempt paper simulation NMA wrapper and proves clear-site and ordinary structural NMA bounds. 5874–6624	Checked exact equivalence family.	Uses public-sim equalities, paper-sample equalities, and sampler module restrictions.	Builds the NMA-side sampler chain later consumed by Security.
structural_to_exact_hyperball_paper_sample_loss_obligation and concrete_ro_*loss family; 6615–6986	Conditional premise plus machine-checked probability inequalities.	Premises include sampler freshness, old-log cleanliness, and rejection-sampling loss terms.	This is the key statistical-loss bridge; it must not be reported as a proved paper sampler theorem unless the antecedent is discharged.
ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA; ROM-internal transcript game to the RO signing-attempt exact hyperball paper simulation NMA wrapper and proves clear-site and ordinary structural NMA bounds. 7002–7291	probability inequality family with conditional branches.	Uses the sampling-loss obligation, budgeted NMA wrappers, clean-event lemmas, and sometimes coarse mu_bounded reasoning.	Provides the RO-signing-attempt to RO-exact-hyperball NMA bounds used by final Security lemmas.
ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA; ROM-internal transcript game to the concrete RO signing-attempt exact hyperball paper simulation NMA wrapper and proves clear-site and ordinary structural NMA bounds. 7318–7817	concrete ROM lifting.	Uses concrete lazy-ROM facts, dexact_hyperball support facts, and sampler-event preservation.	Supplies concrete evidence for the checked RO sampling hop used by the final counted-ROM theorem.
ROMInternalTranscriptHAETAERejectionPaperSimNMA; ROM-internal transcript game to the HAETAER rejection paper simulation NMA wrapper and proves clear-site and ordinary structural NMA bounds. 7819–8230	Checked equivalence and no-fallback theorem family.	Uses rejection acceptance/no-abort lemmas from HAETAE_Rejection.ec .	Removes rejection-specific wrappers before the final exact-hyperball NMA assumption is applied.

ROMInternalTranscriptPaperSimHop 8232–8254	Public-sim-to-paper-sim sampling hop into an NMA probability bound with <code>rejection_sampling_loss_term</code> .	Machine-checked probability inequality.	Premise: a concrete sampling-hop inequality for the two NMA wrappers.	Used by Security when composing paper-simulation NMA bounds.
TranscriptLoggingErasureRealSigningSimulatorExact 8256–8422	Shows transcript logging is observationally erased and the real signing simulator matches the scheme signing oracle.	Checked exact equivalence.	Uses module restrictions excluding game internals and simulator state.	Connects the simulator-based games in Security back to the generic <code>SIG.EUF_CMA</code> game.
ForkingAdversary, TranscriptExtractor, BimodalTranscriptExtractor, BimodalSpecialSoundnessGame, SpecialSoundnessGame, BimodalTranscriptExtractor_As_ModuleSIS; 8424–8499	Defines the forking adversary and extractor interfaces <code>forking</code> , the games that test bimodal self-target MSIS and Module-SIS extraction from transcript forks.	Imported-interface/game boundary.	Uses transcript extraction operators and Module-SIS adapters.	Provides the special-soundness interface normalized before final hardness composition.
BimodalSpecialSoundnessInterfaces_*, fs_with_aborts_interfaces_*; 8500–8649	Proves the bimodal-to-Module-SIS special-soundness lift and packages forking, public reconstruction, signer soundness, rejection-bound, and FS interfaces as ready.	Machine-checked interface theorem family.	Uses <code>forking_extraction_obligation_holds</code> , <code>bimodal_to_module_sis_lift_obligation_holds</code> , and reduction-term nonnegativity.	Supplies the interface premise consumed by several Security theorem variants.
structural_to_exact_hyperball_* and dexact_hyperball_*boundary_*; 8651–8800	Relates structural-to-exact-hyperball obligations to attempt/coin-sample-pair loss premises and proves exact-hyperball rejection/no-fallback distribution boundary lemmas.	Machine-checked implications from conditional premises and model-boundary facts.	Uses <code>HAETAE_Rejection.ec</code> exact-hyperball support and loss-obligation lemmas.	Closes local boundary checks but does not prove paper/implementation sampler equivalence.
Module restriction families; 1768–1771, 2981–2984, 3578–3585, 3773–3775, 3908–3911, 4522–4525, 5232–5236, 5603–5606, 5708–5711, 5876–5879, 6043–6046, 6214–6217, 6420–6423, 7004–7008, 7821–7824, 8032–8035, 8234–8241, 8258–8264, 8342–8346, 8502–8505	Declares which adversary, oracle, sampler, and game modules cannot access one another’s private state.	Imported/module-separation fact checked by EasyCrypt.	Depends on EasyCrypt’s module system and the declared game interfaces.	Prevents circular calls and hidden-state leaks in every equivalence and Hoare proof used by the final security composition.

1.18 Script companion coverage for HAETAE Security

`HAETAE_Security.ec` is the composition file. It does not implement the sampler or ROM machinery itself. Instead, it imports the previous scripts and assembles theorem families by repeatedly

applying already-proved hops, hardness premises, and arithmetic rewrites.

Section or lemma family	Mathematical and proof-script reading
correctness_obligation and correctness_obligation_holds	Defines perfect correctness as verification of every internally generated signature. The proof unfolds <code>keygen_internal</code> and applies <code>verify_internal_sign_internal</code> . This is checked internal correctness, not a security reduction.
correctness_perfect	Shows the generic correctness game fails with probability 0. The proof inlines the game and HAETAE procedures, uses Hoare reasoning, samples keys and coins, and discharges the final verification facts with <code>valid_signature_sign_internal</code> and <code>verify_norm_ok_current</code> .
NoSigningSecurity	Uses <code>SIG.nma_as_cma_exact</code> to embed an NMA adversary as a CMA adversary with no signing queries. The lemma composes an NMA-to-hardness premise, the bimodal-to-Module-SIS premise, and MLWE/Module-SIS hardness bounds into the legacy <code>haetae_euf_bound</code> .
Generic lemmas euf_cma_security	These are algebraic composition lemmas. Their inputs are a Fiat-Shamir hop bound, an NMA hardness bound, a bimodal-to-Module-SIS bound, and the two hardness-term bounds. The proof uses <code>ler_trans</code> , <code>ler_add</code> , and bound rewrites.
FullReductionWithSimulatorHop	Replaces the generic EUF-CMA game by <code>EUF_CMA_SimulatedSign</code> using <code>real_signing_simulator_exact</code> . The section is an exact-game interface bridge followed by the same real-order composition as the generic lemma.
FullReductionWithROMTranscriptHop	Replaces the generic hop from ordinary signing to ROM-internal transcript signing. The <code>rom_internal_transcript_hop_from_clear_site_*</code> lemmas split success into a clear-site branch plus a bad-forgery branch and charge the bad branch to either the legacy FS-with-aborts terms or the counted-ROM programming term.
MechanizedBimodalModuleSISHardness	The <code>bimodal_to_module_sis_reduction_bound</code> rewrites the checked exact adapter <code>bimodal_msis_as_module_sis_exact</code> plus the zero adapter-loss term. It is not an independent proof of Module-SIS hardness.
MechanizedROMTranscriptBimodalHardness	Replaces the ROM transcript hop with the mechanized bimodal-to-Module-SIS adapter. The counted version targets <code>haetae_euf_counted_rom_bound</code> ; the legacy version targets <code>haetae_euf_bound</code> .
MechanizedROMTranscriptComposition	Replaces the NMA composition lane. It progressively replaces structural NMA, public-simulation NMA, paper-simulation NMA, real-signing paper simulation, HAETAE-rejection simulation, exact-hyperball simulation, and random-oracle exact-hyperball simulation, then applies MLWE and bimodal/Module-SIS bounds.

euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal	This is the representative checked counted-ROM theorem shape. Its premise is an NMA bound for <code>ROExactHyperballPaperSimSampler</code> plus <code>rejection_sampling_loss_term</code> . The conclusion is the formal EUF-CMA bound <code>haetae_euf_counted_rom_bound</code> .
euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal	This variant exposes the budgeted lifting premises explicitly: unbudgeted-to-budgeted for the signing-attempt sampler, budgeted-to-unbudgeted for the exact-hyperball sampler, the structural sampling-loss obligation, and a final NMA bound containing <code>rom_signature_query_budget · rejection_sampling_loss_term</code> .

The final composition section uses module restrictions to enforce information flow. The exclusions should be read by category, not as arbitrary syntax.

Restricted modules	Why the restriction matters
SIG.EUF_CMA and EUF_CMA_*	Prevent the adversary or oracle from inspecting game-private secret keys, query logs, transcript logs, bad flags, or return events.
ROMInternalTranscript*AsNMA	Prevent circular access to the NMA wrappers that are being used as simulated adversaries in the reduction.
Paper-simulation sampler modules	Restrictions on <code>ROMPaperSimSigningSampler</code> , <code>RealSigningPaperSimSampler</code> , <code>ROSigningAttemptPaperSimSampler</code> , <code>HAETAERejectionPaperSimSampler</code> , <code>ExactHyperballHAETAERejectionPaperSimSampler</code> , <code>ExactHyperballPaperSimSampler</code> , and <code>ROExactHyperballPaperSimSampler</code> prevent adversaries from using sampler internals except through the declared signing-oracle interface.
HAETAE	When excluded from the adversary, prevents direct calls to the scheme module outside the intended oracle/game interface.
H and A exclusions in Samp	Ensure a sampler cannot call back into the random oracle or adversary except through the interfaces explicitly passed to it by the wrapper.
Bmlwe, Bbimodal, and Bsis	These are abstract reduction modules. Their success probabilities are bounded by premises or assumption terms; the security file composes those bounds but does not implement the reductions' internals.

The proof-script patterns in `HAETAE_Security.ec` are deliberately simple. A `rewrite` of an exactness lemma changes the game whose probability is being bounded without loss. A `rewrite` of a grouped bound such as `haetae_euf_counted_rom_bound_groupedE` unfolds the arithmetic ledger. An `apply` of a previous security lemma selects the next composition theorem. Nested `ler_trans` and `ler_add` blocks compose inequalities and preserve nonnegative loss terms. The checker verifies all of these real-order steps, but the cryptographic assumptions remain exactly the assumptions

named in the premises: MLWE hardness, Module-SIS hardness, NMA-to-hardness bounds, and sampling-hop obligations.

Consequently, this file should be read as a machine-checked composition argument, not as a source of new assumptions. The statements that are machine-checked are the conditional theorem statements and their real-order proofs. The hardness estimates are assumed through the real-valued terms and module-game premises. The prose in this report explains that structure; it is not itself part of the EasyCrypt trusted base.

1.18.1 Lemma-ledger companion for HAETAE Security

Lemma-ledger criticism for HAETAE Security. The existing companion coverage correctly says that `HAETAE_Security.ec` is a composition file, but it still underexplains the theorem boundaries. The script has many lemmas whose conclusions look like the same EUF-CMA bound, while the premises expose different hop surfaces: legacy FS-with-aborts terms, counted-ROM programming loss, exact erasure, mechanized bimodal-to-Module-SIS lifting, exact-hyperball sampling, checked RO sampling, or budgeted paper-sample lifting. A reader cannot reconstruct the logical path unless each lemma family is classified as checked arithmetic composition, imported hop theorem, external hardness premise, or conditional sampling premise. The ledger below is therefore a theorem-boundary map, not a claim that the report alone replays the EasyCrypt proofs.

EasyCrypt item and lines	Mathematical statement	Category	Premises and imports	Downstream role
<code>requireimport</code> and <code>import</code> block; 1–19	Imports generic signature games, HAETAE scheme/algebra/distributions, reductions, assumptions, rejection facts, transcript predicates, ROM-programming facts, HopGames, and real-order lemmas.	Imported/library fact and external-assumption boundary.	Uses <code>HAETAE_HopGames.ec</code> as the main checked hop provider and <code>HAETAE_Assumptions.ec</code> for hardness games.	Defines the entire theorem-composition environment.
<code>correctness_obligation</code> , <code>correctness_obligation_holds</code> ; 21–34	Every internally generated signature verifies for the formal HAETAE model.	Machine-checked fact.	Uses <code>keygen_internal</code> , <code>sign_internal</code> , and <code>verify_internal_sign_internal</code> .	Local correctness fact used before security composition; not a hardness assumption.
<code>correctness_perfect</code> ; 35–70	The generic correctness game fails with probability zero for the formal scheme.	Machine-checked Hoare/probability fact.	Requires module restrictions <code>A<:SIG.CORR_ADV\{-H,-HAETAE\}</code> and algebra lemmas for valid signatures and current verification norms.	Certifies perfect correctness of the EasyCrypt model.
<code>NoSigningSecurity</code> , <code>euf_cma_security_no_signing</code> ; 73–124	Composes an NMA bound, a bimodal-to-Module-SIS bound, MLWE hardness, and Module-SIS hardness to bound an NMA-as-CMA adversary.	Machine-checked composition with external cryptographic assumptions.	Premises are NMA, MLWE, Module-SIS, and bimodal reduction probabilities; uses <code>SIG.nma_as_cma_exact</code> .	Baseline no-signing theorem; not the main adaptive-signing result.

<code>euf_cma_security</code> , <code>euf_cma_security_</code> <code>with_hop_interfaces</code> ; 127–208	Generic EUF-CMA composition from an FS-with-aborts hop, NMA bound, bimodal-to-Module-SIS bound, and MLWE/Module-SIS hardness assumptions.	Machine-checked real-order composition; conditional premises.	Requires <code>fs_with_aborts_interfaces_ready</code> only as an explicit interface premise in the second lemma.	Template used by later sections after replacing the first hop with more specific HopGames lemmas.
<code>FullReductionWithSimulatorHop</code> ; 211–304	<code>disc_hop</code> ; <code>real_signing_simulator_exact</code> to replace generic EUF-CMA by the simulated-sign game, then applies the generic composition theorem.	Checked exact-hop composition.	Imports the simulator exactness lemma from HopGames and external hardness premises.	Connects the generic theorem shape to the simulator game surface.
<code>FullReductionWithROMTranscriptHop</code> ; 306–405	<code>disc_hop</code> ; <code>rom_internal_transcript_erasure_exact</code> ; splits ROM-internal transcript success into clear-site and bad-site branches, charging the bad branch to legacy FS terms or counted-ROM programming loss.	Machine-checked probability split and checked hop composition.	Uses <code>rom_internal_transcript_bad_forgery_bound</code> and <code>rom_internal_transcript_bad_forgery_counted_bound</code> .	Introduces the counted-ROM path used by the final theorem family.
<code>MechanizedBimodalModuleSIS</code> ; 407–459	<code>Bisim</code> ; the bimodal self-target MSIS game is bounded by the Module-SIS adapter game plus the zero adapter-loss term, then composes that into EUF-CMA bounds.	Machine-checked adapter equality and real-order composition.	Uses <code>bimodal_msis_as_module_sis_exact</code> and <code>bimodal_to_msis_reduction_loss_termE</code> .	Removes the need for an arbitrary <code>Bsis</code> module in later counted-ROM statements.
<code>MechanizedROMTranscriptBound</code> ; 461–596	<code>Bisim</code> ; the ROM-transcript hop with the mechanized bimodal-to-Module-SIS adapter, both for legacy and counted-ROM bounds, and adds clear-site variants.	Machine-checked theorem-boundary composition.	Premises are ROM hop or clear-site reductions, an NMA bound, MLWE hardness, and Module-SIS hardness for the adapter game.	Main bridge from HopGames’ ROM transcript lemmas to grouped final bounds.
<code>MechanizedROMTranscriptBound</code> declarations; 598–631	<code>ConstructNMA</code> composition modules and excludes all relevant game, wrapper, sampler, and adversary internals from each other.	Module restriction family.	Depends on EasyCrypt’s module separation checker and all prior game/module interfaces.	Protects every exact equivalence and reduction application in the final composition lane.
<code>euf_cma_security_from_structural_nma_and_counted_bimodal</code> through <code>euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal</code> ; 633–744	Progressively rewrites the NMA bound through structural, public-sim, paper-sim, ROM-paper-sim, and real-signing paper-sim wrappers.	Checked exact-hop composition with external hardness premises.	Uses HopGames exactness lemmas such as <code>rom_internal_nma_public_sim_exact</code> and <code>rom_internal_nma_real_signing_paper_sim_exact</code> .	Moves the final theorem toward the sampler implementations used by the checked RO sampling hop.

<code>euf_cma_security_from_haetae_rejection_*, exact_hyperball_haetae_rejection_nma_bound_*, haetae_rejection_nma_bound_*</code> ; 746–842	Transfers NMA bounds between HAETAE rejection sampling and exact-hyperball samplers, with or without an added rejection-sampling loss term.	Checked theorem-boundary composition; conditional sampling premise when a hop bound is an antecedent.	Uses <code>rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact</code> and exact-hyperball no-fallback lemmas.	Builds the rejection-to-exact-hyperball bridge used by final counted-ROM theorems.
<code>real_signing_ro_exact_hyperball_*, haetae_rejection_ro_exact_hyperball_*</code> ; 843–972	Transfers real-signing and HAETAE-rejection NMA bounds through RO signing attempt and RO exact-hyperball samplers, including budgeted lifting with a signature-budget multiplier.	Machine-checked probability composition with conditional sampling premises.	Uses <code>rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_*</code> from HopGames and the <code>structural_to_exact_hyperball_paper_sample_loss_obligation</code> antecedent where required.	Provides the sampling-hop premises used by the final checked and budgeted theorem variants.
<code>euf_cma_security_from_exact_hyperball_*, euf_cma_security_from_ro_exact_hyperball_*, euf_cma_security_from_ro_signing_attempt_*</code> ; 974–1099	Composes exact-hyperball, RO exact-hyperball, and RO signing-attempt NMA hops with MLWE and Module-SIS hardness to conclude the counted-ROM EUF-CMA bound.	Machine-checked final-bound composition.	Premises include NMA hop inequalities plus MLWE and Module-SIS hardness probability bounds.	These are alternate final theorem surfaces for different supplied sampling-hop evidence.
<code>real_signing_ro_exact_hyperball_hop_checked_from_loss_budget, real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting</code> ; 1102–1138	Derives the real-signing-to-RO-exact-hyperball hop either from the checked loss-budget lemma or from an explicit paper-sample lifting premise.	Machine-checked implication; conditional premise for the lifting variant.	Uses HopGames’ <code>rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_*</code> lemmas.	Supplies the first premise for the final checked RO sampling theorem.
<code>euf_cma_security_from_paper_sample_lifting_and_counted_bimodal, euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal</code> ; 1140–1214	Final counted-ROM bounds that expose either the paper-sample lifting premise or the budgeted unbudgeted/budgeted conversion premises explicitly.	Machine-checked theorem-boundary composition with conditional sampling premises.	Requires MLWE hardness, Module-SIS hardness, an RO exact-hyperball NMA bound, and sampling-lifting premises.	Documents which assumptions must be supplied to avoid relying solely on coarse checked fallback bounds.

<code>euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal</code> ; 1217–1237	Representative checked counted-ROM theorem: if the RO exact-hyperball NMA probability plus rejection-sampling loss is bounded by MLWE plus bimodal success, and MLWE/Module-SIS are bounded by their hardness terms, then the formal HAETAE EUF-CMA game is bounded by <code>haetae_euf_counted_rom_bound</code> .	Machine-checked final theorem boundary with external hardness premises.	Uses <code>real_signing_ro_exact_hyperball_hop_checked_from_loss_budget</code> and counted-ROM grouped arithmetic.	Best current final theorem shape, but still conditional on the formal ROM model and rejection/sampling loss ledger.
<code>euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal</code> ; 1239–1269	Alternative final theorem starting from a real-signing-to-exact-hyperball hop plus an exact-hyperball NMA bound.	Machine-checked final theorem boundary with conditional premises.	Requires MLWE/Module-SIS hardness and the supplied exact-hyperball NMA inequality.	Records a non-RO exact-hyperball route through the same counted-ROM final bound.

1.18.2 Proof-obligation replay for the final Security theorem path

Replay-level criticism. The lemma ledger above is enough to locate the final theorem surfaces, but it is not yet enough for a mathematician or cryptologist to reconstruct the exact EasyCrypt proof obligations used by the two terminal theorem shapes. In particular, the report previously did not spell out which antecedent is passed to which `apply`, where the random-oracle/rejection loss enters, which HopGames lemma is imported, or whether a sampling hop is proved non-vacuously, proved by a coarse boundedness fallback, or simply assumed as a theorem premise. The replay below closes that gap at the proof-obligation level. It is still not a tactic transcript: the EasyCrypt source remains necessary for checking module restrictions, exact syntax, local names, and SMT side conditions.

Common final-composition frame. Both terminal theorem paths live in `MechanizedROMTranscriptConstruct`. The modules declared at `HAETAE_Security.ec` lines 598–631 are part of the theorem boundary: `H` is a signature oracle, `A` is the EUF-CMA adversary, and `Bmlwe`/`Bbimodal` are reduction modules for the MLWE and bimodal self-target MSIS games. EasyCrypt checks the module-separation restrictions, but the numerical hardness estimates for `Pr[MLWE_Game(Bmlwe).main]` and `Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main]` enter as premises. The final arithmetic target is the imported bound expression `haetae_euf_counted_rom_bound`; expanding that expression is a reduction-file exercise, not a new theorem in `HAETAE_Security.ec`.

The common counted-ROM composition engine is `euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal` at lines 746–766. It rewrites the real-signing paper-simulation NMA game to the HAETAE rejection paper-simulation NMA game using `rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact`, then passes the NMA, MLWE, and Module-SIS premises into the earlier real-signing-paper-simulation counted-ROM theorem. Thus the terminal theorems do not redo the EUF-CMA game-hop proof; they supply a suitable NMA premise for that engine and forward the two hardness premises.

Replay for the checked RO-sampling final theorem. The theorem `euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal` appears at `HAETAE_Security.ec` lines 1217–1237. Its statement says: if the RO exact-hyperball paper-simulation NMA success probabil-

ity plus `rejection_sampling_loss_term` is bounded by MLWE success plus bimodal self-target MSIS success, and if those MLWE/Module-SIS games are bounded by their hardness terms, then the formal HAETAE EUF-CMA game is bounded by `haetae_euf_counted_rom_bound`.

Step	EasyCrypt evidence	Proof obligation replay	Status boundary
1	Lines 1230–1232: <code>move=>ro_exact_nma_hopmlwe_boundmsis_bound</code> and <code>applyeuf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal</code> .	The final theorem is reduced to the RO exact-hyperball counted-ROM theorem at lines 1036–1070. That theorem requires four inputs: a real-signing to RO exact-hyperball NMA hop, the RO exact-hyperball NMA-to-MLWE/bimodal premise, the MLWE hardness premise, and the Module-SIS hardness premise.	Machine-checked proof replay step; not an external assumption.
2	Lines 1233 and 1102–1114: <code>real_signing_ro_exact_hyperball_hop_checked_from_loss_budget</code> .	The first required input is discharged by proving $\Pr[\text{RealSigningPaperSimSampler}] \leq \Pr[\text{ROExactHyperballPaperSimSampler}] + \text{rejection_sampling_loss_term_ge1}$ The local proof applies <code>real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop</code> and supplies the HopGames lemma <code>rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound</code> .	Machine-checked implication, but its imported HopGames input is a coarse fallback bound.
3	HAETAE_Security.ec lines 865–884 and HAETAE_HopGames.ec lines 7291–7314.	The real-signing sampler is first rewritten to the RO-signing-attempt sampler by <code>rom_internal_nma_real_signing_ro_attempt_paper_sim_exact</code> . The remaining RO-signing-attempt to RO exact-hyperball inequality is bounded in HopGames by <code>mu_bounded</code> , nonnegativity of the right probability, and <code>rejection_sampling_loss_term_ge1</code> .	Machine-checked, but vacuous as a concrete sampling-distance estimate. The script comment explicitly marks this as a coarse fallback.
4	Lines 1234 and 1218–1223: <code>ro_exact_nma_hop</code> .	The theorem consumes, rather than proves, the premise $\Pr[\text{ROExactHyperballPaperSimSampler}] + \text{rejection_sampling_loss_term} \leq \Pr[\text{MLWE_Game}] + \Pr[\text{BimodalMSIS_Game}]$ This premise is the cryptographic/sampler reduction surface for the RO exact-hyperball NMA endpoint.	Conditional premise. The report must not describe it as derived in this final lemma.
5	Lines 1235–1236 and 1224–1226.	<code>mlwe_bound</code> and <code>msis_bound</code> are forwarded unchanged. The MSIS premise is for the adapter game <code>ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal))</code> .	External cryptographic assumption boundary expressed as theorem premises.
6	Lines 1057–1069 and 746–766.	The RO exact-hyperball theorem internally applies <code>euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal</code> . The real-order step is <code>ler_trans</code> : first convert HAETAE rejection NMA to RO exact-hyperball NMA with one rejection-loss term, then use <code>ro_exact_nma_hop</code> .	Machine-checked probability algebra and exact-hop composition.

The phrase “checked RO sampling hop” should therefore be read narrowly. The EasyCrypt theorem is checked, and it imports a checked HopGames bound, but that HopGames bound currently uses the fallback `rejection_sampling_loss_term_ge1`. It is a valid formal implication, not a non-vacuous concrete sampler-distance proof.

Replay for the real-signing exact-hyperball final theorem. The theorem `euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal` appears at `HAETAESecurity.ec` lines 1239–1269. Its statement says: if real-signing paper simulation is bounded by exact-hyperball paper simulation plus one rejection-sampling loss, if exact-hyperball paper simulation plus that same loss is bounded by MLWE plus bimodal success, and if MLWE/Module-SIS are bounded by their hardness terms, then the formal HAETAESecurity EUF-CMA game is bounded by `haetae_euf_counted_rom_bound`.

Step	EasyCrypt evidence	Proof obligation replay	Status boundary
1	Lines 1259–1261: <code>move=>real_exact_hopexact_nma_hopmlwe_boundmsis_bound and applyeuf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal</code> .	The theorem reduces to the exact-hyperball counted-ROM theorem at lines 1007–1034. That theorem requires a HAETAESecurity-rejection to exact-hyperball sampling hop, an exact-hyperball NMA-to-MLWE/bimodal premise, and the two hardness premises.	Machine-checked proof replay step.
2	Lines 1262–1264 and 819–841: <code>haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop</code> .	The required HAETAESecurity-rejection sampling hop is derived from the supplied <code>real_exact_hop</code> . The proof rewrites real-signing paper simulation to HAETAESecurity-rejection paper simulation and exact-hyperball rejection paper simulation to exact-hyperball paper simulation.	Machine-checked exact-rewrite composition, but only conditional on the <code>real_exact_hop</code> premise.
3	Lines 1240–1246: <code>real_exact_hop</code> .	This premise is the non-RO sampler bridge: $\Pr[\text{RealSigningPaperSimSampler}] \leq \Pr[\text{ExactHyperballPaperSimSampler}] + \text{rejection_sampling_loss}$ The final theorem does not prove this sampling distance.	Conditional premise. It may be discharged by another lemma or <code>HyperballPaperSimSampler</code> by lines 1239–1269 alone.
4	Lines 1247–1252: <code>exact_nma_hop</code> .	This premise bounds the exact-hyperball NMA endpoint plus one <code>rejection_sampling_loss_term</code> by MLWE plus bimodal success. It is the reduction/hardness interface for the exact-hyperball endpoint.	Conditional premise connecting sampler endpoint to hardness games.
5	Lines 1253–1255.	<code>mlwe_bound</code> and <code>msis_bound</code> are forwarded unchanged, with Module-SIS stated for <code>BimodalMSIS_As_ModuleSIS(Bbimodal)</code> .	External MLWE and Module-SIS hardness assumption boundary.
6	Lines 1028–1033 and 787–817.	The exact-hyperball counted-ROM theorem applies <code>euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal</code> after deriving a HAETAESecurity-rejection NMA bound by <code>ler_trans</code> . The intermediate rewrite uses <code>rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact</code> .	Machine-checked real-order arithmetic plus exact no-fallback rewrite.

The second final theorem is therefore stronger as a replay surface for non-vacuous sampling analysis: it exposes the real-signing-to-exact-hyperball hop as an explicit premise instead of hiding it behind the current coarse RO fallback. It is also less self-contained: the user must supply or prove that sampling-hop premise separately before the theorem gives a concrete end-to-end security estimate.

Replay checklist for reimplementers. To replay either final theorem in EasyCrypt, a reimplementation should first instantiate the module restrictions from lines 598–631, then prove or assume the relevant NMA endpoint premise, then provide MLWE and Module-SIS hardness bounds for

the declared reductions, and only then invoke the final composition lemma. The only arithmetic performed in the final theorem bodies is forwarding premises through earlier lemmas, `ler_trans`, exact `rewrite` steps, and SMT-dispatched probability facts such as `mu_bounded`. None of these steps proves MLWE hardness, Module-SIS hardness, SHAKE-to-ROM soundness, implementation equivalence, or a non-vacuous rejection-sampler distance unless the corresponding premise or imported lemma already supplies it.

1.18.3 Non-vacuous replacement roadmap for the RO exact-hyperball fallback

Fallback criticism. The current EasyCrypt scripts and the preceding report text do identify the coarse fallback, but they still do not give enough information to replace it. In `HAETAE_HopGames.ec`, the comment at lines 7287–7290 says that the intended replacement is a push-forward of the structural-to-exact-hyperball attempt loss through `paper_sim_sample_from_rejection_attempt`, lifted across the NMA wrapper once the lazy-ROM sampler law is exposed. The checked lemma immediately below, however, proves `rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound` by a boundedness argument: the left probability is at most 1, the right probability is nonnegative, and `rejection_sampling_loss_term_ge1` makes the additive loss at least 1. That is machine-checked, but it proves no meaningful closeness between the RO-signing-attempt sampler and the RO exact-hyperball sampler.

Thus the report can truthfully say that the final RO-sampling theorem is checked, but it cannot say that the present proof contains a non-vacuous random-oracle sampler-distance theorem. A reader trying to reimplement the proof needs the following roadmap, not just the theorem name.

Desired replacement statement. The strongest drop-in replacement would prove the same inequality as the current fallback, but without using `rejection_sampling_loss_term_ge1`:

$$\begin{aligned} & \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptPaperSimAsNMA}(A, \text{ROSigningAttemptPaperSimSampler}))] \\ & \leq \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptPaperSimAsNMA}(A, \text{ROExactHyperballPaperSimSampler}))] + \text{rejection_sampling_loss_term_ge1} \end{aligned}$$

If `structural_to_exact_hyperball_paper_sample_loss_obligation` is not globally discharged, the honest replacement should expose it as an antecedent, as the existing boundary lemmas already do. If the proof must go through budgeted wrappers, the resulting theorem may instead have `rom_signature_query_budget*rejection_sampling_loss_term` plus a sampler-prequery term; in that case the final Security bound must be updated rather than silently reusing the one-loss theorem.

Current fallback proof structure.

Script item	What the current script proves	Why it is insufficient	Status
<code>HAETAE_HopGames.ec</code> 7016–7034	States that a per-mode structural paper-sample loss obligation plus an already-lifted NMA inequality gives the same NMA inequality.	This is only an identity wrapper over the supplied lifted premise; it does not lift the per-call sampler loss through adversarial NMA execution.	Machine-checked implication with conditional premise.
Lines 7057–7095	Creates a local one-step fact from <code>structural_attempt_exact_hyperball_paper_sample_one_step_loss</code> , but the final proof still closes by <code>mu_bounded</code> and a loss-at-least-one argument.	The one-step fact is not connected to the NMA result, so the proof remains vacuous as a distribution comparison.	Machine-checked coarse fallback.

Lines 7097–7152	Splits sampled NMA success into a clean branch and the <code>sampler_bad_prequery</code> bad event, then charges the bad event to the finite-event-logic bound <code>budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound</code> .	This is the right shape for a non-vacuous proof, but it requires a genuine clean-lifting inequality as input.	Machine-checked conditional bridge.
Lines 7291–7314	Proves the unbudgeted RO-signing-attempt to RO exact-hyperball NMA bound by <code>left_le1</code> , <code>right_ge0</code> , and <code>rejection_sampling_loss_term_ge1</code> .	The result is true for any right-hand game with nonnegative probability; it does not use sampler structure, ROM freshness, or the structural loss obligation.	Machine-checked but vacuous.
Lines 7733–7815	Specializes the concrete <code>HAETAE_ROM</code> . <code>FRO</code> budgeted NMA lifting path, but the clean-conditioned lemma at lines 7733–7765 again uses boundedness and loss-at-least-one.	Concrete ROM specialization exists, yet the central clean sampler comparison is still not non-vacuous.	Machine-checked coarse fallback.

Replacement proof obligations.

Obligation	Required content	Existing evidence	Boundary
Per-call sampler push-forward	Show that sampling with <code>ROSigningAttemptPaperSimSampler.sample_with_seed</code> at a fresh sampler site is bounded by <code>ROExactHyperballPaperSimSampler.sample_with_seed</code> plus <code>rejection_sampling_loss_term</code> , for every postcondition on the resulting <code>paper_sim_signature_sample</code> .	The intended local facts are lines 6625–6676: <code>concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed*</code> . They use the structural paper-sample obligation and freshness.	Machine-checked once all premises are available.
Distributional push-forward	Connect <code>dsigning_attempt_state</code> and <code>dexact_hyperball_signing_attempt_state</code> through <code>paper_sim_sample_from_rejection_attempt</code> ; do not compare raw internal states when the games observe paper samples or signatures.	The obligation is named at lines 6615–6623. Lines 8651–8722 show how attempt or coin-sample-pair loss obligations imply the paper-sample obligation.	Conditional unless the lower-level attempt or coin-sample-pair loss is proved.
Lazy-ROM sampler law	Expose that a fresh <code>sampler_expand_query</code> returns signing coins with law <code>drandom_coins</code> , and that projections through <code>ro_signing_coins</code> are the only component used by the sampler.	<code>HAETAE_ROM.ec</code> lines 99–109 prove <code>sampler_expand_query_dro_output_ro_signing_coins</code> . <code>HopGames</code> uses it in sample-with-seed structural lemmas around lines 2285–2480.	Machine-checked formal ROM fact; not SHAKE/FIPS evidence.
Freshness and coverage invariant	Maintain <code>sampler_rom_covered</code> and show that a clean seed has not already been queried by the adversary or sampler before the sampler call.	Lines 168–199 give <code>sampler_rom_covered_fresh_after_clean_seed</code> and <code>sampler_rom_covered_old_log_clean_boundary</code> ; lines 6678–6954 package old-log and self-logged clean surfaces.	Machine-checked invariant family.
Clean-event one-call lifting	Lift the per-call sample comparison to one signing-oracle call under the clean condition <code>!sampler_bad_prequery</code> and an active signing budget.	Lines 7413–7731 provide projection facts between <code>0.sign</code> and <code>sample_with_seed</code> , including active signing and exact-hyperball projections.	Partly machine-checked, but the NMA-level clean lifting still needs a non-vacuous composition proof.

Sampler-prequery event	bad	Split total NMA success into clean success plus <code>sampler_bad_prequery</code> , then bound the bad event by query budgets and challenge support.	Lines 2062–2223 prove <code>budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound</code> ; lines 7097–7152 already use the correct union-bound skeleton.	Machine-checked bad-event bound.
Budgeted-to-unbudgeted adapters		Move from unbudgeted NMA wrappers to budgeted wrappers and back without changing the observable success event, or explicitly charge any extra loss.	The final Security budgeted theorem already exposes <code>attempt_unbudgeted_to_budgeted</code> and <code>exact_budgeted_to_unbudgeted</code> premises at lines 1170–1214.	Conditional unless exact adapters are proved for the chosen theorem surface.
Final real-order composition		Use <code>ler_trans</code> , <code>ler_add</code> , <code>mu_sub</code> , and <code>mu_or</code> to combine clean lifting and bad-event bounds, then feed the result into <code>HAETAE_Security.ec</code> .	Existing scripts show the arithmetic pattern at HopGames lines 7097–7152 and Security lines 1036–1070.	Machine-checked arithmetic once the non-vacuous premises are supplied.

Where the structural obligation enters. The structural-to-exact-hyperball premise must enter before any NMA-level probability comparison: it is a one-call distributional statement over `dsigning_attempt_state` and `dexact_hyperball_signing_attempt_state`. In the current script this is visible at lines 6625–6653 and 6965–6984, where the per-call `sample_with_seed` and `StructuralAttemptPaperSample.sample` bounds invoke `structural_to_exact_hyperball_paper_sample_loss_obligation`. A non-vacuous NMA theorem must propagate exactly that premise through signing oracle calls, adversary calls, clean-event preservation, and sampler-prequery splitting. Introducing it only at the final EUF-CMA composition level is too late unless an imported lemma has already performed this lifting.

Proof status after this roadmap. The proof-engineering spike found no direct non-vacuous replacement for the unconditional theorem as written. No new EasyCrypt proof was implemented, because existing lemmas stop at either a one-call sampler comparison or an identity wrapper over an already-supplied NMA-level premise. The missing machine-checked statement is one of the following theorem surfaces:

$$\begin{aligned} & \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptPaperSimAsNMA}(A, \text{ROSigningAttemptPaperSimSampler}))] \\ & \leq \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptPaperSimAsNMA}(A, \text{ROExactHyperballPaperSimSampler}))] + \text{rejection} \end{aligned}$$

derived from `structural_to_exact_hyperball_paper_sample_loss_obligation` without `rejection_sampling_loss_term_ge1`; or the budgeted clean-event variant

$$\begin{aligned} & \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptBudgetedPaperSimAsNMA}(A, \text{ROSigningAttemptPaperSimSampler})) : \text{res} \wedge \dots] \\ & \leq \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptBudgetedPaperSimAsNMA}(A, \text{ROExactHyperballPaperSimSampler}))] + \dots \end{aligned}$$

plus exact or monotone budgeted-to-unbudgeted adapters. In the second route, the checked `budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound` then supplies the bad-event charge, but it does not itself compare the samplers.

The exact local blockage is therefore not the marginal sampler law: `concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed*` and `sampler_expand_query_dro_output_ro_signing_coins` already give the per-call ingredients. The missing proof is the adaptive lifting through `ROMInternalTranscriptPaperSimAsNMA` or its budgeted variant, including signing-oracle sequencing, clean-event preservation, freshness/coverage, and loss accumulation over at most the

signing-query budget. Until that theorem is added, the roadmap above is explanatory documentation. The machine-checked facts are the existing EasyCrypt lemmas cited by line range. The structural sampler-distance premise, SHAKE-to-ROM interpretation, and any implementation equivalence remain outside the checked fallback lemma unless future EasyCrypt code proves them or exposes them explicitly as assumptions.

Focused budgeted clean-event proof spike. The follow-up proof spike inspected the two existing `fel` patterns in `HAETAE_HopGames.ec`: the sampler-prequery bound around lines 2069–2125 and the sampled-direct ROM-programming prequery bound around lines 4935–5000. Both are machine-checked bad-event accumulators for one execution. They sum the probability of a flag becoming true over a counter, but they do not compare two oracle implementations and do not propagate a continuation predicate through `A.forge`. Consequently, the current `fel` surface cannot directly prove the adaptive lossy-oracle statement needed for `concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting`.

The checked one-call ingredients are present. In particular, `concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface` connects an active clean signing call using `ROSigningAttemptPaperSimSampler` to the corresponding `ROExactHyperballPaperSimSampler` call with one `rejection_sampling_loss_term` charge, assuming the structural sampler obligation, `sampler_rom_covered`, a fresh self-logged seed, active `signing_count`, public-key equality, and equality of the sampler secret keys. The source-side projection is `concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection`; the target-side projection is `concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection`.

The proof-infrastructure spike sharpened the blocker further. The current one-call loss lemma quantifies over predicates of type `signature->bool`. The continuation needed inside an adaptive adversary is not merely a predicate on the returned signature: after `O.sign` returns, the rest of `A.forge` can depend on adversary private state, the lazy-ROM table, public query and transcript logs, signing records, counters, and future oracle calls. Thus the missing induction principle must be stateful. Its forge-level shape is: for every continuation predicate C over the final forge result and the public final state later used by `UF_NMA.main`, and for related attempt/exact memories with

$k = \text{signature_query_budget_count} - \text{ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count}$,

prove that the attempt game on the clean branch is bounded by the exact game plus $k \cdot \text{rejection_sampling_loss_term}$. The active `O.sign` premise must itself be stateful:

$$\Pr[0_attempt.sign : R(res, post_state) \wedge \neg sampler_bad_prequery] \leq \Pr[0_exact.sign : R(res, post_state)]$$

where R includes the post-call relation required by the recursive continuation. The existing signature-only one-call theorem is a component of that active-call premise, not the adversary-level lifting principle.

The missing theorem is now isolated as a conditional EasyCrypt proof obligation, not as an external cryptographic assumption. A non-vacuous proof would require an adaptive lifting rule with the following invariant before and after every active signing-oracle call: both games expose the same adversary-visible public key, hash log, transcript log, signing-record log, adversary hash counter, and signing counter; the two sampler secret keys are equal; the attempt side satisfies `sampler_rom_covered`; the attempt side is on the clean branch $\neg \text{ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery}$; and the next sampler seed has not already appeared in the adversary or sampler logs. At each active call, the one-call loss lemma should be instantiated with the continuation success predicate

induced by the rest of `A.forge`; the constant charge should then be summed over at most `signature_query_budget_count` active calls to obtain `rom_signature_query_budget` times `rejection_sampling_loss_term`.

No new proof was added in this spike because the repository currently exposes exact `equiv/byequiv` adversary-call patterns, local `phoare` inequalities, and bad-event `fel` machinery, but not the required stateful relational/probability-loss adversary-call rule. The status is therefore: the local one-call facts remain machine-checked; the clean-event adaptive lifting theorem remains an unproved EasyCrypt proof obligation; the structural sampler-distance premise remains a conditional premise; and SHAKE-to-ROM and implementation-equivalence claims remain external to this checked theorem path.

1.19 Feedback 3 criticism for assumptions, transcripts, and ROM programming

Before this expansion, the report still did not give enough evidence for a reader to understand `HAETAE_Assumptions.ec`, `HAETAE_Transcript.ec`, and `HAETAE_ROM_Programming.ec` at the level required by the new `HAETAE_HopGames.ec` and `HAETAE_Security.ec` companion sections. The old roadmap named the files and listed representative lemma names, but it did not explain which objects were external assumption interfaces, which were checked adapter equalities, which transcript predicates fed special soundness, and which random-oracle flags were later charged to counted-ROM loss terms.

That gap matters because the downstream proof scripts rely on these files without restating their mathematics. The hop-game script assumes the reader already knows the meaning of `valid_forking_pair`, `transcript_valid`, `actual_signature_programming_site`, `bad_prequery`, `bad_reprogram`, and `bad_min_entropy`. The security script assumes the reader knows that `MLWE_Game`, `ModuleSIS_Game`, and `BimodalSelfTargetMSIS_Game` are assumption surfaces rather than proved cryptanalytic estimates. Without the companion material below, the report could still let a reader follow the names of the final theorem while missing the logical boundary between checked reductions, external hardness premises, conditional sampling premises, and explanatory prose.

1.20 Script companion coverage for HAETAE Assumptions

`HAETAE_Assumptions.ec` defines the formal hardness surfaces and the checked adapters between the bimodal self-target relation and Module-SIS. It does not estimate concrete MLWE or Module-SIS security. Its role is to give the rest of the proof typed games and exact interface adapters.

Script item		Mathematical and checker role
<code>mlwe_instance,</code> <code>real, dmlwe_random</code>	<code>dmlwe_</code>	Abstract MLWE instance space and two lossless distributions. The script does not define their concrete algebraic distribution; it exposes a typed distinguishing game surface. Losslessness is checked from the declared <code>[lossless]</code> annotations.

<code>module_sis_instance,</code> <code>module_sis_solution,</code> <code>module_sis_valid</code>	Module-SIS instances are pairs (A, t) , solutions are <code>polyvec1</code> vectors, and validity is the equation $A \cdot s = t$ plus well-formedness. The lemmas around <code>module_sis_eval</code> and <code>module_sis_zero_valid</code> are checked algebraic sanity facts.
<code>bimodal_selftarget_msis_instance</code> and solution aliases	The bimodal self-target relation is represented by the same carrier types as Module-SIS in this model. This is a formal interface decision, not a cryptanalytic theorem.
<code>bimodal_to_module_sis_instance,</code> <code>bimodal_to_module_sis_solution,</code> <code>bimodal_to_module_sis_valid</code>	Identity adapter from bimodal self-target MSIS to Module-SIS. EasyCrypt checks that validity is preserved by unfolding definitions.
<code>MLWE_Distinguisher,</code> <code>MLWE_Real_Game,</code> <code>MLWE_Random_Game</code>	Classical distinguishing surface for MLWE. These games sample from the abstract distributions and call an adversary. No hardness bound is proved here; bounds enter later as premises or as <code>mlwe_hardness_term</code> .
<code>ModuleSIS_Solver</code> and <code>ModuleSIS_Relation_Game</code>	Relation-game surface for a solver that outputs an optional Module-SIS solution. Success means <code>sol <> None</code> and <code>module_sis_valid</code> .
<code>BimodalSelfTargetMSIS_Solver</code> and relation game	Analogous relation-game surface for the bimodal self-target relation. The success event is later related exactly to Module-SIS through the identity adapter.
<code>bimodal_solver_lift_exact</code> and <code>bimodal_mode_solver_lifted_distribution_exact</code> <code>MLWE_Reduction,</code> <code>BimodalSelfTargetMSIS_Reduction,</code> <code>ModuleSIS_Reduction</code> <code>MLWE_Game,</code> <code>BimodalSelfTargetMSIS_Game,</code> <code>ModuleSIS_Game</code> <code>BimodalMSIS_As_ModuleSIS</code> and <code>bimodal_msis_as_module_sis_exact</code>	<code>byequiv</code> proofs showing that solving the bimodal relation and then viewing the result as Module-SIS has the same success probability as the lifted Module-SIS game. These are checked equivalences, not hardness estimates. One-procedure reduction interfaces used by the final theorem family. They are opaque modules whose success probabilities are bounded outside this file. Thin wrappers around the corresponding reduction modules. They exist so that security theorems can state probabilities uniformly. Final exact adapter used in <code>HAETAE_Security.ec</code> . The proof is a <code>byequiv</code> that calls the same underlying module on both sides.

The assumption boundary is therefore exact: checked material in this file includes type-correct games, adapter definitions, algebraic validity lemmas, losslessness of declared distributions, and equivalence lemmas for adapters. External material includes the actual hardness of MLWE, Module-SIS, and the bimodal self-target relation. The final security file may assume or compose bounds for `Pr[MLWE_Game(...).main():res]` and `Pr[ModuleSIS_Game(...).main():res]`, but those bounds are not derived in `HAETAE_Assumptions.ec`.

1.21 Script companion coverage for HAETAE Transcript

HAETAE_Transcript.ec defines the transcript vocabulary used by random-oracle programming and special soundness. It is the bridge between a signature object and the public proof transcript extracted from that signature.

Transcript family	Mathematical reading
<code>transcript</code>	An eight-component tuple consisting of public key, message, context, commitment high bits, commitment low bits, message hash, challenge, and signature.
Projection operations	<code>transcript_pk</code> , <code>transcript_message</code> , <code>transcript_context</code> , <code>transcript_commitment_highbits</code> , <code>transcript_commitment_lowbits</code> , <code>transcript_message_hash</code> , <code>transcript_challenge</code> , and <code>transcript_signature</code> are tuple projections. Their role is readability and stable proof targets.
<code>transcript_of_signature</code>	Builds a transcript from a signature by extracting the commitment, message-hash, and challenge fields from the signature. This is used whenever signing games append a transcript to their log.
<code>transcript_challenge_query</code> and <code>transcript_signature_challenge_query</code>	Two views of the challenge random-oracle query: one from transcript fields and one from signature fields. Valid transcripts make them equal.
<code>transcript_verifies</code> , <code>transcript_fields_match_signature</code> , <code>transcript_valid</code>	Validity means the signature verifies, the signature is well-formed, and the stored transcript fields match the signature fields. This is the predicate used by ROM programming and special-soundness lemmas.
<code>transcript_challenge_matches</code> and <code>transcript_signature_challenge_matches</code>	Relate a random-oracle output to the transcript or signature challenge. Valid transcripts let the proof replace one predicate by the other.
<code>same_fork_target</code> and <code>distinct_fork_challenges</code>	Forking-pair structure: two transcripts must share public key, message, context, commitment high bits, commitment low bits, and message hash, while having different challenges.
<code>valid_forking_pair</code>	The exact precondition for extraction: same target, distinct challenges, and both transcripts valid. This avoids the logical error of claiming extraction from one accepting transcript.
<code>forking_difference_solution</code> , <code>fork_module_matrix</code> , <code>fork_module_target</code>	Build the bimodal self-target MSIS instance and candidate solution from a valid pair of transcripts. The solution is the response difference.

<code>extract_bimodal_</code> <code>selftarget_msis_solution</code> and <code>extract_module_sis_</code> <code>solution</code>	Partial extractors that return <code>None</code> unless the pair is valid. The Module-SIS extractor applies the adapter from <code>HAETA_E_Assumptions.ec</code> .
Honest-transcript operations	<code>transcript_from_honest_signing</code> , <code>honest_transcript_verifies</code> , and <code>honest_transcript_challenge</code> connect deterministic internal signing to the transcript vocabulary consumed by ROM programming.

The lemma families in this file are proof-script glue for downstream scripts.

Lemma family	What EasyCrypt checks
Signature-field consistency	<code>transcript_fields_match_signature_challenge_query</code> , <code>transcript_fields_match_signature_challenge_matches</code> , <code>transcript_valid_signature_challenge_query</code> , <code>transcript_valid_signature_challenge_matches</code> , and <code>transcript_valid_signature_challengeE</code> unfold definitions to show that valid transcript fields and signature fields agree.
Public-equation and reconstruction lemmas	<code>transcript_valid_public_equation</code> , <code>transcript_valid_commitment_reconstruction</code> , <code>transcript_reconstructionE</code> , and <code>transcript_active_reconstructionE</code> justify using verification equations as public reconstruction equations in special-soundness reasoning.
Forking-pair query and challenge lemmas	<code>valid_forking_pair_challenge_query</code> , <code>valid_forking_pair_signature_challenge_query</code> , and <code>valid_forking_pair_distinct_signature_challenges</code> convert the abstract forking-pair predicate into equal challenge-query sites and distinct challenge values.
Extraction obligation lemmas	<code>extract_bimodal_forking_some</code> , <code>extract_bimodal_forking_nonempty</code> , <code>bimodal_forking_solution_valid</code> , and <code>forking_extraction_obligation_holds</code> prove that a valid forking pair produces a valid bimodal solution.
Module-SIS lift obligations	<code>bimodal_to_module_sis_lift_obligation_holds</code> , <code>module_sis_extraction_from_bimodal</code> , and <code>special_soundness_from_bimodal_lift</code> package the adapter from bimodal extraction to Module-SIS extraction.
Public reconstruction obligations	<code>fork_public_reconstruction_obligation_holds</code> and <code>special_soundness_with_public_reconstruction_from_bimodal_lift</code> record that public reconstruction accompanies extraction when later proofs need both facts.

The checked content here is definitional and algebraic: projections, tuple equality, validity propagation, extraction from a valid pair, and adapter validity. The explanatory boundary is important. This file does not prove a forking lemma that produces two transcripts from an adversary; it only states and checks what follows once a valid forking pair is available.

1.21.1 Lemma-ledger companion for HAETAE Transcript

Lemma-ledger criticism for HAETAE Transcript. The preceding companion section explains the purpose of `HAETAE_Transcript.ec`, but by itself it is still not sufficient for a mathematician or cryptologist to reconstruct the proof obligations in that script. The evidence is in the script structure: lines 50–112 define several different challenge-query, matching, validity, and forking predicates, while lines 470–604 turn those predicates into extraction and special-soundness obligations. Without a ledger, a reader can easily conflate a checked definition such as `valid_forking_pair`, a theorem-boundary predicate such as `special_soundness_obligation`, and the external cryptographic meaning of the Module-SIS hardness game imported through `HAETAE_Assumptions.ec`. The report plus script can support reconstruction only if the reader is told which operators are pure transcript syntax, which lemmas are checked algebraic consequences, and which adversarial production of forks is left to downstream HopGames reductions.

EasyCrypt item and lines	Mathematical statement	Category	Premises and imports	Downstream role
AllCore, List, HAETAE_Params, HAETAE_Algebra, HAETAE_ROM, HAETAE_Assumptions; 1–9	Imports the tuple/list environment, HAETAE algebra, ROM query vocabulary, and SIS adapter vocabulary used by all transcript predicates.	Imported/library fact; external-assumption boundary for the hardness-side interfaces.	No local proof; relies on imported EasyCrypt and project theories.	Makes transcript extraction compatible with HopGames and the final HAETAE_Security.ec reduction chain.
transcript; 11–12	Models a transcript as public key, message, context, commitment high bits, commitment low bits, message hash, challenge, and signature.	Machine-checked fact.	Imports HAETAE carrier types from algebra and parameters.	Carrier for transcript logs, forking games, and special-soundness extraction in HAETAE_HopGames.ec.
signature_commitment_highbits, signature_commitment_lowbits, signature_message_hash, signature_challenge; 14–29	Defines the transcript-visible fields extracted from a signature, ignoring mode, key, message, and context except for typing.	Machine-checked fact.	Uses <code>sig_*</code> projections from the algebraic signature model.	Lets later proofs compare stored transcript fields with signature fields without unpacking signatures repeatedly.

transcript_pk through transcript_ signature, transcript_ of_signature; 31–48	Gives tuple projections and the canonical tran- script obtained from a public signature.	Machine- checked fact.	Definitional tuple projections plus the signature- field operators above.	Used by signing-record logs and ROM program- ming sites in HopGames.
transcript_ challenge_query, transcript_ signature_ challenge_query; 50–63	Builds the challenge random-oracle query from either stored transcript fields or the embedded signature fields.	Machine- checked fact.	Uses challenge_ hash_query from HAETAE_ROM.ec.	Supplies the query equality needed by ROM pro- gramming and valid- fork query alignment.
transcript_ verifies, transcript_ challenge_ matches, transcript_ signature_ challenge_ matches, transcript_ fields_match_ signature, transcript_ valid, transcript_ public_equation; 65–95	Defines verification, output-challenge match- ing, field consistency, full transcript valid- ity, and the public verification equation.	Machine- checked fact.	Uses verify_ internal, valid_ signature, ro_challenge_ hash, and verify_public_ equation.	Defines the invariant con- sumed by valid-forking- pair and transcript-log soundness lemmas in HopGames.
same_fork_ target, distinct_ fork_challenges, valid_forking_ pair; 97–112	A valid fork is two valid transcripts for the same public target with dis- tinct challenges.	Conditional premise and machine- checked predi- cate.	Requires both transcripts to sat- isfy transcript_ valid.	Primary pre- condition for extraction games; no lemma here proves that an adversary produces such a pair.

transcript_ response, transcript_ response_aux, transcript_ public_key_ challenge_term, active_public_ reconstruction, inactive_public_ reconstruction; 114–129	Extracts response com- ponents and the public- key challenge contribu- tion used in verification reconstruction.	Machine- checked fact.	Imports signa- ture response projections, public_key_ challenge_term, and polyveck_ zero.	Feeds public- reconstruction lemmas and special- soundness obligations.
forking_ difference_ solution, fork_response_ aux_difference, fork_public_ challenge_ relation, fork_response_ aux_relation, fork_left_ active_public_ challenge_ relation, fork_right_ active_public_ challenge_ relation, fork_public_ reconstruction_ cases; 131–166	Defines the response dif- ference and public recon- struction relations ob- tained from two forked transcripts.	Machine- checked fact.	Uses vector sub- traction/addition and public-key challenge terms from algebra.	Local alge- braic vocabu- lary for later extraction and reconstruction obligations.

fork_matrix_ source, fork_ module_matrix, fork_module_ target, bimodal_ selftarget_ msis_instance_ of_fork, extract_bimodal_ selftarget_msis_ solution, module_ sis_instance_ of_fork, extract_module_ sis_solution; 168–215	Constructs the bi- modal self-target MSIS instance, candidate so- lution, and Module-SIS adapter output from a valid fork.	Machine- checked fact with external- assumption boundary for the hardness game.	Uses determin- istic matrix generation and bimodal_to_ module_sis_* from HAETAE_ Assumptions.ec.	Provides the objects re- turned by HopGames special- soundness and Module- SIS reduction modules.
bimodal_ extraction_ success; 216–220	States that bimodal extraction returns a nonempty solution and that the solu- tion validates for the fork-derived instance.	Theorem- boundary predicate.	Depends on the extractor and bimodal_ selftarget_ msis_valid.	Used to pack- age extraction success be- fore lifting to Module-SIS.
extract_bimodal_ forking_some, extract_ bimodal_forking_ nonempty; 222–238	A valid forking pair makes the bimodal ex- tractor return exactly the response difference, hence not None.	Machine- checked fact.	Premise: valid_forking_ pairmdtr1tr2.	Internal support for forking_ extraction_ obligation_ holds.
transcript_ fields_match_ signature_ challenge_*, transcript_ valid_signature_ challenge_*; 240–288	Field consistency and transcript validity make the stored chal- lenge query, signature challenge query, and challenge-match predi- cates agree.	Machine- checked fact.	Premises: transcript_ fields_match_ signature or transcript_ valid.	Required by ROM pro- gramming to switch between tran- script and signature views of the programmed challenge site.

transcript_ valid_public_ equation, transcript_ valid_ commitment_ reconstruction; 299–331	A valid transcript satisfies the public verification equation and reconstructs the stored commitment high bits.	Machine-checked fact.	Unfolds <code>verify_internal</code> , <code>verify_public_equation</code> , and <code>transcript_validity</code> .	Connects transcript validity to public reconstruction used in special soundness.
valid_forking_ pair_*; 333–461	Forking pairs have public equations, common challenge-query sites, distinct signature challenges, equal reconstructed high bits, and public challenge relations.	Machine-checked fact family.	Premise: <code>valid_forking_pair</code> ; relies on the field-query and public reconstruction lemmas above.	Supplies the proof obligations consumed by HopGames’ special-soundness interface readiness lemmas.
extraction_ success, forking_ extraction_ obligation, fork_public_ reconstruction_ obligation; 470–484	Defines the Module-SIS extraction-success predicate and the two universal obligations over all valid forking pairs.	Conditional premise and theorem boundary.	Quantifies over <code>valid_forking_pair</code> ; uses Module-SIS validity.	Named assumptions/-targets for downstream interface readiness; they are not adversarial fork-generation lemmas.
forking_ difference_ solution_wf, fork_module_ matrix_rows_wf, fork_module_ matrix_size, bimodal_forking_ solution_valid, forking_ extraction_ obligation_ holds, fork_public_ reconstruction_ obligation_ holds; 486–527	Proves well-formedness, matrix size, bimodal solution validity, and the two local fork obligations.	Machine-checked fact.	Uses algebraic well-formedness, <code>mode_k</code> , and previous fork lemmas.	Discharges the transcript-side premises used by HopGames special-soundness wrappers.

<code>bimodal_to_module_sis_lift_obligation, bimodal_to_module_sis_lift_obligation_holds;</code> 535–549	States and proves that every valid bimodal self-target solution lifts to a valid Module-SIS solution under the imported adapter.	Machine-checked fact, imported-interface boundary.	Uses <code>bimodal_to_module_sis_valid</code> from assumptions.	Bridge from bimodal extraction to the Module-SIS game used in final security.
<code>special_soundness_obligation, special_soundness_with_public_reconstruction_obligation, module_sis_extraction_from_bimodal, special_soundness_*</code> ; 551–604	Packages bimodal extraction plus the lift obligation into Module-SIS special-soundness, optionally preserving public reconstruction.	Machine-checked fact; theorem boundary.	Premises are the fork, lift, and public-reconstruction obligations explicitly listed in the lemmas.	Used by HopGames to certify the special-soundness interface before composing with NMA/CMA reductions.
<code>transcript_from_honest_signing, honest_transcript_verifies, transcript_of_signature_*, honest_transcript_challenge;</code> 607–660	Builds the transcript produced by honest signing and proves it verifies, has fields matching the signature, and carries the expected challenge hash.	Machine-checked fact.	Uses <code>sign_internal, verify_internal, sign_internal</code> , and signature field projections.	Supports signing-record soundness, min-entropy no-failure facts, and transcript logging in ROM programming and HopGames.

1.22 Script companion coverage for HAETAE ROM Programming

`HAETAE_ROM_Programming.ec` defines the counted random-oracle programming surface consumed by `HAETAE_HopGames.ec`. It is where transcript challenge sites become programmable ROM sites and where bad-event flags are named.

ROM-programming object	Mathematical reading
<code>programming_site</code>	A pair (q, y) consisting of a random-oracle query and the output to program at that query.

programming_site_of_transcript, signature_programming_site_of_transcript, actual_signature_programming_site	Ways to identify the challenge site induced by a transcript or by the signature fields inside that transcript. Validity lemmas show the transcript and signature views agree.
honest_signing_programming_site	The challenge site generated by honest signing coins. This is the random site whose freshness is needed for safe ROM programming.
programmed_site_query_min_entropy_bound	Point-probability bound for honest programming-site queries over signing coins. This is the min-entropy input used for prequery bounds.
programming_site_matches_transcript and reprogramming_failure_challenge_entropy_support_ok, min_entropy_failure, transcript_log_min_entropy_clear	A programmed site is correct for a transcript when it has the transcript's challenge query and output matching the transcript challenge. Failure is the negation of that match. Challenge-support predicates. They are used to prove that honest transcripts do not trigger the min-entropy bad flag.
fs_with_aborts_failure and transcript_log_signature_programming_sites_clear	Legacy Fiat-Shamir-with-aborts failure combines reprogramming failure and min-entropy failure. The clear-log predicate says no transcript in the signing log fails at its actual signature programming site.
counted_rom_event and counted_trace_*	Event vocabulary for hash queries, signature sites, programmed sites, and min-entropy checks. These are counters and traces, not probability bounds by themselves.
ro_query_is_challenge and challenge_query_log_count	Classifies query constructors and counts distinct challenge queries in a log. Message, matrix-expansion, and sampler-expansion queries are not challenge queries.
programming_site_prequeried, programming_site_reprograms, programming_site_fresh	The three core bad-event predicates: a query was already asked, a site conflicts with an earlier programmed output for the same query, or neither bad event holds.
rom_counted_programming_bad and rom_counted_state_bad	Boolean summary of the counted bad state: min-entropy failure, prequery, or reprogramming conflict.

The stateful ROM module is central to the downstream proof.

CountedLazyROM method	state or Meaning
hash_queries	Log of queries passed to <code>get</code> . Challenge-query budgets are derived from this log.
programmed_sites	Sites passed to <code>program</code> . Reprogramming is detected by comparing a new site against this list.
signature_sites	Sites observed from signing transcripts. Hop games use this to connect transcript validity and ROM programming.

<code>bad_prequery</code>	Set when <code>program(site)</code> is called after the site's query already appears in <code>hash_queries</code> .
<code>bad_reprogram</code>	Set when <code>program(site)</code> conflicts with a previously programmed output for the same query.
<code>bad_min_entropy</code>	Set when observing a transcript whose challenge is outside the expected sparse support.
<code>init</code> , <code>get</code> , <code>program</code> , <code>observe_signature_site</code> , <code>observe_transcript</code> , <code>bad</code>	<code>init</code> clears logs and flags; <code>get</code> logs a query; <code>program</code> sets prequery/reprogram flags and calls the underlying oracle's <code>set</code> ; <code>observe_transcript</code> records the induced signature site and updates the min-entropy flag; <code>bad</code> returns the disjunction of the three flags.

The lemma families needed by `HAETAE_HopGames.ec` and `HAETAE_Security.ec` are:

Lemma family	Proof-script reading
CountedLazyROMStateFacts	Hoare facts showing that <code>init</code> clears flags, <code>get</code> preserves clear flags or min-entropy clear state, <code>program</code> preserves bad-clear state when a site is fresh, and <code>bad</code> is false when all flags are clear. These are state invariants, not loss bounds.
Programming-site self-match lemmas	<code>programming_site_self_matches</code> , <code>programming_site_self_no_reprogramming_failure</code> , and <code>fs_with_aborts_failureE</code> unfold the definitions connecting a transcript, an oracle output, and a programming site.
Challenge-query counting lemmas	<code>challenge_query_log_count_cons_le</code> and the nonchallenge-cons lemmas show how query logs affect the counted challenge budget. These facts are used inside budgeted HopGames invariants.
Honest-site entropy and spread lemmas	<code>honest_signing_programming_site_query_entropy_token</code> , <code>signing_distribution_programming_site_query_spread</code> , and <code>honest_signing_programming_site_query_spread_bound</code> reduce programming-site min-entropy to the signing-coin token-spread assumption from the distribution layer.
Prequery probability bounds	<code>programmed_site_prequery_one_step_*</code> , <code>direct_signing_coin_sampler_prequery_bound</code> , and <code>direct_signing_coin_sampler_message_hash_prequery_bound</code> bound the probability that a freshly sampled signing site was already queried. These are the local ingredients for counted-ROM prequery terms.
Reprogramming conflict lemmas	<code>honest_signing_programming_sites_no_conflict</code> , <code>programming_site_log_conflict_free_for_honest_*</code> , <code>programmed_site_reprogram_one_step_conflict_free_zero</code> , and <code>programmed_site_prequery_reprogram_one_step_concrete_bound</code> show that honest sites do not conflict and combine reprogramming with prequery bounds.

Counted bad-flag budget lemmas	<code>counted_lazy_rom_bad_flags_budget_sound</code> , <code>counted_lazy_rom_bad_flags_query_budget_bound</code> , and <code>counted_lazy_rom_bad_flags_query_budget_bound_subunit</code> express the condition under which the probability of <code>CountedLazyROMBadGame</code> is charged to <code>counted_rom_programming_loss_term</code> .
Transcript no-failure lemmas	<code>honest_transcript_no_min_entropy_failure</code> , <code>signing_transcript_relation_no_min_entropy_failure</code> , <code>signing_record_log_sound_transcripts_no_min_entropy</code> , and <code>fs_with_aborts_no_failure_*</code> explain why honest signing records do not trigger the legacy failure predicate.
Valid-transcript programming-site lemmas	<code>transcript_valid_signature_programming_site_matches</code> , <code>transcript_valid_actual_signature_programming_site_matches</code> , and <code>valid_forking_pair_programming_site_query</code> connect transcript validity and forking pairs to identical programming-query sites.
Bound-obligation lemmas	<code>counted_rom_programming_bound_obligation_holds</code> and <code>fs_with_aborts_bound_obligation_holds</code> prove only nonnegativity of the named loss terms. They do not prove that the loss is small; the reductions file records separately that the old rejection ledger is vacuous.

The checked content in `HAETAE_ROM_Programming.ec` is therefore a set of definitions, Hoare invariants, and probability bounds for explicit bad events. The file does not justify replacing SHAKE by a random oracle, does not give a concrete MLWE/SIS estimate, and does not by itself complete the final EUF-CMA theorem. It supplies the exact bad-event vocabulary and counted-loss interfaces that HopGames and Security compose.

1.22.1 Lemma-ledger companion for HAETAE ROM Programming

Lemma-ledger criticism for HAETAE ROM Programming. The existing ROM-programming companion describes the main object families, but it still leaves a reconstruction gap. The script is not merely a dictionary of bad-event names: it defines a stateful counted ROM module at lines 250–301, Hoare invariants at lines 319–500, min-entropy and prequery bounds at lines 617–1386, bad-flag budget wrappers at lines 1455–1478, and valid-transcript programming-site closure lemmas at lines 1639–1723. A reader who sees only the family names cannot determine which statements are checked probability bounds, which are conditional on query-budget hypotheses, which are formal ROM facts, and which are external SHAKE/FIPS modeling assumptions. The ledger below therefore separates formal ROM programming from implementation hash assumptions and from the final cryptographic hardness composition.

EasyCrypt and lines	item	Mathematical statement	state-	Category	Premises and imports	and role	Downstream
------------------------	------	---------------------------	--------	----------	-------------------------	-------------	------------

AllCore, Real, Distr, List, FSet, PROM, StdOrder, Mu_mem, and local imports; 1–16	Imports probability, finite-set, random-oracle, transcript, rejection, and loss-term infrastructure.	Imported/library fact; external ROM-assumption boundary.	No local proof; relies on EasyCrypt libraries and project theories.	Defines the formal ROM environment used by HopGames and indirectly by HAETAE_Security.ec.
programming_site, programming_site_query, programming_site_output; 18–22	A programmable site is a pair of a ROM query and the output to install at that query.	Machine-checked fact.	Uses ro_query and ro_output from HAETAE_ROM.ec.	Base type for every prequery, reprogramming, and transcript-site predicate.
programming_site_of_transcript, signature_programming_site_of_transcript, transcript_signature_challenge_output, actual_signature_programming_site; 23–37	Builds the challenge programming site from transcript fields, signature fields, or the signature’s own challenge output.	Machine-checked fact.	Depends on transcript challenge queries and ro_output_of_challenge.	Used when HopGames switches between transcript and signature views of a programmed challenge.
honest_signing_programming_site, honest_signing_programming_site_query; 39–50	Defines the programming site induced by an honest signing transcript generated from secret key, message, context, and signing coins.	Machine-checked fact.	Uses transcript_from_honest_signing.	Source of the randomly sampled site in prequery and reprogramming-loss lemmas.
programmed_site_query_min_entropy_bound_for, programmed_site_query_min_entropy_bound; 51–60	States that every query in a log has at most a given point probability as an honest signing programming-site query.	Conditional premise.	Uses mu over a signing-coin distribution; default instance is drandom_coins.	Input condition for one-step prequery probability bounds in HopGames.

programming_ site_matches_ transcript, reprogramming_ failure, challenge_ entropy_support_ ok, min_entropy_ failure; 65–77	Defines correct tran- script programming, reprogramming fail- ure, sparse challenge support, and the min- entropy bad event.	Machine- checked predi- cate; external modeling boundary for entropy support.	Uses tran- script challenge matching and challenge_ sparse.	Bad-event vocabulary for programmed transcript games.
transcript_ log_min_ entropy_clear, signing_record_ log_min_entropy_ clear, fs_with_ aborts_failure, transcript_ signature_ programming_ site_clear, transcript_ log_signature_ programming_ sites_clear, fs_with_ aborts_bound_ obligation; 80–99	Lifts bad-event pred- icates to transcrip- t/signing logs and names the legacy Fiat- Shamir-with-aborts loss-obligation nonnega- tivity check.	Machine- checked predi- cate; theorem- boundary obligation.	Depends on re- programming and min-entropy predicates plus reduction loss terms.	Used by HopGames transcript-log invariants; one clear-log predicate is also visible in HAETAE_ Security.ec.
counted_rom_ event and counted_event_ is_*; 103–131	Defines counted events for hash queries, signa- ture sites, programmed sites, and min-entropy checks, with boolean classifiers.	Machine- checked fact.	Pure local sum- type analysis.	Audit vo- cabulary for counted ROM traces and bad-flag explanations.
ro_query_is_ challenge, challenge_ query_log_count, challenge_query_ log_count_*; 133–192	Classifies challenge-hash queries and proves how consing challenge or nonchallenge queries changes the distinct challenge-query count.	Machine- checked fact; query-budget invariant.	Uses finite-set cardinality over filtered query logs.	Discharges HopGames budget obli- gations for message-hash, matrix, sam- pler, and challenge query steps.

<code>counted_trace_*</code> ; 193–202	Counts the four counted event families by filtering a trace.	Machine-checked fact.	Uses list filtering and event classifiers.	Provides trace-accounting vocabulary, though most downstream use goes through higher-level bad-flag lemmas.
<code>programming_sites_</code> , <code>same_query</code> , <code>programming_sites_conflict</code> , <code>programming_site_prequeried</code> , <code>programming_site_reprograms</code> , <code>programming_site_fresh</code> , <code>programming_transcript_fresh</code> ; 205–234	Defines prequery, reprogramming conflict, freshness, and transcript freshness.	Machine-checked predicate.	Depends only on query equality, output inequality, query logs, programmed-site logs, and min-entropy failure.	Core bad-event guards for ROM programming games.
<code>rom_counted_programming_bad</code> , <code>rom_counted_state_bad</code> , <code>counted_rom_programming_bound_obligation</code> ; 235–247	Summarizes min-entropy, prequery, and reprogramming bad states and names the nonnegativity obligation for the counted loss term.	Machine-checked predicate; theorem-boundary obligation.	Uses <code>counted_rom_programming_loss_term</code> .	Connects local bad flags to the counted-ROM loss term later composed by HopGames and Security.
<code>CountedLazyROM</code> ; 250–301	Stateful wrapper over an oracle with logs for hash queries, programmed sites, signature sites, and flags for prequery, reprogramming, and min-entropy bad events.	Formal ROM modeling fact; module/procedure boundary.	Requires an <code>Oracle</code> ; uses <code>0.get</code> and <code>0.set</code> .	Instantiated inside HopGames to expose counted bad events. It is not a SHAKE implementation theorem.

SigningCoinSampler, 302–313	Abstracts coin sampling gives the direct sampler from <code>drandom_coins</code> .	Machine-checked module interface.	Uses <code>drandom_coins</code> and its imported distribution facts.	Target of <code>phoare</code> prequery bounds used in sampled signing games.
CountedLazyROMState, 315–500	Facts invariants: initialization clears state, <code>get</code> preserves clear flags and nonchallenge budgets, <code>observe_transcript</code> preserves clear state when min-entropy is clear, <code>program</code> preserves clear state when fresh, and <code>bad</code> is false under clear flags.	Machine-checked fact.	Premises include freshness, nonchallenge-query classification, and min-entropy-clear conditions.	Supplies HopGames state-invariant steps for counted ROM hybrids.
CountedROMInterface, CountedROMClient, CountedLazyROMBadGame, ProgramChallenge, TranscriptProgrammer, and TranscriptProgrammerFromSite, 501–549	Defines the client interface, bad-game wrapper, direct challenge-programming procedure, and transcript-to-site programming procedure.	Formal ROM modeling fact; module/procedure boundary.	Depends on <code>CountedLazyROM</code> and the generic oracle interface.	Gives the game API whose bad probability is bounded by later lemmas.
<code>programming_site_self_*</code> , <code>fs_with_aborts_failureE</code> , <code>programming_site_prequeried_*</code> , <code>honest_signing_programming_site_queryE</code> ; 550–610	Unfolds self-matching, failure decomposition, prequery preservation, and the honest signing challenge query.	Machine-checked fact family.	Uses definitions from transcript, ROM, and honest signing.	Local rewriting library for later no-failure, query-budget, and prequery bounds.

programmed_site_ entropy_token_ from_query, honest_signing_ programming_ site_query_ entropy_token, honest_signing_ programming_ site_query_ token_injective, signing_ distribution_ programming_ site_query_ spread; 611–671	Extracts an entropy token from a challenge query and bounds the probability that a signing distribution hits any fixed programming-site query.	Machine-checked fact; conditional premise for arbitrary distributions.	Needs signing_randomness_token_spread for arbitrary distributions and the signing-coin token lemmas from distributions.	Min-entropy engine for prequery loss.
honest_signing_ programming_ site_query_ spread, honest_signing_ programming_ site_query_ spread_bound, honest_signing_ programming_ site_query_ spread_ bound_for, honest_signing_ programming_ site_query_is_ challenge; 673–713	Instantiates the query-spread bound to drandom_coins and proves honest programming queries are challenge queries.	Machine-checked fact.	Uses signing_coin_distribution_token_spread.	Turns entropy assumptions into concrete challenge-query budget bounds.

programming_ site_prequeried_ honest_*, honest_signing_ programming_ site_outputE, honest_signing_ programming_ site_functional_ output, honest_signing_ programming_ sites_no_ conflict, actual_ signature_ programming_ site_*; 713–883	Shows nonchallenge message-hash consing does not create honest- site prequeries, honest sites have functional outputs, honest sites do not conflict, and actual signature sites equal honest signing sites.	Machine- checked fact.	Unfolds sign_internal , challenge query construction, and signature-field definitions.	Supports message-hash- aware pre- query bounds and no- reprogramming arguments in HopGames.
programming_ site_ conflict_free, programming_ site_log_ conflict_free_ for_honest, programming_ site_conflict_ free_no_ reprograms, actual_ signature_ programming_ site_sign_ internal_ conflict_step, programmed_site_ reprogram_one_ step_conflict_ free_zero; 884– 968	Defines conflict-free programmed-site logs and proves honest sites remain conflict- free, making one-step reprogramming prob- ability zero under the conflict-free premise.	Machine- checked fact; conditional premise for the zero- probability lemma.	Premise: all honest sites are conflict- free against the current log.	Separates reprogram- ming loss from prequery loss in the counted-ROM argument.

programmed_site_prequery_one_step_*; 969–1230	Bounds the probability that a sampled honest site is already in a query log, using either finite-set cardinality, total budget, challenge-query budget, or concrete counter hypotheses.	Machine-checked probability bound; query-budget invariant.	Premises include min-entropy bounds, cardinality bounds, and $hc \leq \text{hash_query_budget_count}$.	Main prequery-loss family used by HopGames sampled-signing hybrids.
programmed_site_prequery_one_step_message_hash_counter_bound, direct_signing_coin_sampler_*; 1231–1351	Lifts the prequery bound through a leading message-hash query and into phoare statements for the direct signing-coin sampler.	Machine-checked probability bound.	Premises: nonnegative and bounded challenge-query counter plus query-count hypothesis.	Used when HopGames samples signing coins after message-hash interactions.
programmed_site_prequery_one_step_concrete_bound, programmed_site_prequery_reprogram_one_step_concrete_bound; 1353–1386	Combines concrete prequery and conflict-free reprogramming reasoning into a single one-step bad-event bound.	Machine-checked probability bound; conditional premise.	Requires a cardinality budget and conflict-free honest-site premise.	Local one-step bound feeding counted-ROM bad-event accounting.
programmed_site_query_min_entropy_bound_constant_impossible, programming_site_reprograms_conflict_cons, rom_counted_programming_bad_*, counted_rom_programming_bound_obligation_holds; 1404–1448	Records sanity checks: constant low-entropy query generation contradicts a subunit bound; conflicting cons creates reprogramming; min-entropy/prequery conditions imply the counted bad predicate; the counted loss term is nonnegative.	Machine-checked fact.	Uses signing-coin losslessness and nonnegativity of reduction loss terms.	Guards against vacuous min-entropy and bad-state accounting mistakes.

CountedLazyROMBadFlags, packages counted_lazy_ rom_bad_flags_ budget_sound, counted_lazy_ rom_bad_flags_*; 1455–1478	Proves the probability of CountedLazyROMBadGame and states that, under a budget-soundness premise, it is bounded by counted_rom_ programming_loss_ term and is subunit.	Conditional premise and machine- checked proba- bility wrapper.	Premise: counted_lazy_ rom_bad_flags_ budget_sound for the exact game probability.	Theorem boundary used by HopGames when charging bad events to the counted- ROM loss term.
fs_with_aborts_ no_programming_ failure, transcript_ signature_ challenge_ output_matches, honest_ transcript_ challenge_ entropy_support, honest_ transcript_ no_min_entropy_ failure, signing_ transcript_ relation_*, signing_record_ log_sound_*, fs_with_aborts_ no_failure_*; 1491–1624	Proves honest tran- scripts and sound signing records do not trigger min-entropy or legacy Fiat-Shamir- with-aborts failure when programming matches.	Machine- checked fact.	Uses signing tran- script relations, challenge_ from_seed_ sparse through the challenge- sparsity path, and transcript validity.	Used by HopGames to maintain clear tran- script logs and no-failure side conditions.

<code>transcript_ valid_ programming_ site_*, transcript_ valid_actual_ signature_ programming_ site_*, valid_forking_ pair_signature_ programming_ site_query, valid_ forking_pair_ programming_ site_query, valid_forking_ pair_self_ programming_ sites_no_ reprogramming; 1639–1723</code>	Valid transcripts make transcript and signature programming-site queries agree; valid forks share programming-site queries; self-programmed valid sites do not count as reprogramming failures.	Machine-checked fact.	Premises: <code>transcript_ valid, valid_ forking_pair, and challenge matches.</code>	Bridge from transcript special-soundness facts to ROM programming sites in HopGames.
<code>fs_bound_parts_ nonnegative, fs_with_ aborts_bound_ obligation_ holds; 1739–1745</code>	Proves nonnegativity of the legacy Fiat-Shamir-with-aborts reprogramming and min-entropy loss terms.	Machine-checked fact; theorem-boundary obligation.	Uses nonnegativity lemmas from <code>HAETAE_Reductions.ec</code> .	Allows downstream bounds to add these terms safely; it does not prove the old paper-style rejection ledger is tight or nonvacuous.

1.23 Feedback 3 criticism for the foundational dependency layer

The current report is still not enough for a mathematician or cryptologist to understand the EasyCrypt proof scripts from the report plus scripts alone at the layer below `HAETAE_HopGames.ec` and `HAETAE_Security.ec`. The missing material is not cosmetic. The hop and final-security files consume definitions from the formal HAETAE model as if they were ordinary mathematical objects: parameters, polynomial carriers, signature fields, random-oracle query constructors, signing-attempt distributions, rejection predicates, and concrete hash wrappers. Without a companion explanation for those files, a reader can see that a theorem compiles but cannot tell which part of the argument is an algebraic fact, which part is a model definition, which part is a conditional sampling premise, and which part is only explanatory idealization.

The main evidence from the scripts is as follows.

- `HAETAE_Security.ec` proves correctness through `verify_internal_sign_internal`, `valid_`

`signature_sign_internal`, and `verify_norm_ok_current`. Those facts are not security assumptions; they are checked algebra/model facts from `HAETAE_Algebra.ec`.

- `HAETAE_HopGames.ec` repeatedly samples from `drandom_coins`, `dsigning_attempt_state`, and `dexact_hyperball_signing_attempt_state`. A reader must know that the current “exact hyperball” distribution is a checked bounded-product carrier, with comments in `HAETAE_Distributions.ec` marking the paper-faithful hyperball/sphere sampler as remaining correspondence work.
- The rejection bridge uses conditional premises such as `structural_to_exact_hyperball_paper_sample_loss_obligation` in `HAETAE_HopGames.ec` and obligation families in `HAETAE_Rejection.ec`. Those are not proved by the top-level theorem; they are explicitly consumed as premises or discharged only after accepting the current checked carrier/loss boundary.
- The scheme module in `HAETAE_Scheme.ec` makes exactly three random-oracle query kinds in signing: sampler expansion, message hash, and challenge hash. The ROM programming proof relies on this query discipline. Without the `HAETAE_ROM.ec` query/output dictionary, the bad-event and query-budget lemmas are opaque.
- `HAETAE_Algebra.ec` imports `HAETAE_FIPS202.ec`, which imports `HAETAE_Keccak1600.ec`. These files provide checked byte-list and size facts for the formal model, not a standard-model proof that SHAKE is a random oracle.

This section closes that gap at the level needed to read the scripts. It does not claim that the report alone can replace line-by-line EasyCrypt source. The checked source remains authoritative.

Category	What belongs here in this layer	What must not be inferred
Machine-checked facts	Definitions, typing, losslessness, support, size, well-formedness, equality, Hoare, equivalence, and probability inequalities proved inside the manifest files.	They do not automatically prove equality with the HAETAE paper specification, with C/Jasmin code, or with a concrete security estimate.
External cryptographic assumptions	MLWE, Module-SIS, and bimodal self-target MSIS hardness surfaces from <code>HAETAE_Assumptions.ec</code> and real-valued hardness/loss terms from <code>HAETAE_Reductions.ec</code> .	The foundational files below do not estimate these assumptions; they only define the formal carriers and adapters used by the reductions.
Conditional premises	Sampling and hop premises such as structural-to-exact hyperball lifting and random-oracle reprogramming bounds when they appear as antecedents of a security theorem.	They must not be presented as already discharged unless a checked lemma in the same manifest proves the exact antecedent.

Imported/library facts	EasyCrypt distribution, list, finite-map, real-order, and random-oracle library facts, plus the local Keccak/FIPS wrapper files.	Imported facts are part of the verification environment; a standalone proof archive must include the version and import contract.
Explanatory prose	Descriptions such as “formal HAETAE model”, “bounded carrier”, and “paper-shaped sampler interface”.	Such prose is not a theorem. It guides reading and marks boundaries that the checked scripts either prove, assume, or leave conditional.

1.24 Script companion coverage for HAETAE Params

`HAETAE_Params.ec` is the numeric parameter base of the formal model. It has no cryptographic assumption and no probability theorem. Its machine-checked content is definitional arithmetic for modes and positivity facts consumed by algebra, distributions, rejection sampling, and final correctness.

Script object	Mathematical reading	Downstream role
<code>seedbytes</code> , <code>crhbytes</code> , <code>n</code> , <code>q</code> , <code>dq</code>	Global byte, polynomial-degree, and modulus constants. The checked fact <code>dqE</code> rewrites dq to 129026.	Used throughout the algebraic carriers, seed slicing, SHAKE wrappers, public key packing, challenge size, and distribution lengths.
<code>mode=[Mode2 Mode3 Mode5]</code>	The formal model has three parameter modes. There is no abstract well-formedness condition on modes beyond membership in this finite type.	<code>HAETAE_Scheme.ec</code> declares one abstract <code>haetae_mode</code> ; all later security statements are mode-parametric through this value.
<code>mode_k</code> , <code>mode_l</code> , <code>mode_tau</code>	Mode-dependent vector dimensions and sparse-challenge weight prefix.	Used by matrix/vector dimensions, challenge sparsity, transcript extraction, and Module-SIS/bimodal relation shapes.
<code>mode_b0sq</code> , <code>mode_b1sq</code> , <code>mode_b2sq</code>	Mode-dependent squared-norm bounds for secret, response, and verification checks.	Used by <code>secret_norm_ok</code> , <code>response_norm_ok</code> , and <code>verify_norm_ok</code> in <code>HAETAE_Algebra.ec</code> , then by rejection-failure events and correctness.

<code>mode_ln</code> , <code>mode_alpha_hint</code> , <code>mode_m</code>	Formal signing-sample bound, hint-decomposition base, and $m = l - 1$.	Used by bounded/hyperball carriers, decomposition lemmas, public-key verification columns, and rejection-sampling thresholds.
<code>mode_crypto_bytes</code> , <code>mode_polyq_packedbytes</code> , <code>mode_publickeybytes</code>	Formal byte-length constants for signature and public-key encodings.	Used to prove packing well-formedness and that the pack branch in rejection sampling is never the abort source in the current model.
Positivity and size lemmas	<code>mode_k_gt0</code> , <code>mode_l_gt0</code> , <code>mode_tau_le_n</code> , <code>mode_crypto_bytes_gt0</code> , and related lemmas are checked by mode case analysis.	They discharge side conditions for <code>dlist</code> , <code>mkseq</code> , support predicates, norm arithmetic, and real-probability nonnegativity proofs.

Boundary: the parameter file checks internal arithmetic consistency of the formal constants. It does not prove that these constants are the claimed HAETAE standard parameter sets or that they yield a concrete bit-security level.

1.24.1 Lemma-ledger companion for HAETAE Params

Lemma-ledger criticism for HAETAE Params. The previous companion coverage identified the parameter families, but it did not give enough detail for a reader to reconstruct which later EasyCrypt obligations are discharged merely by unfolding numeric constants and which claims are outside `HAETAE_Params.ec`. The gap matters because these constants are used everywhere: vector dimensions, challenge weight, norm bounds, byte lengths, and sampler supports all inherit their side conditions from this file. The script proves only finite-mode arithmetic facts about the formal constants. It does not prove correspondence with any external HAETAE standard document, does not justify a concrete security level, and does not introduce MLWE, Module-SIS, ROM, or sampler assumptions. The ledgers below make that boundary explicit.

Imported-interface and assumption ledger for HAETAE Params.

EasyCrypt object	Lines / category	Mathematical statement paraphrase	Local premises, imports, downstream role
<code>AllCore</code>	1. Imported/library fact.	Provides the base logic and arithmetic environment used by the parameter definitions and case-split proofs.	No local premise. All later imports of <code>HAETAE_Params.ec</code> inherit this base proof environment.
Formal parameter table	5–62. Machine-checked fact.	The file fixes one formal table of values for <code>Mode2</code> , <code>Mode3</code> , and <code>Mode5</code> .	The table is consumed by <code>HAETAE_Algebra.ec</code> , <code>HAETAE_Distributions.ec</code> , <code>HAETAE_Rejection.ec</code> , and the hop and security scripts through those imports.

External standard correspondence	HAETAE-	5–62.	External cryptographic assumption.	The file does not prove that the numeric table matches a particular HAETAE submission, standard, or concrete security claim.	Any such correspondence must be justified outside this EasyCrypt file. The checked security theorem is relative to these formal constants.
Cryptographic assumptions introduced locally		1–113.	External cryptographic assumption.	None are introduced in <code>HAETAE_Params.ec</code> .	MLWE, Module-SIS, Fiat-Shamir, ROM, rejection-sampling, and implementation equivalence assumptions enter only in later files.

Definition ledger for HAETAE Params.

EasyCrypt object(s)	Lines / category	Mathematical statement	Local premises, imports, downstream role
<code>seedbytes</code> , <code>crhbytes</code> , <code>n</code> , <code>q</code> , <code>dq</code>	5–9. Machine-checked fact.	Fixes seed length, CRH byte length, polynomial degree, modulus, and doubled modulus for the formal model.	No local premise. Used by algebraic carriers, SHAKE seed slices, coefficient-reduction facts, byte packing, and seed/CRH distributions.
<code>mode</code>	11. Machine-checked fact.	Declares the three formal modes <code>Mode2</code> , <code>Mode3</code> , and <code>Mode5</code> .	No separate validity predicate exists. <code>HAETAE_Scheme.ec</code> later exposes a single abstract <code>haetae_mode</code> ; downstream theorems are relative to that formal mode.
<code>mode_k</code> , <code>mode_l</code> , <code>mode_tau</code>	13–26. Machine-checked fact.	Defines per-mode public-vector dimension, secret-vector dimension, and sparse challenge prefix weight.	Used by <code>HAETAE_Algebra.ec</code> for vector/matrix and challenge shape, <code>HAETAE_Distributions.ec</code> for product-distribution lengths, and <code>HAETAE_Rejection.ec</code> for attempt-state dimensions.
<code>mode_b0sq</code> , <code>mode_b1sq</code> , <code>mode_b2sq</code>	28–41. Machine-checked fact.	Defines per-mode squared-norm thresholds for secret, response, and verifier norm checks.	Used by <code>secret_norm_ok</code> , <code>response_norm_ok</code> , and <code>verify_norm_ok</code> ; rejection and security scripts inherit these as the formal abort/correctness thresholds.
<code>mode_ln</code> , <code>mode_alpha_hint</code> , <code>mode_m</code>	43–50. Machine-checked fact.	Defines the signing-sample bound, hint decomposition base, and $m = l - 1$.	Used by bounded and hyperball sample carriers, decomposition lemmas, public verification columns, and rejection-sampling loss interfaces.

<code>mode_crypto_bytes</code> , <code>mode_polyq_packedbytes</code> , <code>mode_publickeybytes</code>	52–62. Machine-checked fact.	Defines formal signature/public-key byte lengths and per-polynomial q -packing lengths.	Used by public-key packing roundtrips in <code>HAETAE_Algebra.ec</code> and by rejection packing-abort facts in <code>HAETAE_Rejection.ec</code> .
---	---------------------------------	---	--

Lemma ledger for HAETAE Params.

EasyCrypt lemma(s)	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
<code>seedbytes_gt0</code> , <code>crhbytes_gt0</code> , <code>n_gt0</code> , <code>q_gt0</code> , <code>dqE</code>	65–69. Machine-checked fact.	Proves positivity for global sizes/modulus and rewrites <code>dq</code> to 129026.		No local hypotheses. Discharges nonempty list, distribution, modulus, and division side conditions in <code>HAETAE_Algebra.ec</code> and <code>HAETAE_Distributions.ec</code> .	
<code>mode_k_gt0</code> , <code>mode_l_gt0</code> , <code>mode_tau_gt0</code> , <code>mode_tau_le_n</code>	71–80. Machine-checked fact.	Every formal mode has positive vector dimensions and challenge weight bounded by n .		Premise is only <code>md:mode</code> ; proofs are by case analysis. Used by shape lemmas, sparse challenge support, distribution support, and HopGames dimension side conditions.	
<code>mode_b0sq_gt0</code> , <code>mode_b1sq_gt0</code> , <code>mode_b2sq_gt0</code> , <code>mode_ln_gt0</code> , <code>mode_alpha_hint_gt0</code> , <code>mode_m_gt0</code>	83–98. Machine-checked fact.	Proves positivity of formal norm thresholds, signing-sample bound, hint-decomposition base, and m .		Premise is <code>md:mode</code> . Used by norm proofs, bounded-sampler support, decomposition arithmetic, key-gen seed counters, and rejection threshold lemmas.	
<code>mode_crypto_bytes_gt0</code> , <code>mode_polyq_packedbytes_gt0</code> , <code>n_le_mode_polyq_packedbytes</code> , <code>mode_publickeybytes_gt0</code>	101–110. Machine-checked fact.	Proves positivity of byte-size interfaces and that a formal q -packed polynomial has at least n coefficient positions.		Premise is <code>md:mode</code> . Used by public-key packing/unpacking, verification-matrix reconstruction, and rejection packing-abort proofs.	

1.25 Script companion coverage for HAETAE Algebra

`HAETAE_Algebra.ec` is the formal HAETAE model core. It defines the carrier types and deterministic operations that the security games manipulate. The file is machine-checked, but its definitions are part of the model rather than a proof that the model is identical to the paper algorithm or an implementation.

Script object family	Mathematical reading	Downstream role
----------------------	----------------------	-----------------

Carrier types	<code>byte</code> , <code>seed</code> , <code>crh</code> , <code>random_coins</code> , <code>message</code> , <code>context</code> , <code>coeff</code> , <code>poly</code> , <code>challenge</code> , <code>polyveck</code> , <code>polyvecl</code> , <code>matrix</code> , <code>pkey</code> , <code>skey</code> , and <code>signature</code> are list- and tuple-based carriers.	These are the concrete type instantiations used when <code>HAETAE_Scheme.ec</code> clones <code>Sig_ROM.ec</code> . All games and reductions share these carriers.
Zero values and well-formedness	<code>poly_zero</code> , <code>vector zeroes</code> , <code>matrix_zero</code> , <code>poly_wf</code> , <code>polyveck_wf</code> , <code>polyvecl_wf</code> , <code>crh_wf</code> , <code>challenge_wf</code> , and unit/sparse predicates define local shape constraints.	Used by signature validity, transcript validity, distribution support, and sampling support lemmas.
Polynomial and vector operations	<code>coeff_mod</code> , <code>poly_add</code> , <code>poly_mul</code> , <code>poly_dot</code> , <code>matrix_vec_mul</code> , <code>polyveck_add</code> , <code>polyveck_sub</code> , and <code>polyvecl_sub</code> define the algebraic operations over the formal list model.	The checked <code>_wf</code> and coefficient-bound lemmas make later Module-SIS and verification equations well typed and well shaped.
Decomposition and public-key packing families	<code>coeff_decompose_z1_*</code> , <code>poly_highbits</code> , <code>poly_lowbits</code> , <code>polyveck_vk_highbits</code> , <code>polyveck_highbits_hint</code> , packing/unpacking operations, and roundtrip lemmas describe the formal byte/list encoding layer.	Used by public-key reconstruction, verification-matrix construction, and transcript public-equation lemmas. These are checked formal encodings, not a complete C/Jasmin parsing theorem.
FIPS/SHAKE seed flow	<code>haetae_shake256_bytes</code> , <code>haetae_xof256_*</code> , <code>haetae_keygen_rho_prime</code> , <code>haetae_keygen_sigma</code> , and <code>haetae_keygen_key</code> wrap <code>HAETAE_FIPS202.ec</code> .	Used to define formal key-generation seeds and public matrices. This is checked byte-list plumbing, not a random-oracle theorem about SHAKE.
Key-generation relation	<code>public_matrix_seed</code> , <code>public_secret_vector_seed</code> , <code>public_error_vector_seed</code> , <code>public_key_vector_seed</code> , <code>public_key_relation</code> , <code>public_key_equation_holds</code> , and <code>public_key_of_secret</code> define the formal public key relation.	Used by <code>HAETAE_Transcript.ec</code> extraction, by Module-SIS instances, and by <code>HAETAE_Security.ec</code> correctness.
Signature fields	<code>valid_signature</code> , <code>sig_commitment_highbits</code> , <code>sig_commitment_lowbits</code> , <code>sig_message_hash</code> , <code>sig_challenge</code> , <code>sig_response</code> , <code>sig_response_aux</code> , and <code>sig_hint</code> define the tuple layout of a signature.	Used by transcript construction, challenge-query predicates, rejection attempt states, and verification.

Deterministic model	signing	<code>commitment_raw</code> , <code>commitment_lowbits</code> , <code>challenge_hash</code> , <code>commitment_challenge</code> , <code>commitment_highbits</code> , <code>response_vector</code> , <code>hint_vector</code> , and <code>signature_token</code> define signing fields from secret key, message, context, and coins.	Used by the real, public-simulation, paper-simulation, and rejection-simulation signing games in <code>HAETAE_HopGames.ec</code> .
Internal algorithms		<code>keygen_internal</code> , <code>sign_internal</code> , <code>verify_challenge_consistent</code> , <code>verify_public_equation</code> , <code>verify_norm_ok</code> , and <code>verify_internal</code> define the scheme core.	<code>HAETAE_Security.ec</code> correctness reduces directly to <code>verify_internal_sign_internal</code> . <code>HAETAE_Scheme.ec</code> exposes these operations through the generic signature interface.

The main checked lemma families are:

Lemma family	Checked fact	Boundary
Well-formedness and coefficient bounds	<code>poly_add_wf</code> , <code>poly_mul_wf</code> , <code>matrix_vec_mul_wf</code> , vector well-formedness lemmas, coefficient-bound lemmas such as <code>polyveck_add_coeffs_q_bound</code> , decomposition <code>_wf</code> lemmas, and poly- q bounds show that formal operations preserve shape and ranges.	Machine-checked.
Seed/XOF and packing facts	<code>haetae_reference_keygen_xof_*</code> , <code>haetae_keygen_*_size</code> , <code>haetae_pack_vk_ref_roundtrip_*</code> , <code>haetae_public_key_roundtrip</code> , and public-key byte well-formedness lemmas prove list-size and roundtrip facts for the model.	Machine-checked formal-model facts; implementation equivalence remains outside this file.
Public-equation facts	<code>public_key_of_secret_relation</code> , <code>public_key_of_secret_equation_holds</code> , <code>public_key_vector_seed_mode23_keygen_equation</code> , and <code>public_key_vector_seed_mode5_keygen_equation</code> prove the formal verification equation shape.	Machine-checked; used by transcripts and extraction.

Signature validity and correctness	<code>valid_signature_sign_internal,</code> <code>verify_challenge_consistent_sign_internal,</code> <code>response_norm_ok_current,</code> <code>verify_norm_ok_current,</code> <code>verify_internal_sign_internal.</code>	Machine-checked formal correctness, used by HAETAE_Security.ec.
------------------------------------	---	---

Boundary: this file contains no MLWE/SIS hardness theorem and no ROM theorem. It is the formal model that later security theorems prove statements about.

1.25.1 Line-by-line proof-script companion for HAETAE Algebra

Focused criticism for this pass: the preceding dependency-level table names the major object families, but it is not enough for a mathematician or cryptologist to reconstruct the proof obligations in `HAETAE_Algebra.ec`. In particular, it did not say where the script switches from model declarations to checked preservation lemmas, which preconditions are theorem hypotheses rather than global facts, which proof steps rely on EasyCrypt library lemmas such as `size_mkseq`, `nth_mkseq`, `eq_from_nth`, and integer SMT, or why the final `verify_internal_sign_internal` theorem is only a formal-model correctness theorem. The following line-range companion closes that gap at proof-obligation granularity. It is still not a replacement for reading the source: tactic sequencing and all generated subgoals remain in the `.ec` file and in EasyCrypt’s checker.

Lines	Script content	Classification	Companion reading and downstream role
1–7	Imports and theory wrapper.	Imported/library facts.	<code>AllCore</code> , <code>IntDiv</code> , and <code>List</code> provide boolean logic, integer division/modulus, list constructors, <code>size</code> , <code>nth</code> , <code>nseq</code> , <code>mkseq</code> , <code>range</code> , <code>zip</code> , and <code>foldl</code> . <code>HAETAE_Params.ec</code> supplies all mode constants; <code>HAETAE_FIPS202.ec</code> supplies the formal SHAKE byte-list API. Any proof later using <code>smt</code> , <code>size_mkseq</code> , or SHAKE size lemmas imports those facts rather than reproving them here.
9–29	Carrier declarations for bytes, seeds, polynomials, vectors, matrices, keys, and signatures.	Machine-checked definitions.	These lines fix the formal universe: all algebra is over lists and tuples. They are checked type declarations, not a theorem that these carriers equal the HAETAE paper objects or Jasmin/C memory layouts. <code>HAETAE_Scheme.ec</code> and the generic <code>Sig_ROM.ec</code> clone use these exact carriers.

31–106	Zero objects, well-formedness predicates, challenge sparsity predicates, and their first size/well-formedness lemmas.	Machine-checked facts with imported list facts.	The proof pattern is definitional unfolding followed by <code>size_nseq</code> , <code>all_nseq</code> , <code>allP</code> , and <code>mem_nth</code> . These lemmas are the source of later side conditions saying that a formal polynomial has length n , a k -vector has <code>mode_kmd</code> rows, and a challenge has coefficients in $\{-1, 0, 1\}$.
108–169	Coefficient arithmetic, polynomial operations, vector operations, matrix-vector multiplication, and the list model of negacyclic multiplication.	Machine-checked definitions; explanatory model boundary.	The operations are definitions in the EasyCrypt model. The file checks shape and range facts about them later, but this block is not a mathematical proof that the list operations are isomorphic to a quotient-ring implementation. That correspondence would require a separate algebra or implementation equivalence theorem.
170–289	<code>_wf</code> preservation lemmas for arithmetic and matrix/vector operations.	Machine-checked facts relying on imported list facts.	The script repeatedly unfolds an operation, splits zero-special cases, proves the length of <code>mkseq</code> outputs, and uses induction for <code>foldl</code> in <code>poly_dot_wf</code> . These proofs discharge the routine obligations later needed by <code>valid_signature</code> , transcript predicates, and verification matrix construction.
291–425	<code>coeff_q_bound</code> and coefficient-range preservation lemmas.	Machine-checked facts; imported integer arithmetic.	The key obligation is that every arithmetic constructor returns coefficients modulo q . EasyCrypt’s integer division/modulus library and SMT prove <code>coeff_mod_q_bound</code> ; fold induction lifts that fact to dot products. These range facts feed public-key packing and mode-dependent coefficient packing bounds.
427–562	High/low-bit decomposition, hint high bits, verification-key high bits, and well-formedness of decomposition maps.	Machine-checked definitions and facts.	This block formalizes the bit-decomposition vocabulary used by public-key packing and verification reconstruction. The proofs check only list shape; numeric range refinements appear in the next packing block. Do not read this as a standalone theorem about the paper decomposition unless the definition is separately matched to the paper.

564–672	Seed-derived public-key polynomials, SHAKE/XOF wrappers, seed-buffer layout, and keygen seed slices.	Machine-checked definitions plus imported hash facts.	The script defines formal byte-list plumbing for the reference key-generation seed flow. Later lemmas use <code>HAETAE_FIPS202.ec</code> size/domain facts. This is not a ROM or SHAKE security assumption; it is deterministic byte-list modeling.
674–755	Formal keygen relation: matrix seed, secret/error vectors, rounding vector, mode-specific public-key vector, and <code>public_key_relation</code> .	Machine-checked definitions.	These are the definitions that extraction and verification use as the public equation. The theorem boundary is definitional: the relation is checked inside the model, while cryptographic hardness of finding alternate witnesses is handled outside this file by later reduction assumptions.
757–841	Reference keygen aliases and seed-flow predicate.	Machine-checked definitions and explanatory prose boundary.	The reference names make the script readable against HAETAE naming conventions. The file proves these aliases line up with the formal relation, but does not certify a C or Jasmin reference implementation.
841–902	Verification column/matrix construction, challenge embedding, public-key challenge term, and reconstructed highbits.	Machine-checked definitions.	These lines define what <code>verify_public_equation</code> later checks. They are used downstream by transcript public-equation predicates and rejection-attempt field consistency.
904–1265	Public-key byte encodings, simple pack/unpack definitions, reference 15-byte/16-bit packers, concrete packed-key aliases, and roundtrip predicates.	Machine-checked definitions; explanatory implementation boundary.	There are two encoding surfaces: a simple model packer and a reference-style bit packer. They are checked against their own definitions later. The presence of reference-style names should not be overread as a checked parser equivalence to every external implementation.

1267–1466	Seed/XOF seed-buffer size/layout lemmas.	and	Machine-checked facts with imported hash/list facts and conditional premises.	Representative hypotheses include <code>0<=outlen</code> , <code>0<=len</code> , and <code>sizesd=seedbytes</code> . The proofs use unfolding, SHAKE size lemmas, and offset/slice arithmetic. The theorem boundary is deterministic size/layout correctness, not pseudorandomness.
1468–1767	Public matrix/vector well-formedness and poly- q range lemmas for formal and reference keygen.	ma-	Machine-checked facts with mode-case proof obligations.	The script proves that generated rows and vectors have the right lengths and coefficient bounds. Mode-specific cases <code>Mode2</code> , <code>Mode3</code> , and <code>Mode5</code> are split explicitly where packing ranges differ.
1769–1936	Verification-matrix well-formedness, seed pack/unpack, and byte-normalization lemmas.	well-	Machine-checked facts relying on imported list/integer facts.	The important proof idioms are <code>eq_from_nth</code> for list equality, <code>nth_cat</code> for concatenated encodings, and integer SMT for byte normalization. These lemmas make the public-key roundtrip theorems possible.
1938–2753	Reference pack/unpack arithmetic for Mode2/3 15-byte blocks and Mode5 16-bit blocks; vector-body and public-key reference roundtrips.	for	Machine-checked facts with conditional premises.	The conditions <code>haetae_polyq_coeffs_wf</code> , <code>haetae_polyveck_polyq_coeffs_wf</code> , <code>md=Mode2\md=Mode3</code> , and <code>sizesd=seedbytes</code> are local theorem hypotheses, not global truths. The proof decomposes byte offsets, proves each coefficient offset $0, \dots, 7$, then lifts coefficient equality to polynomial and vector equality using <code>eq_from_nth</code> .
2771–3019	Simple model packing roundtrip, encoding well-formedness, and reference-key well-formedness.	model	Machine-checked facts with explicit hypotheses.	The simple packer proves a model roundtrip for <code>polyveck_wf</code> inputs; the reference packer proves a stronger poly- q -bounded roundtrip. These facts support keygen public-key recovery and verification-matrix construction in later games.

3022–3037	<code>public_key_of_secret_relation</code> and <code>public_key_of_secret_equation_holds</code> .	Machine-checked facts.	These are definitional correctness lemmas for honest keys. They feed transcript validity and extraction predicates, but they do not by themselves prove a hardness statement about producing another relation witness.
3039–3168	<code>valid_mode</code> , <code>valid_keypair</code> , <code>valid_signature</code> , signature projection functions, encoders, deterministic signing-field generators, and the signing entropy token extractor.	Machine-checked definitions; explanatory model boundary.	The formal signer is deterministic once coins are supplied. The low-bits field programs the first coefficient with the entropy token. This modeling choice is essential for ROM prequery and rejection-sampling arguments; it is not a claim about concrete entropy extraction unless separately connected to the implementation.
3171–3371	Well-formedness/unit/sparsity lemmas for deterministic signing fields and challenge hashes.	Machine-checked facts.	The proof pattern is constructor unfolding plus <code>mkseqP</code> , <code>allP</code> , and case analysis on parity and <code>i < mode_taumd</code> . These lemmas feed <code>valid_signature_sign_internal</code> , transcript validity, and rejection attempt acceptance.
3373–3404	<code>keygen_internal</code> , <code>sign_internal</code> , <code>verify_challenge_consistent</code> , <code>verify_reconstructed_highbits</code> , and <code>verify_public_equation</code> .	Machine-checked definitions.	This is the formal scheme core used by <code>HAETAE_Scheme.ec</code> . No probability, ROM, or adversary reasoning occurs here; those enter in <code>Sig_ROM.ec</code> , <code>HAETAE_HopGames.ec</code> , and <code>HAETAE_Security.ec</code> .

3407–3767	Keygen validity, public-key equations, reference keygen equations, packed-key roundtrips, verification-matrix correctness, and <code>valid_signature_sign_internal</code> .	Machine-checked facts with conditional premises.	Mode-specific equations split <code>Mode2/Mode3</code> from <code>Mode5</code> ; roundtrip lemmas require <code>sized=seedbytes</code> or encoding well-formedness. The downstream role is to justify that honest formal keys and signatures satisfy the predicates consumed by transcript and security proofs.
3777–3883	Norm definitions, current-signing norm lemmas, and <code>verify_internal_sign_internal</code> .	Machine-checked facts; theorem boundary.	The final theorem proves: for this formal model, verifying an honestly signed message under <code>public_key_of_secretmdsk</code> succeeds. The norm proofs use unit-vector lemmas and mode case analysis. This is a correctness theorem for <code>sign_internal/verify_internal</code> , not HAETAE EUF-CMA security and not a statement about an external implementation.

The practical proof-script reading rule for this file is therefore: treat every `op` and `type` as a checked model declaration, every `lemma` as a checked theorem only under its displayed hypotheses, every use of `smt`/list rewriting as imported EasyCrypt reasoning, and every connection to the paper algorithm, SHAKE security, or implementation behavior as explanatory unless another source file proves it.

Lemma-ledger criticism for HAETAE Algebra. The line-range companion above makes the file navigable, but it is still not enough, by itself, for a mathematician or cryptologist to reconstruct the proof obligations that later scripts cite. It gives ranges, but not a stable ledger of theorem surfaces: which names are definitions, which lemmas are checked consequences of those definitions, which side conditions are local premises, and which facts enter only because `AllCore`, `IntDiv`, `List`, `HAETAE_Params.ec`, or `HAETAE_FIPS202.ec` were imported. It also risks hiding the strongest boundary in the algebra file: `HAETAE_Algebra.ec` proves correctness of the formal `sign_internal/verify_internal` model, not equivalence to the HAETAE paper sampler, SHAKE-as-ROM security, or a Jasmin/C implementation. The ledgers below close that documentation gap at the level of names later used by `HAETAE_Rejection.ec`, `HAETAE_HopGames.ec`, and `HAETAE_Security.ec`. Repeated coefficient-offset lemmas and sibling `_wf` lemmas are grouped when they have the same proof obligation and the same downstream role; the EasyCrypt source remains authoritative for exact tactic order and generated subgoals.

Imported-interface ledger for HAETAE Algebra.

EasyCrypt interface	Lines / category	Statement paraphrase	Premises, imports, downstream role
---------------------	------------------	----------------------	------------------------------------

AllCore, IntDiv, List		1. Imported/library fact.	Boolean reasoning, integer division/modulus, lists, size , nth , nseq , mkseq , range , zip , foldl , and equality from indexed lists.	No local cryptographic premise. Used in almost every algebra proof; later Rejection and HopGames inherit these list and arithmetic facts through this model.
HAETAE_Params.ec HAETAE_Params	and	2, 6. Imported/library fact.	Provides the formal mode constants, dimensions, modulus, norm bounds, packing lengths, positivity lemmas, and mode-case equations.	Used by vector dimensions, sparse challenges, norm bounds, packing sizes, and the rejection thresholds consumed in HAETAE_Rejection.ec . This is an imported formal-parameter interface, not a proof that the constants match a standard document.
HAETAE_FIPS202.ec HAETAE_FIPS202	and	2, 7. Imported/library fact.	Provides byte-list SHAKE/XOF functions and size/domain lemmas such as shake256_size , shake256_squeeze_size , rate bytes, and domain separator facts.	Feeds keygen seed-flow lemmas. It supplies deterministic byte-list facts, not a random-oracle theorem and not SHAKE pseudorandomness.
Cryptographic assumptions in this file	assump-	1–3885. External cryptographic assumption.	None are introduced by HAETAE_Algebra.ec .	MLWE, Module-SIS, ROM, Fiat-Shamir, rejection-sampling, paper-sampler, and implementation-equivalence assumptions are outside this file and enter only in later security/reduction layers.

Definition ledger for HAETAE Algebra.

EasyCrypt object(s)	Lines / category	Mathematical paraphrase	statement	Local premises, imports, downstream role
byte, seed, crh, random_coins, xof256_state, message, context, coeff, poly, challenge, polyveck, polyvecl, polyvecm, matrix, hint_t, sig_token, pkey, encoded_pkey, skey, signature	9–29. Machine-checked fact.	Fixes the formal carrier universe as integers, lists, and tuples.		No local premises. Consumed by HAETAE_Scheme.ec to instantiate Sig_ROM.ec ; all Rejection, HopGames, and Security statements quantify over these carriers.
secret_key_of_seed, zero_vectors/matrices, poly_wf, polyveck_wf, polyvecl_wf, crh_wf, challenge_wf, challenge_sparse, poly_unit, polyveck_unit, polyvecl_unit	31–87. Machine-checked fact.	Defines zero/default objects and shape, sparse-challenge, and unit-vector predicates.		Imports mode dimensions from HAETAE_Params.ec . These predicates are the side conditions used by signature validity, distributions, rejection samples, and transcript validity.

<code>coeff_mod,</code> <code>coeff_neg,</code> <code>coeff_mul,</code> <code>double,</code> <code>poly_normalize,</code> <code>poly_neg,</code> <code>poly_double,</code> <code>poly_dot,</code> <code>polyveck_add,</code> <code>sub,</code> <code>polyveck_sub</code>	<code>coeff_add,</code> <code>coeff_sub,</code> <code>coeff_</code> <code>poly_coeff,</code> <code>poly_add,</code> <code>poly_sub,</code> <code>poly_mul,</code> <code>matrix_vec_mul,</code> <code>polyveck_</code> <code>polyveck_double,</code> <code>polyveck_sub</code>	108–169. Machine-checked fact.	Defines the list-based formal polynomial, vector, and matrix arithmetic used by the HAETAE model.	Imports q , n , and mode dimensions. Used by public-key equations, verification reconstruction, rejection reject2 balance, and extraction relations. This is not a separate quotient-ring implementation theorem.
<code>coeff_q_bound,</code> <code>poly_coeffs_q_bound,</code> <code>polyveck_coeffs_q_bound</code>		291–297. Machine-checked fact.	Names coefficient-range predicates for formal q -bounded values.	No cryptographic premise. Feeds packing poly- q bounds and public-key vector well-formedness.
<code>coeff_decompose_z1_low,</code> <code>coeff_decompose_</code> <code>z1_high,</code> <code>coeff_lsb,</code> <code>coeff_decompose_vk_low,</code> <code>coeff_decompose_vk_high,</code> <code>coeff_decompose_hint_</code> <code>high,</code> <code>poly_highbits,</code> <code>poly_lowbits,</code> <code>poly_vk_lowbits,</code> <code>poly_vk_highbits,</code> <code>poly_hint_highbits,</code> vector lifts		427–481. Machine-checked fact.	Defines formal high/low-bit, verification-key, and hint decomposition functions.	Imports mode-specific hint bases. Used by public-key packing, public-key verification columns, signer commitments, and Rejection’s public-equation predicates. Paper correspondence is explanatory unless separately proved.
<code>public_key_seed_poly,</code> <code>haetae_shake256_bytes,</code> <code>haetae_xof256_*,</code> <code>haetae_</code> <code>seed_slice,</code> <code>haetae_</code> <code>reference_keygen_*,</code> <code>haetae_keygen_*,</code> <code>haetae_</code> <code>keygen_sampler_seed</code>		564–678. Machine-checked fact.	Defines deterministic seed expansion, XOF byte-list wrappers, seed-buffer slices, and keygen seed components.	Imports FIPS202 byte-list functions and parameter lengths. Used by keygen and packed-public-key lemmas. This is deterministic plumbing, not SHAKE security or ROM modeling.
<code>public_matrix_seed,</code> <code>public_secret_vector_</code> <code>seed,</code> <code>public_error_</code> <code>vector_seed,</code> <code>public_</code> <code>rounding_vector_seed,</code> <code>haetae_public_key_</code> <code>vector_from_relation,</code> <code>public_key_relation,</code> <code>public_key_equation_</code> <code>holds,</code> reference-keygen aliases		680–840. Machine-checked fact.	Defines the formal public-key relation and the reference-named keygen views.	Imports formal arithmetic and mode-specific branches. Used by transcript extraction, HAETAE_Security.ec correctness, and Module-SIS relation setup. Hardness is not proved here.

haetae_mode23_verification_column, haetae_mode5_verification_column, public_key_verification_column, haetae_verification_matrix_from_unpacked, public_verification_matrix, challenge_embedding, public_key_challenge_term, reconstructed_highbits, public_key_of_secret	841–903. Machine-checked fact.	Defines how public keys induce verification matrices, challenge terms, and reconstructed highbits.	Used by <code>verify_public_equation</code> , Rejection field consistency, HopGames paper/public simulation rewrites, and final correctness.
haetae_public_key_*, haetae_pack_*, haetae_unpack_*, haetae_byte_norm, haetae_polyq_*, concrete_haetae_*	904–1265. Machine-checked fact.	Defines the simple and reference-style public-key byte encoders/decoders and their well-formedness predicates.	Imports packing lengths and integer division. Used by verification-matrix correctness and key recovery lemmas. Names containing “concrete” are still formal EasyCrypt objects, not external implementation equivalence.
valid_mode, valid_keypair, valid_signature, sig_* projections, encode_poly, encode_polyveck, encode_pkey	3039–3062. Machine-checked fact.	Defines formal validity and the exact tuple fields of signatures and encoded public keys.	Consumed by <code>HAETAE_Transcript.ec</code> , Rejection attempt relations, HopGames transcript games, and <code>HAETAE_Security.ec</code> .
deterministic_*, commitment_source, commitment_raw, response_source, response_aux_vector, signing_sample_y1, signing_sample_y2, signing_entropy_token_of_coins, commitment_lowbits, message_hash, challenge_source, challenge_from_seed, challenge_hash, commitment_challenge, commitment_highbits, response_vector, hint_vector, signature_token	3064–3169. Machine-checked fact.	Defines the deterministic formal signing-field generator from a key, message, context, and coins. The first lowbits coefficient stores a coin-derived token.	Used by Rejection attempt constructors, HopGames public/paper-simulation rewrites, counted-ROM prequery arguments, and transcript soundness. It is a formal model of the signer, not a sampler correspondence proof.
keygen_internal, sign_internal, verify_challenge_consistent, verify_reconstructed_highbits, verify_public_equation	3373–3405. Machine-checked fact.	Defines the formal keygen, signer, challenge-consistency check, and public equation check.	Used directly by <code>HAETAE_Scheme.ec</code> , Rejection relation lemmas, and <code>HAETAE_Security.ec</code> correctness.

coeff_norm_sq, poly_norm_sq, polyvecl_norm_sq, polyveck_norm_sq, response_norm_ok, verify_norm_ok, verify_internal	3777–3817. Machine-checked fact.	Defines the formal norm predicates and final verifier conjunction.	Imports mode norm bounds. Rejection uses these as acceptance/abort channels; Security uses <code>verify_internal_sign_internal</code> for correctness.
--	--	--	--

Lemma ledger for HAETA E Algebra.

EasyCrypt lemma(s)	Lines / category	Mathematical statement paraphrase	Local premises, imports, downstream role
poly_zero_size, polyveck_zero_size, polyvecl_zero_size, polyvecm_zero_size, polyveck_wf_nth, poly_zero_wf, polyveck_zero_wf, polyvecl_zero_wf	39–106. Machine-checked fact.	Zero objects have the expected lengths; well-formed vectors expose well-formed rows.	Imports list facts <code>size_nseq</code> , <code>allP</code> , <code>mem_nth</code> . Used whenever later scripts take a default row from a public key, commitment, or sample.
polyvecl_sub_size, arithmetic_wf lemmas through polyvecl_sub_wf	170–289. Machine-checked fact.	Formal arithmetic preserves polynomial and vector shape.	Imports <code>size_mkseq</code> , <code>mkseqP</code> , <code>foldl</code> induction, and mode positivity. Downstream: signature validity, verification matrices, Rejection state well-formedness.
coeff_mod_q_bound, poly_*coeffs_q_bound, matrix_vec_mul_coeffs_q_bound, polyveck_*coeffs_q_bound	298–425. Machine-checked fact.	Formal arithmetic constructors produce q -bounded coefficients.	Imports integer modulus facts and <code>smt</code> . Used by poly- q packing bounds and public-key vector range lemmas.
Decomposition_wf lemmas and poly- q bound lemmas, including <code>polyveck_vk_highbits_mode23_polyq_wf</code> and <code>polyveck_q_bound_mode5_polyq_wf</code>	483–1057. Machine-checked fact.	High/low-bit and hint decomposition maps preserve shape and satisfy the formal poly- q bounds used for public-key packing.	Local premises include q -bounded inputs and, for Mode2/3 VK highbits, <code>md=Mode2\md=Mode3</code> . Used by keygen public-vector and reference packing roundtrip lemmas.
public_key_seed_poly_wf, public_key_seed_polyvecl_wf, public_key_seed_polyveck_wf, seed/XOF size and layout lemmas through <code>haetae_keygen_counter_next_gt</code>	1267–1579. Machine-checked fact.	Seed-derived objects and keygen seed buffers have the expected formal lengths, and the XOF wrapper matches the SHAKE byte-list presentation.	Local premises include <code>0<=outlen</code> , <code>0<=len</code> , and <code>sizesd=seedbytes</code> . Imports FIPS202 size/domain facts. Used by keygen well-formedness and reference seed-flow documentation.

public_matrix_seed_*, public_secret_vector_ seed_wf, public_error_ vector_seed_wf, public_ rounding_vector_seed_wf, reference/keygen vector _wf and polyq_wf lem- mas through public_key_ vector_seed_wf	1581–1839. Machine- checked fact.	Formal public matrices, se- cret/error vectors, rounding vectors, and public-key vec- tors are shape-correct and poly- q -bounded.	Mode cases split Mode2/3 from Mode5. Used by public-key pack- ing, verification-matrix construction, and HAETAESecurity.ec correctness.
public_key_mode23_ column_wf, haetae_ verification_column_ from_unpacked_wf, public_key_verification_ column_wf, public_ verification_matrix_size, public_verification_ matrix_rows_wf, challenge_embedding_wf, public_key_of_secret_wf, seed pack/unpack lemmas	1846–1953. Machine- checked fact.	Verification matrices and challenge embeddings are well- shaped; seed packing roundtrips under seed-size hypotheses.	Imports list equality and mode dimensions. Used by verification public equations, transcripts, and packed-key recovery.
haetae_polyq_coeff_ bound_gt0, haetae_ byte_norm_*, haetae_ unpack15_coeff0 through haetae_unpack15_coeff7, reference packer size/wf lemmas	1955–2098. Machine- checked fact.	Byte normalization is bound- ed/idempotent, and each 15-byte Mode2/3 coefficient lane decodes to the original formal coefficient under range hypotheses.	Local premises are explicit numeric ranges. Imports integer division/modulus SMT. Used only for formal reference-style public-key packing.
nth_cat_size_plus, haetae_pack_polyveck_ body_ref_nth, Mode2/3 coefficient-offset lemmas, haetae_unpack_poly_ q_ref_pack_mode23_*, haetae_pack_vk_ref_ roundtrip_mode23	2100–2583. Machine- checked fact.	For Mode2/3, reference-style public-key packing followed by unpacking returns the same formal polynomial/vector/key.	Premises: md=Mode2\ /md=Mode3 , seed size, and haetae_polyq_*wf . Downstream: reference packed-key roundtrip and verification-matrix correctness; not external parser equivalence.
haetae_unpack_poly_ q_ref_pack_mode5_*, haetae_unpack_polyveck_ body_ref_pack_mode5, haetae_pack_vk_ref_ roundtrip_mode5, haetae_ public_key_ref_roundtrip, haetae_keygen_public_ key_ref_roundtrip	2585–2769. Machine- checked fact.	For Mode5 and mode-generic reference keys, formal refer- ence packing roundtrips under the displayed well-formedness premises.	Premises: seed size and poly- q well-formedness. Used by concrete/ref verification-matrix cor- rectness lemmas.
haetae_pack_polyveck_ body_size, haetae_ unpack_poly_q_pack_at, haetae_unpack_polyveck_ body_pack_at, haetae_ public_key_roundtrip, public-key encoding/unpack- ing wf lemmas	2771–2964. Machine- checked fact.	The simpler model public- key packer is size-correct and roundtrips for well-formed inputs.	Premises: poly_wf , polyveck_wf , or unpacked- key well-formedness. Used by keygen public-key roundtrip and matrix- correctness lemmas.

<code>packed_haetae_reference_public_key_*</code> , <code>concrete_haetae_reference_wf</code> , <code>haetae_reference_keygen_public_key_ref_roundtrip</code>	2966–3019. Machine-checked fact.	Reference-named formal public keys are well-formed and roundtrip through the reference-style packer.	Imports earlier reference packer lemmas. Used by concrete/reference verification-matrix facts. Still a formal-object theorem.
<code>public_key_of_secret_relation</code> , <code>public_key_of_secret_equation_holds</code>	3022–3037. Machine-checked fact.	The public key derived from a secret seed satisfies the formal public-key relation/equation.	Definitional proof. Used by transcript validity, Rejection signing-attempt relations, and Security correctness; it is not a hardness theorem.
Deterministic signing-field lemmas: <code>deterministic_wf</code> , <code>deterministic_unit</code> , <code>commitment_wf</code> , <code>commitment_lowbits_entropy_token</code> , <code>challenge_from_seed_*</code> , <code>challenge_hash_*</code> , <code>response_*</code> , <code>signing_sample_*</code> , <code>hint_*</code> , <code>message_hash_size</code> , <code>challenge_hash_size</code>	3171–3371. Machine-checked fact.	Deterministic signer fields are well-formed/unit/sparse and have expected hash lengths; lowbits coefficient zero equals the signing entropy token.	Imports list construction facts and mode dimensions. Used by Rejection field-consistency lemmas, HopGames simulation rewrites, and counted-ROM prequery/token arguments.
<code>keygen_internal_valid</code> , <code>public_key_of_keygen</code> , <code>keygen_equation_lemmas</code> , <code>reference_keygen_equation_lemmas</code> , <code>haetae_reference_keygen_seed_flow_wf_ok</code> , <code>packed_key</code> and verification-matrix identity lemmas through <code>public_verification_matrix_packedE</code>	3407–3605. Machine-checked fact.	Honest formal keygen returns a valid keypair, and the various public-key and reference-keygen views unfold to the same formal equations.	Some lemmas require Mode2/3 premises or seed-size premises. Used by Security correctness and by matrix/public-key bridge lemmas in transcript games.
<code>packed_haetae_keygen_verification_matrixE</code> , <code>honest_public_verification_matrix_eq_packed_keygen</code> , <code>concrete_verification_matrix_correct</code> , <code>concrete_verification_matrix_size</code> , <code>concrete_verification_matrix_rows_wf</code>	3607–3758. Machine-checked fact.	The verification matrix obtained from honest keygen or packed/unpacked formal keys agrees with the corresponding formal concrete/reference matrix and has the expected shape.	Premises include <code>sizesd=seedbytes</code> for keygen concrete matrix roundtrip. Used wherever HopGames or Security move between public keys, encoded keys, and verification matrices.
<code>valid_signature_sign_internal</code> , <code>verify_challenge_consistent_sign_internal</code>	3760–3775. Machine-checked fact.	The formal signer emits a signature satisfying the formal validity predicate and challenge consistency under <code>public_key_of_secret</code> .	Depends on deterministic-field <code>_wf</code> , message/challenge size, and challenge/hash definitions. Used directly in Rejection signing-attempt simulation and Security correctness.

<code>secret_norm_ok_current,</code> <code>response_norm_ok_current,</code> <code>verify_norm_ok_current,</code> <code>verify_internal_sign_</code> <code>internal</code>	3819–3883. Machine- checked fact.	Honest formal secret/signature values satisfy norm bounds, and verification of <code>sign_internal</code> under <code>public_key_of_secret</code> succeeds.	Imports mode bounds and deterministic unit-vector lemmas. This is the local formal correctness theorem cited by <code>HAETAE_Security.ec</code> ; it is not EUF-CMA security and not implementation equivalence.
---	---	---	--

Algebra import and boundary ledger.

Category		Status inside <code>HAETAE_Algebra.ec</code>
Machine-checked facts		All displayed <code>lemma</code> statements in this file, including shape/range preservation, packing roundtrips, keygen equations, public verification-matrix facts, norm facts, and <code>verify_internal_sign_internal</code> .
Imported/library facts		<code>AllCore</code> , <code>IntDiv</code> , and <code>List</code> supply list equality, list indexing, sequence sizes, integer division/modulus, and SMT-dischargeable arithmetic. <code>HAETAE_Params.ec</code> supplies positivity and mode constants. <code>HAETAE_FIPS202.ec</code> supplies SHAKE size/rate/domain facts such as <code>shake256_size</code> , <code>shake256_squeeze_size</code> , <code>shake256_rate_bytesE</code> , and <code>shake256_domain_separatorE</code> .
External cryptographic assumptions		None are introduced locally. This file does not state MLWE, Module-SIS, ROM, Fiat-Shamir, or rejection-sampling security assumptions.
Conditional premises		No global cryptographic premise is assumed locally. Individual lemmas still have mathematical hypotheses, for example <code>sizesd=seedbytes</code> , <code>md=Mode2\md=Mode3</code> , <code>polyveck_wfmdB</code> , or <code>haetae_polyveck_polyq_coeffs_wfmdB</code> ; these hypotheses are theorem premises, not established facts about arbitrary inputs.
Explanatory boundary	prose	Names containing “reference” or “concrete” identify formal EasyCrypt objects in this file. They do not by themselves certify a C/Jasmin implementation, the HAETAE standard document, or SHAKE pseudorandomness.

Final correctness dependency chain. The local correctness theorem decomposes exactly according to the conjunction in `verify_internal`. A reader replaying the proof should check these four obligations rather than treating `verify_internal_sign_internal` as an opaque correctness fact.

<code>verify_internal</code> <code>conjunct</code>	Discharged by	Proof-script dependency reading
---	---------------	---------------------------------

<code>valid_ signaturemdsig</code>	<code>valid_signature_sign_ internal.</code>	Uses well-formedness of commitment highbits, lowbits, message hash, challenge, response, auxiliary response, and hint. These depend on deterministic constructor <code>_wf</code> lemmas and imported list-size facts.
<code>verify_ challenge_ consistentmdpkmtxsig</code>	<code>verify_challenge_ consistent_sign_internal.</code>	Pure definitional unfolding of <code>sign_internal</code> , <code>commitment_challenge</code> , and <code>challenge_hash</code> ; no hardness or probability reasoning enters.
<code>verify_public_ equationmdpkmsig</code>	Direct rewrite inside <code>verify_ internal_sign_internal.</code>	The proof expands <code>verify_public_equation</code> , <code>verify_reconstructed_highbits</code> , <code>sign_internal</code> , and <code>commitment_highbits</code> , then observes that the signature's auxiliary response field is the same value used to build the commitment highbits.
<code>verify_norm_ okmdsig</code>	<code>verify_norm_ok_current.</code>	Uses the unit-vector lemmas for deterministic response, auxiliary response, and hint, then mode case analysis against the constants in <code>HAETAE_Params.ec</code> .

Packing and roundtrip obligation map.

Range	Main obligations	Premises and boundary
Mode2/Mode3 reference packer	The offset lemmas for <code>ofs0</code> through <code>ofs7</code> prove that each 15-byte block decodes eight 15-bit coefficients, then <code>haetae_unpack_poly_q_ref_pack_mode23_coeff</code> and vector-body lemmas lift coefficient equality to polynomials and vectors.	Requires <code>md=Mode2\</code> / <code>md=Mode3</code> and <code>haetae_polyq_coeffs_wf</code> . Arithmetic is discharged by imported integer SMT plus byte-normalization lemmas.
Mode5 reference packer	<code>haetae_unpack_poly_q_ref_pack_mode5_coeff</code> , <code>haetae_unpack_poly_q_ref_pack_mode5_at</code> , and vector-body lemmas prove the 16-bit two-byte decoding roundtrip.	Requires <code>haetae_polyq_coeffs_wfMode5</code> . The checked bound is formal poly- q well-formedness, not an implementation parser theorem.

Simple packer	model	<code>haetae_unpack_poly_q_pack_at</code> , <code>haetae_unpack_polyveck_body_pack_at</code> , and <code>haetae_public_key_roundtrip</code> .	Requires <code>poly_wf</code> or <code>polyveck_wf</code> . This packer is a model roundtrip surface distinct from the reference-style byte-packing surface.
Public-key recovery		<code>haetae_keygen_public_key_roundtrip</code> , <code>haetae_keygen_public_key_ref_roundtrip</code> , and verification-matrix correctness lemmas.	Requires seed-size or unpacked-key well-formedness premises. These lemmas justify downstream formal verification matrices, but not external implementation equivalence.

1.26 Script companion coverage for HAETAE Distributions

`HAETAE_Distributions.ec` defines the distributions used by key generation, random-oracle outputs, signing coins, and signing-attempt samples. Its most important boundary is the distinction between a checked bounded carrier and a paper-faithful exact HAETAE hyperball sampler.

Script object family	Mathematical reading	Downstream role
Byte, seed, CRH, coefficient, polynomial, matrix, and challenge distributions	<code>dbyte</code> , <code>dseed</code> , <code>dcrh</code> , <code>dcoeff</code> , <code>dpoly</code> , <code>dpolyveck</code> , <code>dpolyvecl</code> , <code>dmatrix</code> , and <code>dchallenge</code> are product or mapped distributions over the formal carriers.	<code>HAETAE_ROM.ec</code> uses these as random-oracle output components. <code>HAETAE_Scheme.ec</code> samples <code>dseed</code> and <code>drandom_coins</code> .
Signing randomness token family	<code>signing_entropy_token_cardinality</code> , <code>signing_entropy_token_to_coins</code> , <code>signing_randomness_domain</code> , <code>dsigning_entropy_token</code> , <code>drandom_coins</code> , and <code>signing_randomness_point_bound</code> encode a 58-bit entropy token embedded into signing coins.	Feeds the sampler-prequery and ROM min-entropy bounds in <code>HAETAE_HopGames.ec</code> and <code>HAETAE_ROM_Programming.ec</code> .
Signing sample pairs	<code>signing_sample_pair</code> , <code>projections</code> , <code>signing_sample_pair_wf</code> , <code>signing_sample_pair_unit</code> , <code>signing_sample_pair_bounded</code> , and <code>signing_sample_pair_sample_ok</code> define the shape of the two-vector sample used inside signing attempts.	<code>HAETAE_Rejection.ec</code> lifts these samples into full signing-attempt states and rejection predicates.

Structural and bounded sample distributions	<code>dstructural_signing_sample_pair</code> , <code>dbounded_unit_signing_sample_pair</code> , and <code>dchecked_hyperball_signing_sample_pair</code> provide deterministic, unit-product, and bounded-product sampling surfaces.	Used by hop games to relate real signing, paper simulation, and exact-hyperball sampling surfaces.
Hyperball interface	<code>hyperball_coeff_ok</code> , <code>hyperball_sample_ok</code> , <code>dhyperball_*</code> , point-bound operations, and <code>dexact_hyperball_signing_sample_pair</code> expose the sampler interface needed by rejection-sampling lifts.	The current checked implementation sets the exact hyperball interface to a bounded product distribution. The comments in the script mark the paper sphere/hyperball law as replacement/correspondence work.

Lemma family	Checked fact	Boundary
Losslessness lemmas	<code>byte_distribution_lossless</code> , <code>seed_distribution_lossless</code> , <code>signing_coin_distribution_lossless</code> , and hyperball/sample-pair losslessness lemmas show that each formal distribution has total mass 1.	Machine-checked.
Support/domain lemmas	<code>signing_coin_distribution_domain</code> , <code>signing_sample_pair_of_coins_sample_ok</code> , unit/bounded support lemmas, and hyperball support P lemmas connect sampling to shape predicates.	Machine-checked.
Point-bound lemmas	<code>signing_coin_distribution_token_point_bound</code> , <code>dlist_point_bound</code> , <code>hyperball_poly_point_boundE</code> , <code>hyperball_polyvecl_point_boundE</code> , and <code>exact_hyperball_signing_sample_pair_point_bound</code> provide the probability terms consumed by sampler-prequery bounds.	Machine-checked for the formal distributions.
Checked/exact equality	<code>exact_hyperball_signing_sample_pair_checkedE</code> proves that the current <code>dexact_hyperball_signing_sample_pair</code> equals the checked bounded carrier.	Machine-checked equality, but not a paper-sampler equivalence theorem.

Boundary: the file checks distribution properties for the formal carriers. It does not by itself

prove that the formal signing randomness and bounded sample law are the final HAETAE paper distributions.

1.26.1 Lemma-ledger companion for HAETAE Distributions

Lemma-ledger criticism for HAETAE Distributions. The previous distribution companion made the main boundary visible, but it was still too coarse for reconstructing proof obligations. A reader needs to know which names are merely product-distribution definitions, which support and point-probability facts are machine-checked, where the deterministic `signing_sample_pair_of_coins` surface differs from an entropy-bearing sample surface, and where the word “hyperball” is only the formal interface name of the current bounded-product carrier. Without a ledger, it is easy to overclaim that `HAETAE_Distributions.ec` proves paper-sampler correctness, SHAKE-to-ROM behavior, or the sampling-distance premise used in later hop games. It does not. It proves losslessness, support, domain, and point-bound facts for the formal distributions below; the paper-sampler and ROM correspondence obligations remain separate conditional or external boundaries unless a later checked lemma discharges them.

Imported-interface and assumption ledger for HAETAE Distributions.

EasyCrypt object	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	im- downstream
<code>AllCore</code> , <code>Distr</code> , <code>DInterval</code> , <code>DList</code> , <code>IntDiv</code> , <code>List</code> , <code>Real</code> , <code>StdOrder</code>	1. Imported/library fact.	Provides distributions, intervals, product/list distributions, real probabilities, order reasoning, integer division, lists, <code>mu</code> , support, and losslessness lemmas.		No HAETAE-specific premise. Used by every distribution proof and inherited by Rejection and HopGames sampler arguments.	
<code>HAETAE_Params.ec</code> , <code>HAETAE_Algebra.ec</code> , <code>HAETAE_Params</code> , <code>HAETAE_Algebra</code> , <code>RealOrder</code>	2, 6–8. Imported/library fact.	Imports the formal mode constants, carrier types, well-formedness predicates, signing-field constructors, challenge constructor, and real-order tactics.		All support lemmas are relative to these formal carriers and parameter definitions. <code>HAETAE_Algebra.ec</code> itself does not import this file.	
Paper sampler and ROM correspondence	61–67, 142–202. External cryptographic assumption.	Comments explicitly mark the challenge carrier and hyperball carrier as structural/formal surfaces rather than paper-faithful correspondence theorems.		Later <code>HAETAE_Rejection.ec</code> and <code>HAETAE_HopGames.ec</code> can use these formal distributions, but equality to the HAETAE paper sampler or SHAKE/ROM semantics must be supplied separately.	
Structural-to-exact sampling-loss premises	Not defined in this file. Conditional premise.	The later <code>structural_to_exact_hyperball</code> loss obligations compare attempt-state distributions built from the sample-pair distributions defined here.		The distributions file supplies support and point-bound ingredients, not the full sampling-distance theorem consumed by <code>HAETAE_HopGames.ec</code> and <code>HAETAE_Security.ec</code> .	

Definition ledger for HAETAE Distributions.

EasyCrypt object(s)	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
<code>signing_randomness_source, dbyte, dseed, dcrh, signing_entropy_token_cardinality, signing_entropy_token_in_range, signing_randomness_byte_ok, signing_entropy_token_to_coins, signing_randomness_domain, dsigning_entropy_token, dsigning_randomness_source, signing_randomness_source_valid, signing_randomness_from_source, dsigning_randomness, drandom_coins, signing_randomness_point_bound, signing_randomness_token_spread, signing_randomness_distribution_ok</code>	10–53. Machine-checked fact.	Defines byte, seed, CRH, and signing-coin distributions, with a finite entropy token embedded into formal signing coins and a point-bound predicate for token events.		Imports parameter lengths and algebraic coin-token projection. Used by <code>HAETAE_Scheme.ec</code> signing, <code>HAETAE_Rejection.ec</code> attempt-state sampling, <code>HAETAE_ROM_Programming.ec</code> min-entropy bounds, and <code>HAETAE_HopGames.ec</code> sampler-prequery bounds.	
<code>dcoeff, dpoly, dpolyveck, dpolyvecl, dmatrix</code>	55–59. Machine-checked fact.	Defines uniform coefficient and product list distributions over the formal polynomial/vector/-matrix carriers.		Imports q , n , <code>mode_k</code> , and <code>mode_1</code> . Used as primitive distribution infrastructure; direct security hops mainly consume the signing sample and attempt-state distributions built later.	
<code>dchallenge</code>	61–66. Machine-checked fact.	Maps <code>dcrh</code> through <code>challenge_from_seedmd</code> to obtain a structural prefix-sparse challenge carrier.		Imports the algebraic challenge constructor. The checked support is local to that constructor; equality to a specification challenge sampler is not proved here.	
<code>signing_sample_pair, signing_sample_pair_y1, signing_sample_pair_y2, signing_sample_pair_wf, signing_sample_pair_unit, signing_sample_bound, coeff_bounded_by, poly_bounded_by, polyvecl_bounded_by, polyveck_bounded_by, signing_sample_pair_bounded, signing_sample_pair_sample_ok, signing_sample_pair_of_coins</code>	68–115. Machine-checked fact.	Defines the two-vector signing sample interface, its well-formed/unit/bounded predicates, and the deterministic sample extracted from formal signing coins.		Imports algebraic <code>signing_sample_y1</code> and <code>signing_sample_y2</code> . This is the interface lifted into full signing attempt states in <code>HAETAE_Rejection.ec</code> .	

<code>dstructural_signing_sample_pair</code>	117–120. Machine-checked fact.	Defines a degenerate distribution concentrated on the deterministic sample pair from coins.	Used by structural attempt-state distributions and by loss lemmas comparing structural samples to exact-hyperball samples.
<code>dsigning_unit_coeff,</code> <code>dsigning_unit_poly,</code> <code>dsigning_unit_polyvec1,</code> <code>dsigning_unit_polyveck,</code> <code>signing_bounded_coeff_cardinality,</code> <code>signing_bounded_coeff_point_bound,</code> <code>dsigning_bounded_coeff,</code> <code>dsigning_bounded_poly,</code> <code>dsigning_bounded_polyvec1,</code> <code>dsigning_bounded_polyveck,</code> <code>dbounded_unit_signing_sample_pair</code>	122–149. Machine-checked fact.	Defines unit and bounded product sample carriers and a bounded-unit pair sampler.	Imports <code>mode_ln</code> , <code>mode_k</code> , <code>mode_1</code> , and <code>n</code> . Used by Rejection as an intermediate sampler surface; it is not the paper hyperball law.
<code>dchecked_hyperball_signing_sample_pair,</code> <code>hyperball_coeff_ok,</code> <code>hyperball_poly_ok,</code> <code>hyperball_polyvec1_ok,</code> <code>hyperball_polyveck_ok,</code> <code>hyperball_sample_ok,</code> <code>dhyperball_coeff,</code> <code>dhyperball_poly,</code> <code>dhyperball_polyvec1,</code> <code>dhyperball_polyveck,</code> <code>hyperball_point-bound_operations,</code> <code>dexact_hyperball_signing_sample_pair</code>	156–203. Machine-checked fact.	Defines the checked bounded-product hyperball-shaped carrier, its support predicates, component point bounds, and the current exact interface as a product of bounded components.	Used by <code>HAETAE_Rejection.ec</code> exact-hyperball attempt states and by <code>HAETAE_HopGames.ec</code> exact-hyperball sampling hops. The name “exact” is formal here; paper-sphere equivalence remains outside this file.
<code>signing_sample_pair_distribution_ok</code>	205–209. Machine-checked fact.	Names the contract that a sample-pair distribution is lossless and supports only formally sample-ok pairs.	Used by sampler-lifting lemmas in <code>HAETAE_Rejection.ec</code> .

Lemma ledger for HAETAE Distributions.

EasyCrypt lemma(s)	Lines / category	Mathematical statement	Local premises, imports, downstream role
<code>byte_distribution_lossless,</code> <code>seed_distribution_lossless,</code> <code>crh_distribution_lossless</code>	211–223. Machine-checked fact.	The byte, seed, and CRH product distributions are lossless.	Imports <code>dinter_ll</code> , <code>dlist_ll</code> , and parameter lengths. Used by <code>HAETAE_Scheme.ec</code> , <code>HAETAE_ROM.ec</code> , and challenge-distribution losslessness.

<code>signing_entropy_token_</code> <code>cardinality_positive,</code> <code>signing_entropy_token_</code> <code>distribution_lossless,</code> <code>signing_randomness_</code> <code>source_distribution_</code> <code>lossless,</code> <code>signing_coin_</code> <code>distribution_lossless</code>	226–248. Machine- checked fact.	The entropy-token and signing-coin distributions are well-defined and lossless.	Imports <code>interval</code> and <code>map_losslessness</code> . Used by signing procedures, attempt-state samplers, and HopGames probability calculations over coins.
<code>signing_entropy_</code> <code>token_to_coins_tokenK,</code> <code>signing_randomness_</code> <code>from_source_tokenK,</code> <code>signing_entropy_</code> <code>token_to_coins_size,</code> <code>signing_entropy_token_</code> <code>to_coins_byte_domain,</code> <code>signing_entropy_</code> <code>token_to_coins_domain,</code> <code>signing_randomness_from_</code> <code>source_domain</code>	251–311. Machine- checked fact.	Encoding a token into coins recovers the same token, has seed length, and satisfies the byte/domain predicates under the token-range premise.	Local premise for byte/domain lemmas is <code>signing_entropy_token_in_rangex</code> . Used to prove coin support and ROM prequery/min-entropy facts.
<code>signing_entropy_token_</code> <code>distribution_point_bound,</code> <code>signing_randomness_</code> <code>source_distribution_</code> <code>point_bound,</code> <code>signing_</code> <code>coin_distribution_token_</code> <code>point_bound,</code> <code>signing_</code> <code>coin_distribution_</code> <code>token_spread,</code> <code>signing_</code> <code>coin_distribution_</code> <code>domain,</code> <code>signing_coin_</code> <code>distribution_domain_</code> <code>full,</code> <code>signing_coin_</code> <code>distribution_ok</code>	314–387. Machine- checked fact.	Coin sampling has the stated token point bound, support domain, full-domain mass, and distribution-ok contract.	Imports <code>mu</code> , <code>dmapE</code> , interval support, and real-order facts. Used by <code>HAETAE_ROM_Programming.ec</code> and HopGames sampler-prequery bounds.
<code>coeff_distribution_</code> <code>lossless,</code> <code>poly_</code> <code>distribution_lossless,</code> <code>polyveck_distribution_</code> <code>lossless,</code> <code>polyvecl_</code> <code>distribution_lossless,</code> <code>matrix_distribution_</code> <code>lossless,</code> <code>challenge_</code> <code>distribution_lossless</code>	390–420. Machine- checked fact.	Primitive coefficient, polynomial, vector, matrix, and structural challenge distributions are lossless.	Imports <code>dinter_ll</code> , <code>dlist_ll</code> , <code>dmap_ll</code> , and CRH losslessness. These facts justify well-formed random-oracle output carriers, not SHAKE security.

<code>signing_sample_pair_of_coins_wf, signing_sample_pair_of_coins_unit, signing_sample_bound_gt0, unit_coeff_bounded_by_signing_sample_bound, poly_unit_bounded_by_signing_sample_bound, polyvecl_unit_bounded_by_signing_sample_bound, polyveck_unit_bounded_by_signing_sample_bound, signing_sample_pair_unit_bounded, signing_sample_pair_of_coins_sample_ok</code>	423–513. Machine-checked fact.	The deterministic sample pair derived from coins is well-formed, unit, and bounded by the formal signing-sample bound.	Imports algebraic signer-field <code>_wf/_unit</code> lemmas and <code>mode_ln_gt0</code> . Used by structural sample support and current-signing attempt lemmas.
<code>structural_signing_sample_pair_lossless, structural_signing_sample_pair_support, structural_signing_sample_pair_sample_ok, structural_signing_sample_pair_distribution_ok</code>	516–543. Machine-checked fact.	The deterministic structural sample-pair distribution is lossless and supports sample-ok pairs.	Imports <code>dunit_ll</code> and deterministic sample lemmas. Used by <code>HAETAE_Rejection.ec</code> structural attempt-state samplers.
<code>signing_unit_coeff_lossless, signing_unit_coeff_support, signing_bounded_coeff_cardinality_positive, signing_bounded_coeff_lossless, signing_bounded_coeff_support, signing_bounded_coeff_supportP, signing_bounded_coeff_point_bound, signing_bounded_coeff_point_bound_nonnegative, dlist_size_point_bound, dlist_point_bound, hyperball_coeff_point_bound_nonnegative, hyperball_coeff_point_boundE</code>	553–674. Machine-checked fact.	Unit and bounded coefficient distributions have the stated support, losslessness, nonnegative point bounds, and list-point-bound lifting lemmas.	Imports <code>duniform</code> , <code>dinter</code> , list-distribution, and real multiplication/order facts. Used by hyperball polynomial/vector point bounds and sampling-loss book-keeping.

<code>signing_unit_poly_</code> <code>lossless, signing_</code> <code>unit_poly_support,</code> <code>signing_bounded_poly_</code> <code>lossless, signing_</code> <code>bounded_poly_support,</code> <code>signing_bounded_poly_</code> <code>supportP, hyperball_poly_</code> <code>point_bound_nonnegative,</code> <code>hyperball_poly_lossless,</code> <code>hyperball_poly_support,</code> <code>hyperball_poly_supportP,</code> <code>hyperball_poly_point_</code> <code>boundE</code>	676–777. Machine- checked fact.	Polynomial unit/bounded/hyperball distributions are lossless, have the intended support predicates, and satisfy the product point bound.	Premises include list sizes governed by n . Used by vector sampler support and exact-hyperball pair point-bound proofs.
Vector support, losslessness, and point-bound families for <code>dsigning_unit_polyvec1,</code> <code>dsigning_unit_polyveck,</code> <code>dsigning_bounded_</code> <code>polyvec1, dsigning_</code> <code>bounded_polyveck,</code> <code>dhyperball_polyvec1,</code> and <code>dhyperball_polyveck</code>	778–1027. Machine- checked fact.	Product vector samplers are lossless, support the corresponding unit/bounded or hyperball predicates, and have nonnegative point bounds.	Imports <code>mode_l_gt0,</code> <code>mode_k_gt0,</code> and polynomial support/point lemmas. Used by checked and exact sample-pair support and by Rejection attempt-state support equivalences.
<code>bounded_unit_signing_</code> <code>sample_pair_lossless,</code> <code>bounded_unit_signing_</code> <code>sample_pair_support,</code> <code>bounded_unit_signing_</code> <code>sample_pair_sample_</code> <code>ok, bounded_unit_</code> <code>signing_sample_pair_</code> <code>distribution_ok,</code> <code>checked_hyperball_</code> <code>signing_sample_pair_</code> <code>lossless, checked_</code> <code>hyperball_signing_</code> <code>sample_pair_support,</code> <code>checked_hyperball_</code> <code>signing_sample_pair_</code> <code>distribution_ok,</code> <code>hyperball_sample_</code> <code>ok_sample_ok, exact_</code> <code>hyperball_signing_</code> <code>sample_pair_lossless,</code> <code>exact_hyperball_signing_</code> <code>sample_pair_support,</code> <code>exact_hyperball_signing_</code> <code>sample_pair_supportP,</code> <code>exact_hyperball_signing_</code> <code>sample_pair_sample_ok,</code> <code>exact_hyperball_</code> <code>signing_sample_pair_</code> <code>distribution_ok</code>	1029–1182. Machine- checked fact.	The bounded-unit, checked-hyperball, and current exact-hyperball pair distributions are lossless and satisfy the sample-pair distribution contract.	Imports component vector support/losslessness. Used directly by <code>HAETAE_Rejection.ec</code> sampler-ok lemmas and by HopGames exact-hyperball sampling procedures.

<code>exact_hyperball_signing_</code> <code>sample_pair_point_bound,</code> <code>exact_hyperball_signing_</code> <code>sample_pair_checkedE</code>	1192–1208. Machine- checked fact.	The current exact-hyperball pair distribution has the product point bound and is equal to the checked bounded-product carrier.	Imports product-distribution point-bound facts. This is the strongest local theorem boundary for sampling: it identifies two formal carriers, not a paper-sampler equivalence or a rejection-loop correspondence proof.
--	---	--	---

1.27 Script companion coverage for HAETAE Rejection

`HAETAE_Rejection.ec` is the bridge between deterministic signing fields, attempt-state sampling, transcript logs, and rejection-sampling loss terms. It is where many later hop premises get their vocabulary.

Script object family	Mathematical reading	Downstream role
Attempt-state records	<code>signing_transcript_record</code> , <code>signing_attempt_sample</code> , and <code>signing_attempt_state</code> package coins, sample vectors, commitment fields, challenge, response vectors, hint, and signature.	<code>HAETAE_HopGames.ec</code> samples and trans- forms these states in paper-simulation and exact-hyperball games.
Field reconstruction predicates	<code>signing_attempt_state_</code> <code>programmed_lowbits</code> , <code>signing_</code> <code>attempt_state_lowbits_</code> <code>consistent</code> , <code>signing_attempt_</code> <code>state_programmed_challenge</code> , <code>signing_attempt_state_public_</code> <code>equation_consistent</code> , and <code>signing_attempt_state_fields_</code> <code>match_signature</code> .	Used to prove that a sampled attempt can be read as a valid tran- script and as a paper- simulation signature.
Rejection predicates	<code>signing_attempt_state_reject1_*</code> , <code>signing_attempt_state_reject2_*</code> , <code>signing_attempt_state_pack_*</code> , <code>signing_attempt_state_</code> <code>reference_rejection_*</code> , <code>signing_</code> <code>attempt_state_rejection_*</code> , and <code>signing_attempt_state_aborts</code> split the rejection event into named components.	Hop games charge or erase these events when moving from real signing to paper-simulation and exact-hyperball sampling games.

Attempt distributions	dsigning_attempt_state, dstructural_signing_attempt_ state, dbounded_unit_signing_ attempt_state, dchecked_ hyperball_signing_attempt_state, and dexact_hyperball_signing_ attempt_state lift sample-pair distributions to complete attempt states.	Security/hop lemmas compare success prob- abilities under these distributions.
Signing and transcript re- lations	signing_attempt_relation, signing_simulation_relation, signing_transcript_relation, signing_record_relation, signing_record_log_sound, and transcript_log_valid connect attempts to transcripts.	Used by transcript- log invariants in HAETAE_HopGames.ec and by ROM pro- gramming no-failure predicates.
Loss obligations	structural_to_exact_hyperball_ sample_pair_*, structural_ to_exact_hyperball_coin_ sample_pair_*, structural_ to_exact_hyperball_attempt_ loss_obligation, and rejection_ sampling_bound_obligation name the loss assumptions and nonnegativ- ity premises.	Some are conditional premises for hop lem- mas. The nonnegativity obligation is proved from HAETAE_Reductions.ec terms.

Lemma family	Checked fact	Boundary
Current-signing field equations	signing_attempt_state_of_coins_ signatureE, signing_attempt_ state_of_coins_lowbitsE, signing_ attempt_state_of_coins_challengeE, signing_attempt_state_of_coins_ field_accepts, signing_attempt_ state_of_coins_accepts_current, and signing_attempt_state_of_coins_ rejection_accepts_current prove that the formal current signer does not abort.	Machine- checked for the formal signer.
Sampler lifting and support	dsigning_attempt_state_from_ sample_pair_sampler_lossless, dexact_hyperball_signing_attempt_ state_supportP, and distribution sup- port lemmas connect samples, coins, and full states.	Machine- checked for supplied sampler surfaces.

Abort decomposition and zero facts	<code>signing_attempt_state_reference_rejection_aborts_componentE</code> , <code>union-bound lemmas</code> , <code>dexact_hyperball_signing_attempt_state_reject1_abort_zero</code> , <code>dexact_hyperball_signing_attempt_state_reject2_abort_zero</code> , and related zero-abort lemmas show which rejection components vanish under the checked exact-hyperball attempt state.	Machine-checked; relies on the current exact-hyperball carrier definition.
Loss-obligation propagation	<code>structural_to_exact_hyperball_sample_pair_loss_from_point_loss</code> , <code>structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss</code> , and <code>structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss</code> lift a point/sample loss premise to an attempt-state loss premise.	Conditional unless the initial point-loss obligation is supplied.
Record-log soundness	<code>sign_internal_record_relation</code> , <code>signing_record_relation_sound</code> , <code>signing_record_log_sound_cons</code> , and <code>signing_record_log_sound_transcripts_valid</code> turn signing records into valid transcript logs.	Machine-checked; consumed by HopGames transcript invariants.

Boundary: this file is the precise place where rejection-sampling claims can become overclaimed. The checked scripts prove many zero-abort and propagation facts for the formal carrier. A paper-faithful sampler replacement still needs its own checked correspondence or must remain a conditional premise.

1.27.1 Line-by-line proof-script companion for HAETAE Rejection

Focused criticism for this pass: the previous report named the rejection object families, but it did not let a reader reconstruct the proof boundary of `HAETAE_Rejection.ec`. It underexplained that many no-abort theorems are consequences of the current formal signer setting branch bytes to zero and of the current exact-hyperball support being lifted through `signing_attempt_state_of_sample_pair`; it also did not expose the conditional loss-obligation chain from point loss to full attempt-state loss. The line-range companion below records the proof obligations that a reader must track when replaying the EasyCrypt script.

Lines	Script content	Classification	Companion reading and downstream role
-------	----------------	----------------	---------------------------------------

1–14	Imports, theory wrapper, and real-order import.	Imported/library facts.	<code>Distr</code> , <code>Real</code> , <code>StdOrder</code> , and <code>RealSeries</code> provide probability mass <code>mu</code> , support, <code>dmap</code> , <code>dlet</code> , union bounds, summability, and nonnegative real arithmetic. <code>HAETAE_Algebra.ec</code> , <code>HAETAE_Distributions.ec</code> , <code>HAETAE_Events.ec</code> , <code>HAETAE_Reductions.ec</code> , and <code>HAETAE_Transcript.ec</code> provide the domain-specific predicates used below.
16–96	Attempt records, samples, state tuple layout, projections, rejection-branch byte/bit, and reject2 balance vectors.	Machine-checked definitions.	These declarations define the state shape that hop games manipulate. The branch bit is read from signature component 6; because <code>sign_internal</code> sets that component to zero, later current-signing and sample-pair no-abort lemmas can prove reject2 is clear.
102–163	Programmed lowbits, lowbits consistency, programmed challenge, reconstructed highbits, public-equation consistency, signature reconstruction, and field acceptance.	Machine-checked definitions.	This block states the field-equality obligations needed to recognize a sampled attempt as a valid formal signature/transcript. Downstream <code>HAETAE_HopGames.ec</code> invariants use these predicates when comparing real signing, simulated signing, and ROM-programmed transcripts.
166–199	<code>signing_attempt_state_of_coins</code> , <code>signing_attempt_sample_of_pair</code> , and <code>signing_attempt_state_of_sample_pair</code> .	Machine-checked definitions.	The first constructor builds the current formal signing attempt from coins. The second and third constructors replace the sampled pair while keeping the deterministic signing fields from <code>HAETAE_Algebra.ec</code> . This is the bridge used by structural, bounded, checked-hyperball, and exact-hyperball attempt distributions.

203–339	Acceptance predicates, <code>reject1/reject2/pack/pick/verify</code> rejection predicates, verification acceptance, full rejection acceptance, abort predicates, and <code>signing_accepts</code> .	Machine-checked definitions; <code>ex-rejection</code> boundary.	The script decomposes rejection into named events. These definitions are not probability bounds by themselves; bounds appear only in later lemmas using <code>mu</code> . The names are the event vocabulary consumed by hop-game failure terms.
345–421	Attempt-state distributions and sample-pair shape predicates.	Machine-checked definitions plus imported distribution interface.	<code>dsigning_attempt_state</code> maps <code>drandom_coins</code> ; the other distributions use <code>dlet</code> over coins and then map a sample-pair distribution into a full attempt. Losslessness and support are proved later, conditional on <code>sampler-ok</code> predicates supplied by <code>HAETAE_Distributions.ec</code> .
438–488	Signing relation, simulation relation, transcript relation, record projections, record log soundness, and transcript-log validity.	Machine-checked definitions.	These predicates connect rejection attempts to transcript reasoning. The security games use them to say that a logged signing query corresponds to a valid formal transcript under the public key.
491–523	Point, sample-pair, coin-sample-pair, and attempt-state loss obligations, plus nonnegativity obligation.	Conditional premises and machine-checked definitions.	<code>structural_to_exact_hyperball_sample_pair_point_loss_obligation</code> is not proved in this file. It is a conditional premise used to derive the larger distribution-comparison obligations. By contrast, <code>rejection_sampling_bound_obligation</code> is later proved from nonnegativity lemmas imported from <code>HAETAE_Reductions.ec</code> .

525–569	<code>signing_attempt_verifies</code> , <code>signing_simulation_verifies</code> , <code>signing_attempt_simulates</code> , and <code>sign_internal_attempt_relation</code> .	Machine-checked facts.	These lemmas lift <code>HAETAE_Algebra.ec</code> 's <code>verify_internal_sign_internal</code> into the rejection relation vocabulary. The proof obligation is mostly unfolding <code>valid_keypair</code> and applying formal correctness and norm lemmas.
571–623	Current sample-pair equality and sampler-ok lemmas for structural, bounded, checked-hyperball, and exact-hyperball samplers.	Machine-checked facts relying on imported distribution facts.	The sampler-ok lemmas are adapters: they expose the distribution-ok facts from <code>HAETAE_Distributions.ec</code> in the shape required by <code>dsigning_attempt_state_from_sample_pair_sampler</code> .
625–747	Losslessness and support/sample-ok lemmas for lifted attempt distributions.	Machine-checked facts with imported distribution rules.	The proofs use <code>dlet_ll</code> , <code>dmap_ll</code> , <code>supp_dlet</code> , and <code>supp_dmap</code> . They show that if the underlying sampler is ok, then the full attempt distribution is lossless and supported on well-formed bounded sample pairs.
749–799	Exact-hyperball support equivalence, one-bound, and pullback identity.	Machine-checked facts; model bound-ary.	<code>dexact_hyperball_signing_attempt_state_supportP</code> expands support into an existential over coins and a hyperball sample. This is the key lemma used repeatedly to turn a probabilistic zero-abort goal into a pointwise no-abort proof. It inherits the current formal definition of <code>dexact_hyperball_signing_sample_pair</code> ; it does not prove paper-sampler faithfulness.
801–850	Reference-rejection abort decomposition and union bound.	Machine-checked facts with imported probability facts.	<code>signing_attempt_state_reference_rejection_aborts_componentE</code> rewrites the negation of a conjunction into a disjunction. The union-bound lemma uses <code>mu_or</code> and <code>mu_bounded</code> ; these are library probability facts, not HAETAE-specific theorems.

852–964	Pack no-abort for sample-pair states and equality of structural/exact coin-sample-pair mappings.	Machine-checked facts.	Pack acceptance follows from <code>valid_signature_sign_internal</code> and <code>mode_crypto_bytes_gt0</code> . Distribution equalities are functorial rewrites using <code>dmap_dunit</code> , <code>dlet_dunit</code> , <code>dmap_dlet</code> , and <code>dmap_comp</code> .
966–1053	<code>mu_dlet_le_add_all</code> and the loss-obligation propagation chain.	Machine-checked facts conditional on the earlier point/sample loss premise.	The probabilistic lemma proves that a per-coin distribution inequality with additive ϵ lifts through <code>dlet</code> . The three propagation lemmas then derive sample-pair loss, coin-sample-pair loss, and full attempt-state loss. The initial point-loss premise remains conditional unless separately proved.
1055–1270	Field-equality lemmas for <code>signing_attempt_state_of_coins</code> and current field acceptance/no-abort.	Machine-checked facts.	The proofs are definitional rewrites showing that every projection of the attempt state equals the corresponding deterministic field from <code>HAETAE_Algebra.ec</code> . These lemmas are the backbone of current-signing transcript validity and current no-abort facts.
1272–1513	Reject1 bound lift, reject2 concrete rewrites, branch-clear proofs, sample-pair reject1/reject2/pack/hint no-abort, and exact-hyperball reference-rejection abort-zero chain.	Machine-checked facts with imported probability facts.	The pointwise no-abort lemmas rely on <code>sign_internal</code> 's branch byte being zero and on formal current norm bounds. The distribution zero lemmas use <code>dexact_hyperball_signing_attempt_state_supportP</code> , <code>mu_le</code> , <code>mu0</code> , and <code>mu_bounded</code> . The conclusion is zero abort in the current formal exact-hyperball attempt model, not a paper-level rejection-rate theorem.

1515–1672	Current-coin reject2 branch clearing, balance identities, reference-rejection acceptance, verification acceptance, full rejection acceptance, and full current no-abort.	Machine-checked facts.	These lemmas show that attempts generated directly from current signing coins pass all formal acceptance predicates. They are used later to prove zero abort probability for <code>dsigning_attempt_state</code> .
1674–1724	<code>dsigning_attempt_state</code> losslessness, entropy-token spread, and current reference/full rejection abort-zero probabilities.	Machine-checked facts with imported distribution facts and a distribution point-bound imported from <code>HAETAE_Distributions.ec</code> .	The token-spread lemma maps the signing-coin token bound through the attempt state. This feeds sampler-prequery and ROM bad-event arguments in <code>HAETAE_HopGames.ec</code> .
1726–1838	Simulation relation, transcript validity, transcript relation, record relation, record-log soundness, and query/transcript list constructors.	Machine-checked facts.	These lemmas connect accepted signing attempts to transcript logs. They are used by hop games and ROM programming to maintain that logged signing transcripts are valid and field-consistent.
1840–1854	Nonnegativity facts for rejection and Fiat-Shamir-with-aborts loss terms.	Machine-checked facts relying on imported reduction-term lemmas.	<code>rejection_sampling_bound_obligation_holds</code> is checked here, but the meaning of the loss terms comes from <code>HAETAE_Reductions.ec</code> . It proves nonnegativity only, not smallness or cryptographic tightness.
1856–2130	Field-equality and no-abort lemmas for <code>signing_attempt_state_of_sample_pair</code> .	Machine-checked facts.	This is the sample-pair analogue of the current-coins field-equality block. Because the deterministic signature fields still come from <code>sign_internal</code> , the proofs again reduce to definitional rewrites and the formal correctness/norm lemmas from <code>HAETAE_Algebra.ec</code> .

2132–2202	Exact-hyperball support implies reference rejection acceptance, full rejection acceptance, no-abort predicates, and zero full-rejection abort probability.	Machine-checked facts with model boundary.	The proof repeatedly expands exact-hyperball support into coins/sample witnesses and rewrites the state to the sample-pair constructor. The final zero-probability theorem is therefore conditional on the current formal exact-hyperball support definition; replacing that sampler requires replaying or reestablishing these support-to-no-abort arguments.
-----------	--	--	--

For downstream use, the theorem boundary is as follows. Machine-checked: state constructors, field predicates, support/lossless facts, current-formal no-abort facts, record-log soundness, and the conditional propagation from sample-pair loss to attempt-state loss. Conditional: the initial structural-to-exact hyperball point-loss obligation unless separately proved. Imported: probability algebra, list algebra, real summability, and the nonnegativity of loss terms from `HAETAE_Reductions.ec`. External or explanatory: the claim that the exact-hyperball carrier is the paper HAETAE sampler, the cryptographic interpretation of the loss terms, and any implementation-level rejection-sampling equivalence.

Rejection obligation sheet. The four loss obligations in lines 491–519 are not interchangeable. The script proves implications between them, but the first one remains the entry-point premise unless another file supplies it. To keep the formulas readable, write D_s^{str} for `dstructural_signing_sample_pair`, D_s^{ex} for `dexact_hyperball_signing_sample_pair`, D_c^{str} and D_c^{ex} for the structural and exact coin-sample-pair distributions, D_a^{str} for `dsigning_attempt_state`, D_a^{ex} for `dexact_hyperball_signing_attempt_state`, S for `signing_sample_pair_of_coins`, and L_{rej} for `rejection_sampling_loss_term`. With that shorthand, the EasyCrypt obligations are:

$$\begin{aligned}
O_{\text{point}} & \quad \forall md, sk, m, ctx, coins. 1 \leq \mu(D_s^{ex}(md))(\text{pred1}(S(md, sk, m, ctx, coins))) + L_{rej}, \\
O_{\text{sample}} & \quad \forall md, sk, m, ctx, coins, p. \mu(D_s^{str}(md, sk, m, ctx, coins))(p) \leq \mu(D_s^{ex}(md))(p) + L_{rej}, \\
O_{\text{coin}} & \quad \forall md, sk, m, ctx, p. \mu(D_c^{str}(md, sk, m, ctx))(p) \leq \mu(D_c^{ex}(md, sk, m, ctx))(p) + L_{rej}, \\
O_{\text{attempt}} & \quad \forall md, sk, m, ctx, p. \mu(D_a^{str}(md, sk, m, ctx))(p) \leq \mu(D_a^{ex}(md, sk, m, ctx))(p) + L_{rej}.
\end{aligned}$$

Obligation	Status in HAETAE_Rejection.ec	Downstream reading
O_{point}	Conditional premise.	This is the carrier-level point-mass statement from which the script can derive larger loss comparisons. It must not be reported as proved by <code>HAETAE_Rejection.ec</code> .

O_{sample}	Machine-checked implication from O_{point} .	implica-	<code>structural_to_exact_hyperball_sample_pair_loss_from_point_loss</code> uses the structural sample's dunit form and monotonicity of <code>mu</code> .
O_{coin}	Machine-checked implication from O_{sample} .	implica-	<code>structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss</code> uses <code>mu_dlet_le_add_all</code> to lift the bound through coins.
O_{attempt}	Machine-checked implication from O_{coin} .	implica-	<code>structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss</code> uses distribution equalities that map coin/sample pairs into full attempt states.
<code>rejection_sampling_bound_obligation</code>	Machine-checked from imported lemmas.	locally nonnegativ- ity	This proves only nonnegativity of the three named loss terms. It does not prove that the losses are small or that the exact-hyperball law is paper faithful.

Carrier-specific theorem boundary.

Carrier or distribution	Checked theorem surface	Boundary
<code>signing_attempt_state_of_coins</code>	All current-field equalities, current acceptance, current rejection no-abort, zero abort probability for <code>dsigning_attempt_state</code> , and token-spread bound.	Machine-checked for the formal current signer and formal <code>drandom_coins</code> . Does not by itself prove paper rejection-sampling statistics.
<code>signing_attempt_state_of_sample_pair</code>	Field equalities, reference rejection acceptance, verification acceptance, full rejection acceptance, and no-abort predicates for any supplied sample pair.	Machine-checked because the deterministic signature fields still come from <code>sign_internal</code> . If a future sampler changes those fields, this boundary must be revisited.
<code>dexact_hyperball_signing_attempt_state</code>	Support characterization, pack/reject1/reject2/hint zero-abort probabilities, reference rejection zero abort, and full rejection zero abort.	Machine-checked for the current formal exact-hyperball carrier. The paper hyperball/sphere sampler correspondence is outside this file unless separately proved.

Paper	HAETAE sam- pler	No direct theorem in this file.	External or conditional. The report must not claim that these zero-abort lemmas certify the paper sampler without a checked replacement/correspondence argument.
-------	---------------------	---------------------------------	--

Two nontrivial proof blocks. For `mu_dlet_le_add_all`, the mathematical content is an additive-loss lifting rule: if every branch distribution $F_1(x)$ is bounded by $F_2(x) + \epsilon$, and $\epsilon \geq 0$, then the `dlet`-mixtures obey the same additive bound. The proof expands `dletE`, proves summability of the weighted series using imported `summable_mu1_wght`, `summableD`, and `summableZ`, applies `RealSeries.ler_sum`, and uses `weightE` plus `mu_bounded` for the final real-arithmetic side condition.

For the abort-zero chain, the pattern is pointwise-to-probabilistic. The script first proves that sample-pair states cannot trigger the relevant abort predicate. Then, under exact-hyperball support, every sampled state is rewritten to such a sample-pair state. Finally `mu_le` bounds the abort event by `pred0`, `mu0` rewrites the probability of `pred0` to zero, and `mu_bounded/SMT` closes the equality. This pattern proves zero probability for the current formal carrier; it is not a paper-level rejection rate proof.

Lemma-ledger criticism for HAETAE Rejection. Even after the line-range companion above, the report plus `HAETAE_Rejection.ec` still required too much reverse engineering for proof-obligation reconstruction. The missing layer was a ledger that says, for each security-relevant definition or lemma surface, what the checked statement is, what premises or imported facts it depends on, and how it feeds `HAETAE_HopGames.ec` or `HAETAE_Security.ec`. The ledger below adds that layer. Pure tuple projections and repeated field-equality siblings are grouped when they share the same mathematical proof obligation; the row lists the relevant EasyCrypt names and line ranges so the source remains the authority for exact syntax and tactic order. In particular, the ledger names the token/prequery bridge, the abort-channel predicates, the current-formal zero-abort theorems, the transcript/log bridge theorems, and the exact-hyperball support-to-no-abort interface, because these are the surfaces later games can actually cite.

Definition ledger for HAETAE Rejection.

EasyCrypt object(s)	Lines / cate- gory	Statement paraphrase	Premises, imports, downstream role
<code>signing_transcript_</code> <code>record,</code> <code>signing_</code> <code>attempt_sample,</code> <code>signing_attempt_state</code>	16–20. Machine- checked defini- tions.	Tuple carriers for logged signing transcripts, sampled vectors, and full signing-attempt state.	No premises. Imported carriers from <code>HAETAE_Algebra.ec</code> . <code>HopGames</code> uses these as the state space for rejection and transcript-signing games.
<code>signing_attempt_state_*</code> projections, <code>signing_</code> <code>attempt_sample_*</code> , <code>signature_rejection_</code> <code>branch_byte</code>	22–70. Machine- checked defini- tions.	Projection vocabulary for coins, sample components, commitment fields, challenge, response, hint, signature, and <code>reject2</code> branch bit.	No premises. Tuple projection only. The <code>reject2</code> branch reads signature field 6; this matters because <code>sign_internal</code> writes zero there.

polyvec1_double, signing_attempt_state_ reject2_sampled_*, signing_attempt_state_ reject2_balance_*	73–91. Machine- checked defini- tions.	Defines the formal reject2 bal- ance vectors $2z - y$ and $2z_{\text{aux}} - y_2$.	Imports formal vec- tor arithmetic from <code>HAETAE_Algebra.ec</code> . HopGames uses these predicates when charging or erasing reject2 aborts.
signing_attempt_ state_lowbits_ carrier, signing_ attempt_state_token, signing_attempt_state_ programmed_lowbits, signing_attempt_state_ lowbits_consistent	93–114. Machine-checked definitions.	Names the lowbits carrier and signing-token triple, then recon- structs lowbits by placing the signing entropy token in coeffi- cient zero and requiring equal- ity with the attempt state’s low- bits.	Depends on <code>signing_ entropy_token_of_coins</code> . Used by transcript and ROM programming in- variants and the token point-bound lemma that bind sampler coins to challenge input.
signing_attempt_state_ programmed_challenge, signing_attempt_state_ challenge_consistent	116–128. Machine-checked definitions.	Recomputes the challenge from response auxiliary data, low- bits, and message hash under the public key.	Imports <code>challenge_hash</code> and <code>message_hash</code> . Used to show sampled attempts match the same challenge predicate as verification.
signing_attempt_state_ reconstructed_highbits, signing_attempt_ state_public_equation_ consistent	130–139. Machine-checked definitions.	Rebuilds highbits from auxil- iary response and public-key challenge term, then requires equality with the attempt com- mitment.	Imports public-key chal- lenge operations from <code>HAETAE_Algebra.ec</code> . Feeds transcript validity and simulated-signing sound- ness.
signing_attempt_state_ signature_of_fields, signing_attempt_state_ fields_match_signature, signing_attempt_state_ field_accepts	141–163. Machine-checked definitions.	Assembles the signature tuple from attempt fields and com- bines lowbits, challenge, public- equation, and signature-field consistency.	Imports signature layout from <code>HAETAE_Algebra.ec</code> . HopGames uses this as the field-consistency side condi- tion for accepted sampled transcripts.
signing_attempt_state_ of_coins, signing_ attempt_sample_of_pair, signing_attempt_state_ of_sample_pair	166–199. Machine-checked definitions.	Constructs current signing at- tempts from coins, or attempts that replace only the sampled pair while keeping deterministic signer fields.	Imports <code>sign_internal</code> , <code>commitment_*</code> , <code>response_*</code> , and <code>hint_vector</code> . These constructors are the bridge between real signing, structural attempts, and exact-hyperball attempts.
signing_attempt_state_ valid_signature_accepts, signing_attempt_state_ response_norm_accepts, signing_attempt_state_ hint_norm_accepts, signing_attempt_state_ accepts	203–218. Machine-checked definitions.	Defines the non-rejection valid- ity and norm acceptance com- ponents.	Imports <code>valid_signature</code> , <code>response_norm_ok</code> , and <code>verify_norm_ok</code> . Used by current no-abort and tran- script soundness lemmas.
signing_attempt_ reject1_bound, signing_ attempt_reject2_bound, signing_attempt_ reject2_balance_norm_sq, signing_attempt_ reject2_concrete_aborts	221–238. Machine-checked definitions.	Defines reject1/reject2 thresh- olds and the concrete reject2 event $\text{branch} \wedge \ 2z - y\ ^2 < B_0 \ell_n^2$.	Imports mode bounds from <code>HAETAE_Params.ec</code> and norm functions from <code>HAETAE_Algebra.ec</code> . Used in the rejection-abort de- composition in HopGames.

<code>signing_attempt_</code> <code>state_reject1_*</code> , <code>signing_attempt_state_</code> <code>reject2_*</code> , <code>signing_</code> <code>attempt_state_pack_*</code> , <code>signing_attempt_state_</code> <code>reference_rejection_*</code> , <code>signing_attempt_</code> <code>state_rejection_*</code> , <code>signing_attempt_state_</code> <code>response_norm_aborts</code> , <code>signing_attempt_state_</code> <code>hint_norm_aborts</code> , <code>signing_attempt_state_</code> <code>rejection_sampling_</code> <code>aborts</code> , <code>signing_</code> <code>attempt_state_aborts</code> , <code>signing_accepts</code>	240–343. Machine-checked definitions.	Names the <code>reject1</code> , <code>reject2</code> , <code>pack</code> , <code>reference-rejection</code> , <code>verification</code> , <code>full rejection</code> , <code>concrete abort-channel</code> , <code>coarse abort</code> , and <code>signing-acceptance</code> predicates.	No cryptographic assumption locally. These event predicates are the exact bad-event and loss vocabulary used in HopGames; the report must not treat them as probability bounds until later μ lemmas are applied.
<code>dsigning_attempt_state</code> , <code>dsigning_attempt_</code> <code>state_from_sample_pair_</code> <code>sampler</code> , <code>dstructural_*</code> , <code>dbounded_*</code> , <code>dchecked_*</code> , <code>dexact_*</code>	345–419. Machine-checked definitions.	Defines distributions over attempts by mapping signing coins and sample-pair distributions into full states.	Imports <code>drandom_coins</code> and <code>sample-pair</code> distributions from <code>HAETAEDistributions.ec</code> . Used by hop probability comparisons and sampler-prequery arguments.
<code>signing_attempt_</code> <code>state_sample_pair_wf</code> , <code>signing_attempt_state_</code> <code>sample_pair_unit</code> , <code>signing_attempt_state_</code> <code>sample_pair_bounded</code>	421–435. Machine-checked definitions.	Support predicates for sampled y_1, y_2 components.	Imports <code>polyvec1_wf</code> , <code>polyvec1_wf</code> , <code>unit</code> and bounded predicates. Used by support lemmas and sampler-correctness checks.
<code>signing_attempt_</code> <code>relation</code> , <code>signing_</code> <code>simulation_relation</code> , <code>signing_transcript_</code> <code>relation</code>	438–456. Machine-checked definitions.	Relates a signature to an honest attempt, wraps it as a verified simulation, and then as a valid transcript.	Imports <code>valid_keypair</code> , <code>verify_internal</code> , and transcript predicates. Used by NMA transcript construction and signing-log soundness.
<code>signing_record_*</code> , <code>signing_record_log_</code> <code>sound</code> , <code>transcript_log_</code> <code>valid</code>	459–488. Machine-checked definitions.	Projects record fields, lists record queries/transcripts, and states log soundness/validity.	Imports list <code>map</code> and transcript validity. HopGames consumes these to preserve transcript logs across signing-oracle games.
<code>structural_to_exact_</code> <code>hyperball_*obligation</code> , <code>rejection_sampling_</code> <code>bound_obligation</code>	491–523. Conditional premises plus checked definitions.	States the point, sample-pair, coin-sample-pair, attempt-state loss obligations and the nonnegativity obligation.	O_{point} is not proved here. Nonnegativity is proved later from <code>HAETAEReductions.ec</code> . These obligations feed the exact-hyperball transition in HopGames and the final security bound.

Lemma ledger for HAETAETAE Rejection.

EasyCrypt lemma(s)	Lines / category	Statement paraphrase	Premises, imports, downstream role
--------------------	------------------	----------------------	------------------------------------

<code>signing_attempt_verifies,</code> <code>signing_simulation_</code> <code>verifies,</code> <code>signing_</code> <code>attempt_simulates,</code> <code>sign_internal_attempt_</code> <code>relation</code>	525–569. Machine- checked facts.	Honest signing attempts ver- ify, simulation relations ver- ify, attempts lift to simulation relations, and <code>sign_internal</code> forms an honest attempt.	Imports <code>verify_</code> <code>internal_sign_internal,</code> <code>valid_signature_sign_</code> <code>internal,</code> <code>response_</code> <code>norm_ok_current,</code> and <code>verify_norm_ok_current.</code> Used to justify signing- oracle transcripts.
<code>signing_attempt_sample_</code> <code>of_pair_currentE,</code> <code>signing_attempt_state_</code> <code>of_sample_pair_currentE</code>	571–591. Machine- checked facts.	The structural sample pair built from the current coins reconstructs the same attempt as <code>signing_attempt_state_</code> <code>of_coins.</code>	Definitional rewrites using <code>HAETAE_Distributions.ec.</code> Used to identify structural attempts with current signing attempts.
<code>structural_signing_</code> <code>sample_pair_sampler_ok,</code> <code>bounded_unit_signing_</code> <code>sample_pair_sampler_ok,</code> <code>checked_hyperball_</code> <code>signing_sample_</code> <code>pair_sampler_ok,</code> <code>exact_hyperball_signing_</code> <code>sample_pair_sampler_ok</code>	593–623. Machine- checked facts relying on imports.	Each named sample-pair sampler satisfies the generic sampler-ok interface.	Imports distribution- ok lemmas from <code>HAETAE_Distributions.ec.</code> Used to derive attempt- distribution losslessness and support properties.
<code>dsigning_attempt_</code> <code>state_from_sample_</code> <code>pair_sampler_lossless,</code> <code>dsigning_attempt_</code> <code>state_from_sample_pair_</code> <code>sampler_sample_ok</code>	625–676. Machine- checked facts with conditional premise.	If a supplied sample-pair sam- pler is ok, the lifted attempt distribution is lossless and sup- ported on well-formed bounded samples.	Premise: <code>signing_sample_</code> <code>pair_sampler_ok.</code> Imports <code>dlet_ll,</code> <code>dmap_ll,</code> <code>supp_</code> <code>dlet,</code> <code>supp_dmap.</code> Used by all sampler variants in HopGames.
<code>dstructural.*lossless,</code> <code>dbounded.*lossless,</code> <code>dchecked.*lossless,</code> <code>dexact.*lossless,</code> <code>dbounded.*sample_ok,</code> <code>dchecked.*sample_ok,</code> <code>dexact.*sample_ok</code>	678–747. Machine- checked facts with imported sampler facts.	Instantiates the generic lift- ing lemmas for structural, bounded-unit, checked- hyperball, and exact-hyperball attempt distributions.	Imports sampler-ok lem- mas above and <code>HAETAE_</code> <code>Distributions.ec.</code> Used as side conditions in proba- bilistic hop proofs.
<code>dexact_hyperball_</code> <code>signing_attempt_state_</code> <code>supportP,</code> <code>dexact_</code> <code>hyperball_signing_</code> <code>attempt_state_reference_</code> <code>rejection_abort_bound1,</code> <code>dexact_hyperball_</code> <code>signing_attempt_state_</code> <code>mu_pullback</code>	749–799. Machine- checked facts with model boundary.	Characterizes exact-hyperball attempt support, bounds any event by one, and rewrites the exact-hyperball attempt distribution as the underlying <code>dlet.</code>	Imports <code>exact_hyperball_</code> <code>signing_sample_pair_</code> <code>supportP</code> and <code>mu_bounded.</code> Used by all exact-hyperball abort-zero proofs.
<code>signing_attempt_state_</code> <code>reference_rejection_</code> <code>aborts_componentE,</code> <code>dexact_hyperball_</code> <code>signing_attempt_</code> <code>state_reference_</code> <code>rejection_abort_split,</code> <code>dexact_hyperball_</code> <code>signing_attempt_state_</code> <code>reference_rejection_</code> <code>abort_union_bound</code>	801–850. Machine- checked facts with imported probability facts.	Decomposes reference re- jection aborts into re- ject1/reject2/pack/hint events and upper-bounds their proba- bility by the sum of component probabilities.	Imports <code>predU,</code> <code>mu_eq,</code> <code>mu_</code> <code>or,</code> <code>mu_bounded.</code> Used when charging rejection compo- nents in hop bounds.

signing_attempt_state_of_sample_pair_signatureE, signing_attempt_state_of_sample_pair_pack_accepts, signing_attempt_state_of_sample_pair_pack_no_abort, dexact_hyperball_signing_attempt_state_pack_abort_zero	852–902. Machine-checked facts.	Sample-pair attempts use the current <code>sign_internal</code> signature, so pack accepts and exact-hyperball pack abort has probability zero.	Imports <code>valid_signature_sign_internal</code> , <code>mode_crypto_bytes_gt0</code> , support characterization, and <code>mu0/mu_1e</code> . Used to remove the pack component of rejection loss.
dstructural_signing_attempt_state_from_samplerE, dstructural_signing_attempt_stateE, dstructural_signing_attempt_state_from_sampler_coin_sample_pairE, dexact_hyperball_signing_attempt_state_coin_sample_pairE	904–964. Machine-checked facts with imported distribution functor laws.	Equates alternate distribution presentations so loss lemmas can move between coin/sample-pair and full-state distributions.	Imports <code>dmap_dunit</code> , <code>dlet_dunit</code> , <code>dmap_dlet</code> , <code>dmap_comp</code> . Used by the loss-obligation propagation chain.
mu_dlet_1e_add_all	966–1002. Machine-checked imported-library style fact.	Lifts a per-branch additive probability inequality through <code>dlet</code> .	Premises: $\epsilon \geq 0$ and per-branch inequality. Imports <code>RealSeries.ler_sum</code> , summability lemmas, <code>dletE</code> , <code>weightE</code> . Used to lift sample-pair loss through signing coins.
structural_to_exact_hyperball_sample_pair_loss_from_point_loss, structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss, structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss	1004–1053. Machine-checked implications from conditional premises.	Derives sample-pair, coin-sample-pair, and attempt-state loss obligations from the preceding obligation in the chain.	Conditional until O_{point} is supplied. Used by HopGames to charge the structural-to-exact-hyperball transition.
signing_attempt_state_of_coins_*E, signing_attempt_state_of_coins_*consistent, signing_attempt_state_of_coins_field_accepts, signing_attempt_state_of_coins_accepts_current	1055–1270. Machine-checked facts.	Every projected field of a current-coins attempt equals the corresponding deterministic signing field; therefore current attempts satisfy field and norm acceptance predicates.	Imports <code>sign_internal</code> , <code>commitment_*</code> , <code>signature_token</code> , <code>valid_signature_sign_internal</code> , and norm lemmas. Used for current signing soundness and zero-abort probabilities.
signing_attempt_reject1_bound_ge_response_bound, signing_attempt_state_reject1_accepts_from_response_norm_ok	1272–1287. Machine-checked facts.	Response-norm acceptance implies reject1 acceptance because the reject1 bound dominates the response bound.	Imports mode constants from <code>HAETAE_Params.ec</code> . Used in current and sample-pair reference-rejection acceptance proofs.

signing_attempt_ state_balance_norm_ sq_concreteE, signing_ attempt_state_reject2_ under_bound_concreteE, signing_attempt_state_ reject2_aborts_concreteE, signing_attempt_state_ reject2_accepts_from_ branch_clear	1289–1330. Machine- checked facts.	Unfolds reject2 state predicates and proves that clearing the branch bit implies reject2 acceptance.	Definitional dependencies only. Used by current and sample-pair reject2 no-abort lemmas.
signing_attempt_state_ of_sample_pair_reject2_ branch_clear, signing_ attempt_state_of_sample_ pair_reject2_accepts, signing_attempt_ state_of_sample_pair_ reject2_no_abort, dexact_hyperball_ signing_attempt_state_ reject2_abort_zero	1332–1382. Machine- checked facts with model boundary.	Because sign_internal writes zero in the branch byte, sample-pair states do not reject2-abort; exact-hyperball reject2 abort has probability zero.	Imports exact-hyperball support and probability zero pattern. Used to remove the reject2 component from exact-hyperball rejection loss.
dexact_hyperball_ signing_attempt_state_ reference_rejection_ abort_reduced_bound, dexact_hyperball_ signing_attempt_state_ reference_rejection_ abort_hint_bound, dexact_ hyperball_signing_ attempt_state_reference_ rejection_abort_zero	1384–1513. Machine- checked facts with model boundary.	Successively removes reject2, pack, reject1, and hint abort components until reference rejection abort probability is zero.	Imports component zero lemmas and mu_bounded . Used by HopGames when the exact-hyperball formal carrier is in force.
signing_attempt_state_ of_sample_pair_response_ norm_accepts, signing_ attempt_state_of_sample_ pair_reject1_accepts, signing_attempt_ state_of_sample_pair_ reject1_no_abort, dexact_hyperball_ signing_attempt_state_ reject1_abort_zero, signing_attempt_state_ of_sample_pair_hint_ norm_accepts, signing_ attempt_state_of_sample_ pair_hint_norm_no_abort, dexact_hyperball_ signing_attempt_state_ hint_norm_abort_zero	1401–1500. Machine- checked facts.	Sample-pair attempts inherit response and hint norm acceptance from current sign_internal ; exact-hyperball reject1 and hint aborts are zero.	Imports response_norm_ok_current , verify_norm_ok_current , and support-to-zero pattern. Used in the reference-rejection zero chain.

<code>signing_attempt_state_of_coins_reject2_*</code> , <code>signing_attempt_state_of_coins_balance_norm_sq_current</code> , <code>signing_attempt_state_of_coins_reject2_accepts_current</code>	1515–1587. Machine-checked facts.	Current-coins attempts have clear reject2 branch and the expected concrete balance norm decomposition.	Imports projection equalities and branch-byte zero fact. Used by current reference-rejection acceptance.
<code>signing_attempt_state_pack_accepts_from_valid</code> , <code>signing_attempt_state_reference_rejection_accepts_from_accepts</code> , <code>signing_attempt_state_of_coins_reference_rejection_accepts_current</code> , <code>signing_attempt_state_of_coins_verification_accepts_current</code> , <code>signing_attempt_state_of_coins_rejection_accepts_current</code> , <code>signing_attempt_state_of_coins_rejection_no_abort_current</code> , <code>signing_attempt_state_of_coins_no_abort_current</code>	1589–1672. Machine-checked facts.	Current-coins attempts pass pack, reference rejection, verification, full rejection, and coarse abort predicates.	Imports current field acceptance, norm acceptance, reject2 acceptance, and <code>mode_crypto_bytes_gt0</code> . Used by current distribution zero-abort probability lemmas.
<code>signing_attempt_state_distribution_lossless</code> , <code>signing_attempt_state_distribution_token_spread</code> , <code>signing_attempt_state_distribution_reference_rejection_abort_zero_current</code> , <code>signing_attempt_state_distribution_rejection_abort_zero_current</code>	1674–1724. Machine-checked facts.	The current attempt distribution is lossless, preserves the signing-token point bound, and has zero current reference/full rejection abort probability.	Imports <code>signing_coin_distribution_lossless</code> , <code>signing_coin_distribution_token_point_bound</code> , <code>dmapE</code> , and <code>mu0</code> . Used by sampler prequery and current-signing hop steps.
<code>sign_internal_simulation_relation</code> , <code>signing_attempt_transcript_valid</code> , <code>signing_attempt_transcript_relation</code> , <code>sign_internal_transcript_relation</code> , <code>sign_internal_record_relation</code>	1726–1790. Machine-checked facts.	Turns honest formal signing into simulation and transcript/record relations.	Imports <code>transcript_of_signature</code> , <code>transcript_valid</code> , and <code>verify_internal_sign_internal</code> . Used by transcript logging games and NMA lifts.
<code>signing_record_relation_sound</code> , <code>signing_record_log_sound_cons</code> , <code>signing_record_log_sound_transcripts_valid</code> , <code>signing_record_queries_cons</code> , <code>signing_record_transcripts_cons</code>	1792–1838. Machine-checked facts.	Sound record logs imply simulated signatures and valid transcript lists; cons lemmas expose list structure.	Imports list induction and transcript predicates. Used by HopGames transcript log invariants and query-budget bookkeeping.

rejection_bound_ parts_nonnegative, rejection_sampling_ bound_obligation_holds	1840–1854. Machine- checked facts relying on imports.	Shows the named rejection/Fiat-Shamir-with- aborts loss terms are nonnega- tive.	Imports nonnega- tivity lemmas from HAETAE_Reductions.ec . This proves nonnegativity only; smallness and cryp- tographic interpretation remain outside this file.
signing_attempt_state_ of_sample_pair_*E, signing_attempt_ state_of_sample_pair_ *consistent, signing_ attempt_state_of_sample_ pair_field_accepts, signing_attempt_state_ of_sample_pair_valid_ signature_accepts, signing_attempt_state_ of_sample_pair_accepts_ current	1856–2073. Machine- checked facts.	Sample-pair attempts have the same deterministic signature fields as the current signer and therefore satisfy field, validity, response, and hint acceptance.	Imports projection equal- ities, sign_internal , formal correctness, and norm lemmas. Used by exact-hyperball support-to- acceptance proofs.
signing_attempt_ state_of_sample_pair_ verification_accepts_ current, signing_attempt_ state_of_sample_pair_ reference_rejection_ accepts, signing_attempt_ state_of_sample_pair_ reference_rejection_no_ abort, signing_attempt_ state_of_sample_pair_ rejection_accepts_ current, signing_attempt_ state_of_sample_pair_ rejection_no_abort_ current	2075–2130. Machine- checked facts.	Sample-pair states pass verifica- tion, reference rejection, full re- jection, and the corresponding no-abort predicates.	Imports field acceptance, reference-rejection accep- tance, and deterministic signer facts. Used directly by exact-hyperball no-abort lemmas.
dexact_hyperball_ signing_attempt_state_ reference_rejection_ accepts, dexact_ hyperball_signing_ attempt_state_reference_ rejection_no_abort, dexact_hyperball_ signing_attempt_state_ rejection_accepts, dexact_hyperball_ signing_attempt_state_ rejection_no_abort, dexact_hyperball_ signing_attempt_state_ rejection_abort_zero	2132–2202. Machine- checked facts with model boundary.	Any state in the current exact- hyperball attempt distribution passes reference and full rejec- tion; full rejection abort proba- bility is zero.	Imports support charac- terization and sample-pair acceptance lemmas. Used by exact-hyperball hop games. This remains a formal-carrier theorem, not a paper-sampler correspon- dence theorem.

1.28 Script companion coverage for HAETAE ROM

`HAETAE_ROM.ec` instantiates the generic random-oracle library with HAETAE-specific query and output types. This file is small but central: all ROM programming, counted-query, and sampler-prequery events refer to its constructors.

Script object family	Mathematical reading	Downstream role
<code>ro_query</code> constructors	<code>MessageHashQuery</code> , <code>ChallengeHashQuery</code> , <code>MatrixExpandQuery</code> , and <code>SamplerExpandQuery</code> partition random-oracle inputs by protocol role.	Bad-event flags and query-budget predicates count only the relevant constructor families, especially challenge and sampler queries.
<code>ro_output</code> tuple	The random-oracle output is a tuple $\text{crh} \times \text{challenge} \times \text{matrix} \times$ <code>random_coins</code> . Projection functions select the component appropriate to the query.	<code>HAETAE_Scheme.ec</code> uses <code>ro_message_hash</code> , <code>ro_challenge_hash</code> , <code>ro_matrix</code> , and <code>ro_signing_coins</code> .
<code>dro_output</code>	The output distribution dispatches by query kind: message hash sam- ples <code>dcrh</code> , challenge hash samples <code>dchallenge</code> , matrix expansion sam- ples <code>dmatrix</code> , and sampler expansion samples <code>drandom_coins</code> .	The <code>FullRO</code> clone uses this as the oracle’s out- put law. Losslessness is needed for game proofs that call the oracle.
<code>cloneimportFullROasHAETAE_ROM</code>	The generic full random oracle is in- stantiated with HAETAE query/out- put types and <code>dro_output</code> .	<code>HopGames</code> and ROM- programming mod- ules reason about <code>HAETAE_ROM.FROM</code> and oracle methods through this clone.
<code>sampler_expand_query_ dro_output_ro_signing_ coins</code>	For sampler-expansion queries, pro- jecting <code>ro_signing_coins</code> from <code>dro_ output</code> has the same law as <code>drandom_ coins</code> .	Used by sampler- freshness lemmas in <code>HAETAE_HopGames.ec</code> to replace a fresh ROM sampler query by the signing-coin distribu- tion.

Boundary: this is a random-oracle interface definition, not a theorem that any concrete SHAKE invocation behaves randomly. That modeling step remains the ROM assumption boundary.

1.28.1 Lemma-ledger companion for HAETAE ROM

Lemma-ledger criticism for HAETAE ROM. The previous companion coverage described the ROM interface, but it was not yet sufficient for a reader to reconstruct the proof obligations attached to the query dictionary. It did not identify which names are query constructors, which

names are output projections, which facts are distribution losslessness lemmas, and which boundary is only the external interpretation of the formal oracle as a random oracle for SHAKE-like calls. This matters because `HAETAE_ROM_Programming.ec`, `HAETAE_Transcript.ec`, and `HAETAE_HopGames.ec` all count or program particular constructors, and a single confused constructor changes the bad-event statement. The ledger below keeps the formal `FullRO` instantiation separate from the external ROM modeling assumption.

Imported-interface and assumption ledger for HAETAE ROM.

EasyCrypt object	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
<code>AllCore</code> , <code>Distr</code> , <code>List</code> , <code>PROM</code>	1. Imported/library fact.	Imports the core logic, probability distributions, lists, and generic programmable random-oracle library.		No	HAETAE-specific premise. The <code>PROM.FullRO</code> interface is the formal oracle machinery later used by <code>HopGames</code> and <code>ROM</code> programming.
<code>HAETAE_Params.ec</code> , <code>HAETAE_Algebra.ec</code> , <code>HAETAE_Distributions.ec</code>	2, 6–8. Imported/library fact.	Imports formal mode values, carrier types, algebraic challenge/matrix constructors, and output distributions.		Used to type queries and outputs and to prove <code>dro_output</code> lossless. These are formal-model imports, not SHAKE security.	
<code>cloneimportFullROasHAETAE_ROM</code>	80–84. Imported/library fact.	Instantiates the generic full random oracle with HAETAE query and output types and <code>dro_output</code> .		The clone supplies <code>HAETAE_ROM.FRO</code> and its state/method vocabulary used by <code>HAETAE_ROM_Programming.ec</code> and <code>HAETAE_HopGames.ec</code> .	
Concrete SHAKE/FIPS-to-ROM interpretation	1–111. External cryptographic assumption.	The file does not prove that SHAKE or any implementation behaves as this formal random oracle.		That modeling step remains outside <code>HAETAE_ROM.ec</code> ; later theorems are ROM theorems unless a separate standard-model hash argument is supplied.	

Definition ledger for HAETAE ROM.

EasyCrypt object(s)	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
<code>ro_query</code> , <code>MessageHashQuery</code> , <code>ChallengeHashQuery</code> , <code>MatrixExpandQuery</code> , <code>SamplerExpandQuery</code>	10–14. Machine-checked fact.	Defines the four disjoint formal ROM input classes: message hash, challenge hash, matrix expansion, and sampler expansion.		Used by <code>HAETAE_Transcript.ec</code> challenge-site definitions, <code>HAETAE_ROM_Programming.ec</code> counted-query predicates, and <code>HopGames</code> sampler/challenge bad events.	

<code>ro_output</code>		15. Machine-checked fact.	Defines a ROM output as a tuple of CRH bytes, challenge, matrix, and signing coins.	Projection discipline is essential: each query class consumes only the component relevant to that query.
<code>message_hash_query,</code> <code>challenge_hash_query,</code> <code>matrix_expand_query,</code> <code>sampler_expand_query</code>		17–24. Machine-checked fact.	Names the canonical constructors used by the scheme and transcript scripts.	Downstream proofs count challenge queries and sampler queries by recognizing these constructors; they are not interchangeable.
<code>ro_crh_zero,</code> <code>challenge_zero,</code> <code>ro_matrix_zero,</code> <code>ro_random_coins_zero,</code> <code>ro_output_zero,</code> <code>ro_output_to_*</code> , <code>ro_output_of_*</code>	<code>ro_</code>	27–45. Machine-checked fact.	Defines zero/default output components, tuple projections, and embeddings of a single component into the full output tuple.	Used to state and prove output-distribution laws. These are formal tuple operations, not hash-domain-separation proofs.
<code>dmatrix_for_query,</code> <code>dro_output</code>		48–57. Machine-checked fact.	Defines the distribution sampled for each query class and embeds it into the appropriate output component.	Imports <code>dcrh</code> , <code>dchallenge</code> , <code>dmatrix</code> , and <code>drandom_coins</code> . This is the output law passed to <code>FullRO</code> .
<code>Oracle</code> , <code>POracle</code>		85–91. Imported/library fact.	Re-exports full and public oracle module-type interfaces from <code>FRO</code> .	Used by <code>HAETAEScheme.ec</code> , <code>HAETAEROMProgramming.ec</code> , and <code>HopGames</code> modules to state what oracle methods are available.
<code>ro_message_hash,</code> <code>challenge_hash,</code> <code>ro_matrix,</code> <code>ro_signing_coins</code>	<code>ro_</code>	93–97. Machine-checked fact.	Names the output projections used after a ROM call.	<code>HAETAEScheme.ec</code> uses these selectors; <code>HopGames</code> relies on their projection equations when replacing sampler ROM calls by distribution facts.

Lemma ledger for HAETAES ROM.

EasyCrypt lemma(s)	Lines / category	Mathematical statement	Local premises, imports, downstream role
<code>matrix_query_distribution_lossless</code>	59–65. Machine-checked fact.	The matrix distribution selected by a $(mode, seed)$ query is lossless.	Imports <code>matrix_distribution_lossless</code> . Used inside the full output losslessness proof.
<code>ro_output_distribution_lossless</code>	67–77. Machine-checked fact.	For every ROM query, <code>dro_outputq</code> is lossless.	Proof splits by query constructor and imports distribution losslessness for CRH, challenge, matrix, and signing coins. Required by the <code>FullRO</code> clone and all oracle-call game proofs.

<code>sampler_expand_query_dro_output_ro_signing_coins</code>	99–107. Machine-checked fact.	Projecting signing coins from the output law of a sampler-expansion query has exactly distribution <code>drandom_coins</code> .	Used by sampler-freshness and sampler-prequery lemmas in HopGames and ROM programming; it is a formal marginal law, not a SHAKE proof.
---	----------------------------------	---	--

1.29 Script companion coverage for HAETAE Scheme

`HAETAE_Scheme.ec` clones the generic signature-game interface and defines the scheme procedures used by `HAETAE_Security.ec`. It is the interface contract between the algebraic model and generic EUF-CMA/NMA games.

Script object family	Mathematical reading	Downstream role
<code>cloneimportSig_ROMasSIG</code>	Instantiates generic signature-game types with HAETAE carriers and the HAETAE random-oracle output distribution.	All <code>SIG.EUF_CMA</code> , <code>SIG.UF_NMA</code> , adversary, and scheme module types used in Security and HopGames come from this clone.
<code>haetae_mode</code>	An abstract operation selecting the parameter mode for the scheme instance.	Final theorems are for the formal scheme at this mode; later premises such as <code>fs_with_aborts_interfaces_readyhaetae_mode</code> are mode-indexed.
<code>HAETAE(H).kg</code>	Samples <code>sd</code> from <code>dseed</code> , derives <code>rhoprime</code> , queries the ROM at <code>matrix_expand_queryhaetae_moderhopprime</code> , and returns <code>keygen_internalhaetae_moderhopprime</code> .	Defines the key-generation behavior used by correctness and all security games.
<code>HAETAE(H).sign</code>	Samples <code>drandom_coins</code> , queries <code>sampler_expand_query</code> , derives the public key from the secret key, queries message hash and challenge hash, and returns <code>sign_internal</code> .	Fixes the query order and query kinds used by ROM programming and bad-event proofs. The signer ignores the challenge output except through the formal <code>sign_internal</code> model.
<code>HAETAE(H).verify</code>	Returns <code>verify_internalhaetae_modepkmtxsig</code> .	Correctness and EUF-CMA success events use this predicate.

Boundary: this file exposes the formal scheme to generic games. It does not prove security; it also does not prove byte-level implementation equivalence. The checked facts about validity and

correctness are imported from `HAETAE_Algebra.ec` and consumed later in `HAETAE_Security.ec`.

1.29.1 Lemma-ledger companion for HAETAE Scheme

Lemma-ledger criticism for HAETAE Scheme. The earlier report explained the scheme at the object-family level, but it did not expose enough proof-script detail to reconstruct the interface boundary. The scheme file contains no lemmas; its proof significance is in the exact clone instantiation and the exact program text of `kg`, `sign`, and `verify`. In particular, the signing procedure fixes the ROM query order that later bad-event arguments rely on. It samples seed coins, queries `sampler_expand_query`, replaces the coins by `ro_signing_coins`, then queries message hash and challenge hash before returning `sign_internal`. The ledger below records that wiring without treating it as a proof of security, implementation equivalence, or SHAKE correctness.

Imported-interface and assumption ledger for HAETAE Scheme.

EasyCrypt object	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
AllCore, Sig_ROM.ec, HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Distributions.ec, HAETAE_ROM.ec	1–10. Imported/library fact.	Imports the generic signature-game skeleton plus the HAETAE formal types, algorithms, distributions, and ROM dictionary.		No local cryptographic premise. The file only wires those imported surfaces into one scheme module.	
cloneimportSig_ROMasSIG	12–20. Imported/library fact.	Instantiates the abstract signature-game interface with HAETAE public keys, secret keys, messages, contexts, signatures, ROM queries, ROM outputs, and <code>dro_output</code> .		All <code>SIG.Correctness</code> , <code>SIG.EUF_CMA</code> , <code>SIG.UF_NMA</code> , adversary, and oracle module types in HopGames and Security refer to this clone.	
Security theorem assumptions introduced locally	1–66. External cryptographic assumption.	None are introduced by <code>HAETAE_Scheme.ec</code> .		The file defines the formal scheme interface. Hardness assumptions, ROM modeling, sampler correspondence, and implementation equivalence enter elsewhere or remain external.	
Direct theorem boundary	1–66. Explanatory prose.	<code>HAETAE_Scheme.ec</code> contains no local lemmas or theorems.		All theorem-level reasoning over <code>HAETAE</code> occurs in <code>HAETAE_Security.ec</code> and <code>HAETAE_HopGames.ec</code> , usually after inlining these procedures.	

Procedure ledger for HAETAE Scheme.

EasyCrypt object(s)	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
---------------------	------------------	-------------------------	-----------	-------------------------------	----------------------

<code>haetae_mode</code>	22. Conditional premise.	An abstract formal mode parameter selects which parameter table the scheme uses.	All top-level bounds are relative to this mode. Later obligations such as <code>fs_with_aborts_interfaces_readyhaetae_mode</code> are mode-indexed.
<code>moduleHAETAE(H: SIG.POracle)</code>	24–65. Machine-checked fact.	Defines a formal signature scheme parameterized by a public random-oracle interface.	This is the concrete scheme argument to <code>SIG.Correctness</code> , <code>SIG.EUF_CMA</code> , and <code>SIG.UF_NMA</code> .
<code>HAETAE(H).kg</code>	25–34. Machine-checked fact.	Samples <code>sd</code> from <code>dseed</code> , derives <code>rhoprime</code> , queries <code>matrix_expand_queryhaetae_moderhoprime</code> , and returns <code>keygen_internalhaetae_moderhoprime</code> .	Imports seed distribution, keygen seed derivation, ROM query constructor, and <code>keygen_internal</code> . Used in correctness and game-equivalence proofs after inlining.
<code>HAETAE(H).sign</code>	36–57. Machine-checked fact.	Samples <code>drandom_coins</code> , queries the sampler ROM site, replaces coins by <code>ro_signing_coins</code> , derives the public key, queries message hash and challenge hash, then returns <code>sign_internal</code> .	This fixes the formal query order and active-signing surface used by <code>HAETAE_ROM_Programming.ec</code> and <code>HAETAE_HopGames.ec</code> . It is not a proof that a concrete implementation makes the same calls.
<code>HAETAE(H).verify</code>	59–64. Machine-checked fact.	Verification is exactly <code>verify_internalhaetae_modepkmtxsig</code> .	Correctness and EUF-CMA games use this verifier. Algebra proves <code>verify_internal_sign_internal</code> ; this file only exposes the call.

1.30 Script companion coverage for HAETAE Events and hash support

Three smaller manifest files complete the next dependency layer: `HAETAE_Events.ec`, `HAETAE_FIPS202.ec`, and `HAETAE_Keccak1600.ec`. They are not final security files, but they matter because the proof imports their definitions and facts.

File	Definitions and checked lemma families	Boundary and downstream role
------	--	------------------------------

HAETAE_Events.ec	Defines <code>keygen_rejection_failure</code> , <code>signing_rejection_failure</code> , <code>verification_rejection_failure</code> , and <code>any_rejection_failure</code> . Lemmas such as <code>keygen_rejection_free_current</code> , <code>signing_rejection_free_current</code> , <code>verification_rejection_free_current</code> , and <code>accepted_transcript_rejection_free_current</code> prove that the current formal signer avoids these event predicates.	Machine-checked event vocabulary. Used by rejection reductions and by the security proof's rejection-free branches. It does not estimate rejection probabilities for a separate implementation.
HAETAE_FIPS202.ec	Defines <code>byte-list</code> <code>fips202_state</code> , SHAKE256 constants, the one-block absorb/padding model, <code>shake256_squeeze</code> , and <code>shake256</code> . Lemmas prove constant equalities and output-size facts.	Imported by <code>HAETAE_Algebra.ec</code> to model seed derivation. It is checked hash plumbing, not a ROM proof and not a full FIPS validation certificate.
HAETAE_Keccak1600.ec	Defines lanes, byte/lane conversion, rho offsets, round constants, theta, rho-pi, chi, iota, <code>keccak_round</code> , <code>keccak_f1600_lanes</code> , and <code>keccak_f1600_bytes</code> . Lemmas prove sizes, bit-level unfoldings, and round-function projection equations.	Imported by <code>HAETAE_FIPS202.ec</code> . These are machine-checked definitions and local consistency lemmas for the formal byte-list permutation. They do not turn the final theorem into a standard-model SHAKE theorem.

For a reimplementaion, these support files should be handled before claiming the algebra layer is reproduced. Their imported facts are dependencies of the formal model, even though the top-level security theorem remains a random-oracle-model theorem.

1.30.1 Lemma-ledger companion for HAETAE Events

Lemma-ledger criticism for HAETAE Events. The previous report put `HAETAE_Events.ec` in a combined support-file table, which was useful for orientation but too weak for reconstructing the event boundary. The file is small, but it names the rejection-failure predicates and proves that the current formal algorithms avoid them. Those facts must not be mistaken for a probability estimate for a paper rejection loop or for an implementation-level no-abort theorem. They are checked boolean facts about `keygen_internal`, `sign_internal`, and the formal norm predicates imported from `HAETAE_Algebra.ec`. Downstream, `HAETAE_Rejection.ec` imports this vocabulary directly; HopGames and Security see it through the rejection layer rather than by expanding these definitions.

Imported-interface and assumption ledger for HAETAE Events.

EasyCrypt object	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
AllCore, HAETAE_Params.ec, HAETAE_Algebra.ec, HAETAE_Params, HAETAE_Algebra	1–7. Import- ed/library fact.	Imports the formal parameter, key/signature validity, norm, keygen, and signer definitions used to state event predicates.		HAETAE_Rejection.ec imports HAETAE_Events.ec directly; HopGames and Security consume the resulting event vocabulary transitively.	
External rejection-sampling estimates	1–57. External cryptographic assumption.	No rejection probability or paper rejection-loop estimate is introduced in this file.		Such estimates are handled by HAETAE_Rejection.ec, HAETAE_HopGames.ec, HAETAE_Security.ec, or remain external conditional premises.	

Definition and lemma ledger for HAETAE Events.

EasyCrypt object(s)	Lines / category	Mathematical paraphrase	statement	Local premises, imports, role	premises, downstream
keygen_rejection_failure	9–10. Machine-checked fact.	Keygen failure is the disjunction of invalid keypair or failed secret-norm predicate.		Imports <code>valid_keypair</code> and <code>secret_norm_ok</code> . Used as named event vocabulary for the rejection layer.	
signing_rejection_failure	12–13. Machine-checked fact.	Signing failure is the disjunction of invalid signature or failed response norm predicate.		Imports <code>valid_signature</code> and <code>response_norm_ok</code> . Downstream rejection proofs can refer to this event without restating the predicate.	
verification_rejection_failure	15–16. Machine-checked fact.	Verification failure is the disjunction of invalid signature or failed verification-norm predicate.		Imports <code>valid_signature</code> and <code>verify_norm_ok</code> . Used only as formal event vocabulary, not as a probability bound.	
any_rejection_failure	18–22. Machine-checked fact.	Combines keygen, signing, and verification rejection failures by disjunction.		This is the strongest combined event predicate exposed locally; later proofs generally reason through HAETAE_Rejection.ec.	
keygen_rejection_free_current	24–30. Machine-checked fact.	The current formal <code>keygen_internal</code> output does not satisfy <code>keygen_rejection_failure</code> .		Proof unfolds current keygen, key validity, secret norm, and mode cases. This is formal-model no-failure, not an implementation theorem.	

<code>signing_rejection_free_current</code>	32–38. Machine-checked fact.	The current formal <code>sign_internal</code> output does not satisfy <code>signing_rejection_failure</code> .	Imports <code>valid_signature_sign_internal</code> and <code>response_norm_ok_current</code> . Used to justify clean current-signing branches.
<code>verification_rejection_free_current</code>	40–46. Machine-checked fact.	The current formal <code>sign_internal</code> output does not satisfy <code>verification_rejection_failure</code> .	Imports <code>valid_signature_sign_internal</code> and <code>verify_norm_ok_current</code> . Used as the verifier-side clean-event fact.
<code>accepted_transcript_rejection_free_current</code>	48–55. Machine-checked fact.	The keygen-plus-sign transcript produced by the current formal algorithms is outside <code>any_rejection_failure</code> .	Depends on the three component rejection-free lemmas. It is the local combined theorem boundary for events; top-level security still relies on later hop/reduction arguments and hardness premises.

1.31 Evidence required for a standalone claim

If a future version of the documentation is advertised as sufficient by itself, it must contain enough evidence for an independent reader to rebuild the proof without opening the repository. At minimum, it must satisfy the following requirements.

Required evidence	Why it is necessary	Present here?
Literal EasyCrypt source for every manifest file	EasyCrypt checks exact syntax, definitions, imports, and proof scripts, not prose descriptions.	No
Exact module restriction sets	Security games rely on separation between adversaries, oracles, simulators, and hidden game state.	Representative only
Full helper-lemma statements and proofs	Top-level theorems depend on local facts whose names alone do not determine their statements or proof obligations.	No
Complete import and library version contract	Proof success can depend on library names, theories, notations, and solver behavior.	Partial
Traceability matrix from prose to checked declarations	A reader needs to know which checked object discharges each mathematical step.	Required, not included in full
Transcript of a successful verification run	The claim is machine-checked only after the checker has accepted the complete source tree.	Build command only

This table is an acceptance gate. The current document intentionally leaves several entries as “No” or “Partial”, so it must not be cited as a self-contained proof archive.

1.32 What this report deliberately does not claim

The checked development is precise, but its scope is narrow. Reimplementing the proof from this report does not prove any of the following.

- A standard-model theorem replacing the random oracle by SHAKE.
- Concrete MLWE or Module-SIS bit-security estimates for HAETAE parameter sets.
- Constant-time implementation security.
- Byte-level parsing correctness for all concrete encodings.
- A machine-checked equivalence theorem

$$\text{Spec}_{260204} \equiv \text{ECModel}.$$

- A cost-semantics theorem bounding the running time of the reductions.

Those are boundary obligations. The report explains where each boundary enters so that a reimplementaion does not silently promote the checked theorem into a stronger end-to-end certification claim.

1.33 Repository paths

The proof gate lives under the project directory `haetae-security`. The active proof-management surface is:

```
provable-security/  
  proof-files.txt  
  verify-provable-security.sh  
  easycrypt/  
  logs/
```

The manifest source files also exist at the project root. The current proof gate compiles the copies under `provable-security/easycrypt/`. A reimplementaion may keep only one copy, but then the verification script and import paths must be adjusted consistently.

Chapter 2

Verification Workflow

2.1 Prerequisites

Install an EasyCrypt version capable of compiling the current scripts and make sure the command is visible in the shell:

```
easycrypt --version
```

The proof also imports supporting EasyCrypt libraries from **kyber-security**. A fresh reimplementation should preserve this include path until all dependencies have been audited and replaced.

2.2 The manifest

The manifest is the list of files that define the proof gate. It should contain exactly these entries, in dependency-compatible order:

```
HAETAE_Security.ec  
Sig_ROM.ec  
HAETAE_HopGames.ec  
HAETAE_Reductions.ec  
HAETAE_ROM.ec  
HAETAE_ROM_Programming.ec  
HAETAE_Scheme.ec  
HAETAE_Transcript.ec  
HAETAE_Events.ec  
HAETAE_Assumptions.ec  
HAETAE_Distributions.ec  
HAETAE_Rejection.ec  
HAETAE_Algebra.ec  
HAETAE_Params.ec  
HAETAE_FIPS202.ec  
HAETAE_Keccak1600.ec
```

Generated known-answer-test files, archival experiments, and unrelated implementation validation artifacts should not be added to this gate merely because they compile. The manifest is a proof-surface boundary, not a list of all EasyCrypt files in the repository.

2.3 Running the proof gate

From the project root, run:

```
sh provable-security/verify-provable-security.sh
```

The script compiles each manifest entry with include paths equivalent to:

```
easycrypt compile provable-security/easycrypt/<file>.ec \  
-I provable-security/easycrypt \  
-I kyber-security
```

The run is successful only if the final line is:

```
RESULT: PASS
```

2.4 Independent audit commands

A reimplementation should also provide small audit commands. The following checks are not substitutes for EasyCrypt, but they catch common proof-surface drift.

```
# Count declarations and confirm that the inventory is stable.  
./dissertation/etc/count-easycrypt-declarations.sh  
  
# Confirm that no local proof file uses unchecked proof holes.  
rg -n "\b(axiom|admit\.|sorry\.)\b" provable-security/easycrypt *.ec  
  
# Confirm that manifest copies match root copies, if both are retained.  
rg -v "^#|^$" provable-security/proof-files.txt |  
while read f; do  
  cmp -s "$f" "provable-security/easycrypt/$f" || echo "$f_differs"  
done
```

For the current artifact, the declaration inventory is:

```
TOTAL,22046,63,678,113,1088,0,105
```

The columns are file lines, type declarations, operation declarations, module declarations, lemma declarations, axiom declarations, and Hoare/equivalence judgments.

Chapter 3

Mathematical Theorem Boundary

3.1 Security game

The informal target is the random-oracle EUF-CMA experiment. An adversary A receives a public key, random-oracle access, and signing-oracle access. It outputs (m^*, ctx^*, σ^*) . It wins when

$$\text{Verify}^H(pk, m^*, ctx^*, \sigma^*) = 1$$

and (m^*, ctx^*) was not previously asked of the signing oracle. The formal checked advantage is the success probability of

$$\text{SIG.EUF_CMA}(H, \text{HAETAE}, A) . \text{main}.$$

3.2 Final checked implication

The strongest checked theorem emphasized by the current proof surface is the following EasyCrypt implication.

Theorem 3.1 (Checked counted-ROM implication, informal rendering). *For any EasyCrypt memory parameter, if the final exact-hyperball NMA hop satisfies*

$$\begin{aligned} & \Pr[\text{UF_NMA}(\text{ROMInternalTranscriptPaperSimAsNMA}(A, \text{ROExactHyperballPaperSimSampler}))] \\ & + \text{rejection_sampling_loss_term} \leq \Pr[\text{MLWE_Game}(B_{\text{MLWE}})] + \Pr[\text{BimodalSelfTargetMSIS_Game}(B_{\text{Bimodal}})] \end{aligned}$$

and if

$$\Pr[\text{MLWE_Game}(B_{\text{MLWE}})] \leq \text{mlwe_hardness_term},$$

and

$$\Pr[\text{ModuleSIS_Game}(\text{BimodalMSIS_As_ModuleSIS}(B_{\text{BimodalSelfTargetMSIS}}))] \leq \text{module_sis_hardness_term},$$

then

$$\Pr[\text{SIG.EUF_CMA}(H, \text{HAETAE}, A) . \text{main}] \leq \text{haetae_euf_counted_rom_bound}.$$

The EasyCrypt theorem name is:

euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal

3.3 Why the theorem is conditional

The modules B_{MLWE} and $B_{\text{BimodalSelfTargetMSIS}}$ are declared reduction modules in the final theorem surface. The top-level composition lemma does not by itself construct these solvers from A . It composes explicitly stated probability premises. A paper proof may describe constructed reductions, but a faithful description of this checked theorem must say that the final composition consumes declared reduction modules and antecedents.

3.4 The counted-ROM bound

The reduction file defines:

```

op mlwe_hardness_term : real.
op module_sis_hardness_term : real.
op bimodal_to_msis_reduction_loss_term : real = 0%r.

op signature_query_count : real = 2%r.
op hash_query_count : real = 2%r.
op challenge_support_cardinality_lower_bound : real =
  288230376151711744%r.

op counted_rom_collision_term =
  (rom_total_query_budget * rom_total_query_budget) /
    challenge_support_cardinality_lower_bound.

op counted_rom_prequery_term =
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) /
    challenge_support_cardinality_lower_bound.

op counted_rom_min_entropy_term =
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) /
    challenge_support_cardinality_lower_bound.

```

Thus

$$q_H = 2, \quad q_S = 2, \quad N = 288230376151711744 = 2^{58}.$$

The ROM loss is

$$\frac{(2 + 2 + 1)^2}{2^{58}} + \frac{2(2 + 1)}{2^{58}} + \frac{2(2 + 1)}{2^{58}} = \frac{25 + 6 + 6}{2^{58}} = \frac{37}{2^{58}}.$$

3.5 The vacuity warning

The proof also defines a paper-style rejection/Fiat-Shamir-with-aborts loss:

```

op rejection_sampling_loss_term =
  fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.

```

The checked constants imply:

```

lemma fs_with_aborts_min_entropy_termE :
  fs_with_aborts_min_entropy_term = 12%r.

```

```
lemma rejection_sampling_loss_term_ge1 :  
  1%r <= rejection_sampling_loss_term.
```

Therefore a premise of the form

$$\Pr[G] + \text{rejection_sampling_loss_term} \leq \Pr[\text{MLWE}] + \Pr[\text{BimodalSelfTargetMSIS}]$$

is not a useful concrete-security premise in the current constants. The right side is at most 2, while the extra loss term is already at least 1 and has a min-entropy component equal to 12. The checked theorem is still a valid logical implication, but it should be cited with this warning attached.

Chapter 4

Pen-and-Paper Reimplementation

4.1 Proof skeleton

A paper proof reconstructed from the checked files should be written as the following chain.

1. Define the real EUF-CMA game G_0 .
2. Replace real signing by a simulator while preserving the visible distribution or charging an explicit signing-hop loss.
3. Expose transcript state so that commitments, challenges, responses, and oracle queries become explicit.
4. Move from adaptive signing access to an NMA-style internal transcript surface.
5. Program the random oracle and charge collision, prequery, and min-entropy bad events.
6. Replace structural or paper sampling views by the exact hyperball sampler used by the formal model.
7. Apply special soundness to compatible accepting transcript pairs.
8. Convert the extracted bimodal self-target relation to the Module-SIS hardness game.
9. Compose MLWE, Module-SIS, bimodal-to-MSIS, and counted-ROM terms.

Every step must be labelled either as an exact equivalence, a bad-event bound, a sampling-distance bound, a reduction step, or a real-arithmetic normalization. Do not add rejection, exact-sampling, or active-signing losses as final summands if the EasyCrypt theorem already internalizes them into hop premises. Doing so double-counts proof obligations.

4.2 Game notation

Use the following names in the paper proof.

Paper object	EasyCrypt counterpart
Real EUF-CMA game	<code>SIG.EUF_CMA(H, HAETAE, A).main.</code>

Paper object	EasyCrypt counterpart
No-message game	<code>SIG.UF_NMA(H, HAETAE, A).main.</code>
Random oracle	HAETAE_RO.FRO cloned from the generic full random oracle.
Signing oracle	Modules named <code>O.sign</code> inside transcript and budgeted NMA games.
MLWE reduction game	<code>MLWE_Game(Bmlwe).main.</code>
Bimodal self-target MSIS game	<code>BimodalSelfTargetMSIS_Game(Bbimodal).main.</code>
Module-SIS game	<code>ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main.</code>

4.3 Bad events

The random-oracle and sampler bad events should be separated.

- The counted-ROM collision term accounts for collisions among programmed or observed ROM sites.
- The counted-ROM prequery term accounts for programmed challenge sites that were already queried in the counted transcript ledger.
- The counted-ROM min-entropy term accounts for the challenge entropy bad event in the counted transcript ledger.
- The sampler prequery event `sampler_bad_prequery` is a local active-signing event. It is bounded by a different expression involving the adversary hash log and the sampler self-log.

The paper proof must not identify `sampler_bad_prequery` with `counted_rom_prequery_term`. They are related proof ideas, but they are different events in the checked source.

4.4 Special soundness and extraction

The extraction step is not a conclusion from a single valid transcript. It requires a forking pair: two accepting transcripts sharing the appropriate public data and carrying distinct challenges. The paper proof should state the obligation as:

$$\text{ValidForkingPair}(tr_1, tr_2) \implies \text{Solve}_{\text{BimodalSelfTargetMSIS/Module-SIS}}(tr_1, tr_2).$$

The checked anchors are:

```

extract_bimodal_forking_some
extract_bimodal_forking_nonempty
valid_forking_pair_public_equations
bimodal_to_module_sis_valid
bimodal_solver_lift_exact
bimodal_mode_solver_lifted_distribution_exact
bimodal_msis_as_module_sis_exact

```

Chapter 5

EasyCrypt Reimplementation Plan

5.1 Development order

Reimplement the files from leaves to root. Do not begin with the final theorem. Use the following order.

1. HAETAE_Params.ec
2. HAETAE_Keccak1600.ec
3. HAETAE_FIPS202.ec
4. HAETAE_Algebra.ec
5. HAETAE_Distributions.ec
6. HAETAE_Events.ec
7. HAETAE_Assumptions.ec
8. Sig_ROM.ec
9. HAETAE_ROM.ec
10. HAETAE_Scheme.ec
11. HAETAE_Transcript.ec
12. HAETAE_Reductions.ec
13. HAETAE_Rejection.ec
14. HAETAE_ROM_Programming.ec
15. HAETAE_HopGames.ec
16. HAETAE_Security.ec

The manifest compiles root-first because each file is checked independently with imports resolved by EasyCrypt. Human reimplementation should proceed leaf-first because each later file uses definitions from earlier ones.

5.2 Global proof idioms

Use four recurring EasyCrypt idioms.

Typed interfaces. Use module types for adversaries, schemes, oracles, and hardness solvers. This prevents games from depending on implementation details that should remain abstract.

State exposure. When a paper proof says “the simulator records queries”, make a module with explicit state variables for logs, counters, and bad flags.

Local lemmas before global theorem. First prove preservation lemmas, support lemmas, losslessness facts, and pointwise arithmetic bounds. Then assemble them in top-level security lemmas.

No silent axioms. Do not use `axiom`, `admit..`, or `sorry..` If a fact is an external assumption, represent it as a declared operation or module interface and make the final theorem conditional on it.

5.3 Placeholder convention

The snippets in the next chapter are intentionally schematic where the full source would obscure the proof architecture. In particular, code fragments containing `...` are not EasyCrypt programs. They mark locations where the complete project source must be consulted or rederived. This convention is a guardrail against a false kind of self-containment: copying the snippets alone will not produce a compiling proof, and the report should not be cited as a literal replacement for the checked `.ec` files.

For a genuinely standalone manual, every occurrence of `...` would have to be expanded into exact EasyCrypt source, every helper lemma would need its full statement and proof, and every module declaration would need its exact restriction set. That is a different artifact from this roadmap.

Before claiming that a documentation-derived proof tree is machine checked, run a placeholder audit over the generated files. The audit is deliberately simple: any remaining ellipsis or proof-hole marker must be explained as a comment or removed from the proof surface.

```
rg -n "\.\.|TODO|FIXME|admit\.|sorry\.|\baxiom\b" \
provable-security/easycrypt
```

The expected result for the active proof surface is no unchecked proof hole. A hit on `...` in executable EasyCrypt means the roadmap has not yet been expanded into source.

Chapter 6

File-by-File Blueprint

6.1 HAETAE_Params.ec

Define the fixed HAETAE parameter vocabulary.

```
type mode = [ Mode2 | Mode3 | Mode5 ].

op n : int = 256.
op q : int = 64513.
op dq : int = 2 * q.

op mode_k (md : mode) : int = ...
op mode_l (md : mode) : int = ...
op mode_tau (md : mode) : int = ...
```

Required lemmas:

- positivity of byte sizes, n , q , and all mode-dependent bounds;
- $dq = 2q = 129026$;
- $\tau \leq n$ for every mode;
- positivity of public-key and signature-size constants.

These lemmas are simple case splits, but they are essential because later probability and support facts need positive denominators and nonempty domains.

6.2 HAETAE_Keccak1600.ec and HAETAE_FIPS202.ec

These files provide modeled hash-interface support. They do not replace the random oracle in the security theorem. A reimplementation should define:

- byte and state list types;
- state-size, rate, capacity, domain-separator, and padding constants;
- deterministic absorb, squeeze, and permutation operations;
- length and well-formedness lemmas needed by the algebra layer.

Keep the cryptographic boundary explicit: compiling these files does not prove standard-model SHAKE security.

6.3 HAETAE_Algebra.ec

This is the algebraic model of HAETAE objects. Define carrier types for:

- bytes, seeds, CRHs, random coins, messages, and contexts;
- coefficients, polynomials, challenges, polynomial vectors, and matrices;
- public keys, secret keys, and signatures.

Then define total operations for:

- modular coefficient arithmetic;
- polynomial and vector operations;
- key generation internals;
- signing internals;
- verification internals;
- transcript commitment and challenge components;
- norm and validity predicates.

The most important final facts are:

```
lemma verify_internal_sign_internal ...
lemma valid_signature_sign_internal ...
lemma secret_norm_ok_current ...
lemma response_norm_ok_current ...
lemma verify_norm_ok_current ...
```

The proof model stores squared norm thresholds. The validity reading is:

$$\|z\|^2 \leq B^2 \quad \text{implemented as} \quad \text{normsq}(z) \leq \text{mode_b*sq}(md).$$

Do not replace this by a strict inequality unless the corresponding rejection predicate actually uses a strict bound.

6.4 HAETAE_Distributions.ec

Define the distributions used by key generation, signing, challenges, and structural samplers.

Essential distribution properties:

- losslessness of byte, seed, CRH, random-coin, coefficient, polynomial, vector, matrix, and challenge distributions;
- support facts showing sampled objects satisfy the predicates expected by the algebra layer;

- projection facts connecting structural signing samples to response, challenge, and randomness components.

Every sampler used by a program that should terminate with probability one must have a corresponding losslessness lemma or a local proof of losslessness.

6.5 HAETAE_Events.ec

Define named event predicates rather than spelling conditions repeatedly. Events should include rejection, bad prequery, bad min-entropy, collision, and validity conditions needed by the game hops.

This file should contain no heavy proof. Its job is vocabulary alignment: later files can reason about the same event name instead of duplicating Boolean formulas.

6.6 HAETAE_Assumptions.ec

Introduce the external hardness games and reduction interfaces.

```
module type MLWE_Reduction = {
  proc main() : bool
}.

module type BimodalSelfTargetMSIS_Reduction = {
  proc main() : bool
}.

module type ModuleSIS_Reduction = {
  proc main() : bool
}.
```

Define game wrappers that call these modules:

```
module MLWE_Game(B : MLWE_Reduction) = {
  proc main() : bool = {
    var b : bool;
    b <@ B.main();
    return b;
  }
}.
```

Also define the bimodal-to-Module-SIS adapter and prove exactness lemmas:

```
bimodal_to_module_sis_valid
bimodal_solver_lift_exact
bimodal_mode_solver_lifted_distribution_exact
bimodal_msis_as_module_sis_exact
```

These are not concrete cryptanalytic estimates. They are formal assumption surfaces used by the final reduction theorem.

6.7 Sig_ROM.ec

Define the generic random-oracle signature game independent of HAETAE.

```
type pkey.  
type skey.  
type message.  
type context.  
type signature.  
type ro_query.  
type ro_output.  
  
op [lossless] dro_output : ro_query -> ro_output distr.
```

Define module types:

```
module type Oracle = { include FRO [init, get] }.  
module type POracle = { include FRO [get] }.  
module type Scheme(O : POracle) = { ... }.  
module type SignOracle = { proc sign(m : message, ctx : context) : signature }.  
module type Adversary(H : POracle, O : SignOracle) = { ... }.  
module type NMA_Adversary(H : POracle) = { ... }.
```

Define:

- correctness game;
- EUF-CMA game;
- UF-NMA game;
- NMA-as-CMA adapter;
- freshness predicates for signed message-context pairs.

The generic game should not contain HAETAE-specific facts. It must be reusable.

6.8 HAETAE_ROM.ec

Instantiate random-oracle query and output types for HAETAE. Use typed query constructors for distinct hash domains:

- matrix expansion;
- message hashing;
- sampler expansion;
- challenge hashing.

The reason for typed constructors is domain separation. A proof should not be able to confuse a sampler-expansion query with a challenge query.

6.9 HAETAE__Scheme.ec

Clone the generic signature interface and instantiate it with HAETAE types and the HAETAE random-oracle output distribution:

```
clone import Sig_ROM as SIG with
  type pkey <- pkey,
  type skey <- skey,
  ...
op dro_output <- dro_output.
```

Define the formal scheme module:

```
module HAETAE(H : SIG.POracle) = {
  proc kg() : pkey * skey = { ... }
  proc sign(sk : skey, m : message, ctx : context) : signature = { ... }
  proc verify(pk : pkey, m : message, ctx : context,
    sig : signature) : bool = { ... }
}.
```

The current wrapper makes ROM calls visible around deterministic internal operations. In particular, signing samples typed random coins, asks the sampler ROM query, interprets the output as signing coins, computes message and challenge ROM queries, and finally calls `sign_internal`.

6.10 HAETAE__Transcript.ec

Define transcript records and validity predicates. A transcript should contain at least the public key, message, context, signature, commitment components, challenge query, and challenge value needed by extraction.

Key proof obligations:

```
transcript_valid_signature_challenge_query
transcript_valid_signature_challenge_matches
transcript_valid_signature_challengeE
extract_bimodal_forking_some
extract_bimodal_forking_nonempty
valid_forking_pair_public_equations
```

The extraction lemmas should require a valid forking pair. Do not prove or state special soundness from one accepting transcript alone.

6.11 HAETAE__Reductions.ec

Define all real-valued loss terms and prove their arithmetic properties.

Required groups:

- abstract hardness terms: `mlwe_hardness_term`, `module_sis_hardness_term`;
- bimodal-to-MSIS loss slot, currently zero;
- fixed query counts $q_H = q_S = 2$;

- challenge support lower bound $N = 2^{58}$;
- counted-ROM collision, prequery, and min-entropy terms;
- paper-style rejection/Fiat-Shamir-with-aborts terms;
- grouped final bounds `haetae_euf_bound` and `haetae_euf_counted_rom_bound`.

Arithmetic lemmas to reprove:

```
haetae_euf_counted_rom_bound_groupedE
counted_rom_collision_termE
counted_rom_prequery_termE
counted_rom_min_entropy_termE
counted_rom_programming_loss_termE
counted_rom_programming_loss_term_concreteE
counted_rom_programming_loss_term_subunit
fs_with_aborts_min_entropy_termE
rejection_sampling_loss_term_ge1
bimodal_to_msis_reduction_loss_termE
```

This file is where most accidental theorem inflation can be caught. The final counted-ROM bound is not the same as the older paper-style rejection bound.

6.12 HAETAE_Rejection.ec

Define signing-attempt state, rejection predicates, and the relationship between reference rejection and the modeled signing state.

Important predicates:

- `signing_attempt_state_valid_signature_accepts`;
- `signing_attempt_state_reject1_accepts`;
- `signing_attempt_state_reject2_accepts`;
- `signing_attempt_state_reference_rejection_accepts`;
- `signing_attempt_state_rejection_accepts`.

Important obligations:

- bound rejection probabilities by named terms;
- show exact-hyperball signing attempt states satisfy the reference rejection predicates;
- prove zero-abort facts for the exact paths currently used by the model;
- expose coarse fallback bounds honestly when they are used.

6.13 HAETAE_ROM_Programming.ec

Define the counted random-oracle programming surface.

This file should model:

- lazy ROM maps;
- observed query logs;
- programmed query sites;
- collision and prequery bad events;
- preservation of good state under oracle operations;
- probability bounds for programming failures.

Use explicit counters and support lower bounds. Do not write a prose “fresh with high probability” argument without a program variable tracking the queries that make the statement meaningful.

6.14 HAETAE_HopGames.ec

This is the largest file because it contains the concrete hybrid machinery. Reimplement it in layers.

Layer 1: concrete lazy-ROM laws

Prove sampler expansion freshness:

```
concrete_lazy_rom_sampler_expand_query_fresh_le
concrete_lazy_rom_sampler_expand_query_fresh_eq
```

Mathematical reading: if `sampler_expand_query( $r$ )` is not in the lazy-ROM table, then reading the ROM at that point yields a signing-coin distribution bounded by the ideal coin distribution.

Layer 2: signing-oracle clean invariants

Expose the active signing oracle state and prove that clean conditions are preserved:

```
concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh
```

The clean condition should include sampler-ROM coverage, absence of the current sampler prequery bad event, budget discipline, public-key consistency, and secret-key consistency.

Layer 3: one-call lifting

Prove the clean one-call bound:

```
concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface
```

Its mathematical shape is:

$$\begin{aligned} & \Pr[O_{\text{attempt}}.\text{sign} : p(\text{res}) \wedge \neg \text{sampler_bad_prequery}] \\ & \leq \Pr[O_{\text{exact}}.\text{sign} : p(\text{res})] + \text{rejection_sampling_loss_term}. \end{aligned}$$

Layer 4: NMA and transcript game lifting

Lift the one-call facts through NMA-style transcript games:

```
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting  
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
```

This is where the current development still contains coarse fallback bounds. A faithful reimplementation should keep those fallbacks visible and should not describe the final result as non-vacuous.

6.15 HAETAE_Security.ec

Assemble the final theorem family.

First prove correctness:

```
op correctness_obligation = ...  
lemma correctness_obligation_holds : correctness_obligation.  
lemma correctness_perfect &m :  
  Pr[SIG.Correctness(H, HAETAE, A).main() @ &m : res] = 0%r.
```

Then assemble progressively stronger security lemmas:

- generic EUF-CMA security from hop interfaces;
- ROM-transcript security;
- bimodal-to-Module-SIS lift;
- counted-ROM grouped bound;
- paper-sampling and exact-hyperball bridges;
- final checked counted-ROM implication.

The final theorem should look like:

```
lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &m :  
  Pr[SIG.UF_NMA(H, HAETAE,  
    ROMInternalTranscriptPaperSimAsNMA  
    (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +  
    rejection_sampling_loss_term <=  
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] +  
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>  
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>  
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()  
    @ &m : res] <= module_sis_hardness_term =>  
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <=  
    haetae_euf_counted_rom_bound.
```

The proof should be mostly composition: apply the previous bridge lemma, pass the hop premise, pass the MLWE premise, and pass the Module-SIS premise.

Chapter 7

Reimplementation Checklists

7.1 Pen-and-paper checklist

Before writing EasyCrypt, verify the paper proof has these properties.

1. The starting game is the same EUF-CMA game later encoded in `Sig_ROM.ec`.
2. The source game has no syntactic query cap; fixed counts belong to the wrapper and bound ledger.
3. Every hop is classified as exact, bad-event, sampling-distance, or reduction.
4. Random-oracle programming freshness is stated in terms of explicit query sets.
5. Sampler-prequery events are separated from counted-ROM challenge prequery events.
6. Forking/extraction uses two compatible accepting transcripts with distinct challenges.
7. MLWE and Module-SIS hardness are external assumptions represented by abstract games.
8. The final counted-ROM bound is not mixed with the older `haetae_euf_bound` rejection ledger.
9. The final theorem is marked conditional and carries the vacuity warning.

7.2 EasyCrypt checklist

Before accepting a reimplementation, run these checks.

1. The reproduction mode is declared: audit mode with companion sources, literal standalone mode with full source in the document, or independent reimplementation mode with a traceability matrix.
2. If `.ec` scripts are supplied in script companion mode, every security-relevant script block has a corresponding mathematical explanation in the report.
3. The script-companion coverage ledger is complete: no manifest file, security-critical declaration, nontrivial lemma, tactic block, module restriction, or external assumption remains without a prose explanation.
4. Every manifest file compiles.

5. There are no `axiom`, `admit.`, or `sorry.` tokens in the proof surface.
6. There are no executable `...`, `TODO`, or `FIXME` placeholders in the proof surface.
7. Each declared module in `HAETAESecurity.ec` has the right restriction set, so the adversary cannot call hidden game internals.
8. All distributions sampled by lossless procedures have losslessness lemmas.
9. All denominators in real bounds have positivity lemmas.
10. The counted-ROM concrete arithmetic rewrites to $37/2^{58}$.
11. The rejection min-entropy term rewrites to 12.
12. The final theorem consumes declared reduction modules and explicit premises rather than pretending to construct all solvers inside the final composition.
13. Every theorem, loss term, and declared module used by the final theorem is traceable either to the companion source or to a checked object in the new reimplementation.

7.3 Module restriction-set caveat

Module restriction sets are part of the proof, not formatting. They prevent an adversary or oracle module from calling game internals that should be hidden. The checklist item above therefore requires exact source-level comparison, not a verbal approximation.

For example, the final mechanized ROM-transcript/NMA section begins with declarations of this form:

```
section MechanizedROMTranscriptConstructedNMALift.

declare module H <: SIG.Oracle {-SIG.EUF_CMA,
    -EUF_CMA_InternalTranscriptSign,
    -EUF_CMA_ROMInternalTranscriptSign,
    -ROMInternalTranscriptAsNMA,
    -ROMInternalTranscriptPublicSimAsNMA,
    -ROMInternalTranscriptPaperSimAsNMA,
    -ROMInternalTranscriptBudgetedPaperSimAsNMA,
    -ROMPaperSimSigningSampler,
    -RealSigningPaperSimSampler,
    -ROSigningAttemptPaperSimSampler,
    -HAETAERejctionPaperSimSampler,
    -ExactHyperballHAETAERejctionPaperSimSampler,
    -ExactHyperballPaperSimSampler,
    -ROExactHyperballPaperSimSampler}.

declare module A <: SIG.Adversary {-H, -HAETAESecurity, -SIG.EUF_CMA,
    -EUF_CMA_InternalTranscriptSign,
    -EUF_CMA_ROMInternalTranscriptSign,
    -ROMInternalTranscriptAsNMA,
    -ROMInternalTranscriptPublicSimAsNMA,
    -ROMInternalTranscriptPaperSimAsNMA,
    -ROMInternalTranscriptBudgetedPaperSimAsNMA,
    -ROMPaperSimSigningSampler,
    -RealSigningPaperSimSampler,
```

```

-ROSigningAttemptPaperSimSampler,
-HAETAERejectionPaperSimSampler,
-ExactHyperballHAETAERejectionPaperSimSampler,
-ExactHyperballPaperSimSampler,
-ROExactHyperballPaperSimSampler}.
declare module Samp <: PaperSimSigningSampler {-H, -A,
-ROInternalTranscriptPaperSimAsNMA}.
declare module Bmlwe <: MLWE_Reduction.
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.

```

This listing is representative, not exhaustive. A complete reimplementaion must audit every `declare module` line in `HAETAE_Security.ec` and in the imported hop-game files. Omitting one restriction can turn a meaningful game proof into a circular proof in which an adversary has access to hidden state or procedures.

7.4 Common failure modes

Overstating the theorem. Do not write “the EasyCrypt proof proves HAETAE EUF-CMA security” without the modifiers “formal model”, “random oracle”, “conditional”, and “fixed ledger”.

Counting a loss twice. If a loss is already internalized into a hop premise or bridge lemma, do not add it as a separate final summand.

Erasing the query-bound boundary. The constants $q_H = q_S = 2$ are fixed in the current ledger. They are not a symbolic arbitrary-query theorem.

Confusing SHAKE support with ROM security. `HAETAE_FIPS202.ec` and `HAETAE_Keccak1600.ec` are modeled hash support files. They do not prove that SHAKE instantiates the random oracle.

Using proof holes. If a fact is not proved, make it a premise or an explicitly documented assumption. Do not hide it behind an axiom in the manifest proof surface.

Chapter 8

Minimal Build and Verification Artifacts

8.1 Verification script template

A reimplementaion can use this shell skeleton.

```
#!/usr/bin/env sh
set -eu

EASYCRYPT=${EASYCRYPT:-easycrypt}
ROOT=$(pwd)
PROOF_DIR="$ROOT/provable-security/easycrypt"
MANIFEST="$ROOT/provable-security/proof-files.txt"
LOG_DIR="$ROOT/provable-security/logs"

mkdir -p "$LOG_DIR"

failed=0
while IFS= read -r file; do
  case "$file" in
    ""|\#*) continue ;;
    esac

    echo "===_$_file_"
    if "$EASYCRYPT" compile "$PROOF_DIR/$file" \
      -I "$PROOF_DIR" \
      -I "$ROOT/kyber-security" \
      > "$LOG_DIR/$file.log" 2>&1; then
      echo "PASS_$_file_"
    else
      echo "FAIL_$_file_"
      failed=1
    fi
  done < "$MANIFEST"

if [ "$failed" -eq 0 ]; then
  echo "RESULT:_PASS"
else
```

```
echo "RESULT: FAIL"
exit 1
fi
```

8.2 Documentation build

This report can be built from the `report/` directory with:

```
make
```

or directly with:

```
pdflatex -interaction=nonstopmode -halt-on-error \
  haetae-provable-security-reimplementation-guide
pdflatex -interaction=nonstopmode -halt-on-error \
  haetae-provable-security-reimplementation-guide
```

Chapter 9

Final Acceptance Criteria

A reimplementaion should not be considered complete until all criteria below hold.

Checkpoint 9.1. The reimplementaion is checked against the companion EasyCrypt source files or against an independently expanded document that contains complete source, full lemma statements, proof scripts, and exact module restriction sets. This report alone is not treated as sufficient source material.

Checkpoint 9.2. The report user records which reproduction mode is being claimed. A claim of literal standalone reproduction is rejected unless the documentation has been expanded beyond this roadmap into a complete source-and-proof archive.

Checkpoint 9.3. Every schematic snippet copied from this report has been replaced by checked EasyCrypt source. The active proof surface contains no executable ellipsis, `TODO`, `FIXME`, `admit.`, `sorry.`, or hidden `axiom`.

Checkpoint 9.4. The reimplementaion includes a traceability matrix mapping each final-theorem dependency to either an exact companion-source declaration or an independently checked replacement.

Checkpoint 9.5. The manifest-driven EasyCrypt proof gate terminates with `RESULT:PASS`.

Checkpoint 9.6. The declaration inventory has no axioms and no admitted proofs in the active proof surface.

Checkpoint 9.7. The final theorem statement has the same logical shape as `euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal`.

Checkpoint 9.8. The documentation states that the theorem is conditional, fixed-ledger, formal-model, and random-oracle-model.

Checkpoint 9.9. The documentation distinguishes understanding the checked formal-model theorem from understanding or certifying end-to-end security of the HAETAE digital signature algorithm.

Checkpoint 9.10. The documentation gives a clear negative answer to any claim that the report alone suffices both to understand HAETAE algorithm security and to reproduce the machine-checked EasyCrypt verification.

Checkpoint 9.11. Before any completed script-companion claim is made, the documentation is complete enough for mathematicians and cryptologists to understand the mathematical content of the provided `.ec` scripts from the report: declarations, state variables, module restrictions, lemma statements, proof-script blocks, tactic roles, assumptions, and global security use are all explained.

Checkpoint 9.12. Before any completed script-companion claim is made, the script-companion coverage ledger has no open rows. Each manifest file and each security-relevant declaration, module, lemma, proof block, tactic pattern, module restriction, and external assumption is linked to a mathematical explanation in the report.

Checkpoint 9.13. The documentation states that the current final-hop premise is not a non-vacuous concrete-security premise because of `rejection_sampling_loss_term`.

Checkpoint 9.14. Any claim of standalone documentation sufficiency is backed by literal EasyCrypt source, complete proof scripts, exact module restriction sets, full helper-lemma statements, an import/library contract, and a successful verification transcript.

Checkpoint 9.15. The model/specification bridge is recorded as an audited or assumed boundary, not as a checked equivalence theorem.

Checkpoint 9.16. The proof does not claim concrete SHAKE instantiation, implementation correctness, concrete lattice bit security, or reduction runtime bounds.

Chapter 10

Summary

The HAETAE EasyCrypt proof is best reimplemented as a layered proof surface, using this report as the map and the checked EasyCrypt files as the authoritative source. First define parameters, algebra, distributions, events, and assumption games. Then define generic ROM signature games and instantiate them with the HAETAE scheme model. Next expose transcript state, rejection state, random-oracle programming state, and budgeted signing-oracle invariants. Finally assemble the top-level theorem by composing checked hop premises, declared hardness bounds, and the counted-ROM arithmetic ledger.

The central discipline is not merely to make EasyCrypt accept a long file. The theorem must say exactly what has been proved. For the current artifact, that is a conditional probability implication for the formal HAETAE random-oracle model, with a fixed counted-ROM ledger and an explicit vacuity warning on the final rejection-sampling premise. When the document-only question is asked, the answer is no: this report is not, by itself, a proof of deployed algorithm security, a SHAKE-instantiation theorem, or a standalone machine-checkable archive. A faithful reimplementation preserves that boundary while making every program, event, loss, and reduction interface visible to the checker. The report is therefore an entry point for understanding and auditing the proof; the proof itself remains the checked EasyCrypt source plus its manifest-driven verification run.

If the `.ec` scripts are distributed together with this document, the documentation burden increases. The report must then serve as a self-contained mathematical reading guide to those scripts: every security-relevant definition, game, invariant, module restriction, lemma, tactic block, and assumption must have a documented mathematical interpretation. That still does not make the prose a substitute for machine checking, but it should make the scripts intelligible to a mathematician or cryptologist without relying on unwritten project context. The present version records this obligation and supplies a partial guide; it becomes a completed script companion only after the coverage ledger has no open rows.